

UNIVERSIDAD ALFONSO X EL SABIO
FACULTAD DE BUSINESS & TECH
GRADO EN COMPUTACIÓN E INTELIGENCIA ARTIFICIAL



TRABAJO DE FIN DE GRADO

Orquestación de Equipos de Agentes LLM con Roles Especializados para Resolución Colaborativa de Tareas Complejas de Ingeniería de Software

NICOLÁS TIMONEDA MARTÍNEZ

Junio de 2026

Declaración de autoría y originalidad

Yo, **Nicolás Timoneda Martínez**, con NP 147254, estudiante del Grado en Computación e Inteligencia Artificial de la Universidad Alfonso X el Sabio, declaro bajo mi responsabilidad que el presente Trabajo Fin de Grado, titulado «Orquestación de Equipos de Agentes LLM con Roles Especializados para Resolución Colaborativa de Tareas Complejas de Ingeniería de Software», es un trabajo original realizado por mí, que respeta los derechos de propiedad intelectual de terceros y que se han citado debidamente todas las fuentes utilizadas en su elaboración.

Declaro asimismo que el trabajo no ha sido presentado previamente, en su totalidad o en parte, para la obtención de otro título académico, y que no contiene materiales plagiados ni reproducidos sin la debida atribución a sus autores originales.

El uso de herramientas asistidas por inteligencia artificial en la elaboración del trabajo se documenta de forma explícita en la sección «Declaración sobre el uso de herramientas de inteligencia artificial» de los preliminares de esta memoria.

En Madrid, a Junio de 2026.

Fdo.: Nicolás Timoneda Martínez

Agradecimientos

«La ciencia será siempre una búsqueda, jamás un descubrimiento real. Es un viaje, nunca una llegada.»

— Karl Popper

En agradecimiento a mi familia, sobre todo a mis padres que han hecho esto posible. No tendré vida para compensar todo lo que habéis hecho por mí.

A Sofía, por darme rigor y cabeza cuando más lo he necesitado y por haber estado ahí para apoyarme.

Os quiero.

Resumen

Este Trabajo Fin de Grado diseña, implementa y evalúa empíricamente un sistema multi-agente basado en modelos de lenguaje de gran tamaño para la resolución colaborativa de tareas de generación automática de código Python. El sistema orquesta cinco agentes especializados —Product Manager, Arquitecto, Developer, QA Tester y Code Reviewer— mediante un grafo de estado tipado implementado con LangGraph, con comunicación estructurada a través de artefactos en lugar de conversación libre.

Se comparan tres configuraciones incrementales: un baseline monolítico con un único LLM, un pipeline secuencial de cinco roles, y un pipeline con un ciclo iterativo de auto-revisión entre el Code Reviewer y el Developer. La evaluación se realiza sobre el benchmark HumanEval (164 problemas) con tres réplicas indexadas por semilla pseudoaleatoria, lo que permite contrastar las hipótesis del estudio mediante el test pareado de McNemar y mediante intervalos de confianza al 95 % obtenidos por bootstrap percentil. Las métricas reportadas son pass@1, pass@3 (estimador insesgado de Chen et al., 2021), tasa media de superación de pruebas, coste en tokens y latencia. Variantes de ablación de rol quedan implementadas en el runner para una segunda pasada.

El sistema se implementa en Python con el modelo Qwen 2.5 Coder de 7 mil millones de parámetros servido en local mediante Ollama, sobre un equipo MacBook Air M2. El banco experimental es resumible y fault-tolerant: cada ejecución individual se persiste atómicamente y una corrida interrumpida se retoma con un único comando, sin pérdida de progreso. Un pipeline de análisis post-hoc genera las figuras y tablas de la memoria a partir de los CSV resultantes, incluyendo un borrador automático de la narrativa de hallazgos.

Resultado principal. El experimento no respalda la hipótesis intuitiva de que la especialización por roles mejora la corrección. El baseline alcanza 80,08 % de pass@1; el pipeline secuencial cae a 58,33 % (McNemar $p < 0,0001$) y la configuración con self-reflection a 64,84 % ($p < 0,0001$ frente al baseline; $p = 0,0066$ frente al secuencial, lo que respalda H2 para $r = 1$). Las configuraciones multi-agente consumen alrededor de 40 veces más tokens y 77 veces más latencia por problema para producir peor pass@1, y la frontera de Pareto coste-calidad la ocupa por completo el baseline. La adherencia estructural al protocolo, en cambio, se mantiene cercana al 100 % en las tres configuraciones, lo que confirma la robustez del formato pero no rescata la calidad del contenido generado. El trabajo discute tres explicaciones plausibles para el resultado —propagación de errores entre roles, sobrecarga del prompt de rol en un modelo de 7 B, e inadecuación de HumanEval como banco para evaluar pipelines multi-agente— y los enmarca en una línea crítica reciente de la literatura (Chen et al., 2024; Olausson et al., 2024; Huang et al., 2024) que cuestiona que componer más llamadas a un LLM mejore, sin condiciones, el rendimiento agregado.

Las contribuciones principales del trabajo son: una arquitectura multi-agente con definición explícita de roles, estado compartido tipado y flujo de control condicional, completamente implementada y documentada; una métrica original de adherencia estructural que operacionaliza la afirmación —frecuente en la literatura pero raramente cuantificada— de que el protocolo de comunicación basado en artefactos reduce la incidencia de alucinaciones de formato; un análisis empírico del trade-off coste-calidad sobre el plano de Pareto entre configuraciones; evidencia

empírica controlada contra la hipótesis intuitiva de mejora multi-agente para el régimen HumanEval × modelo 7 B local; y la implementación completa publicada como repositorio reproducible.

Palabras clave: sistemas multi-agente, modelos de lenguaje de gran tamaño, generación automática de código, LangGraph, HumanEval, pass@k, self-reflection, ingeniería de software automatizada.

Abstract

This Bachelor's Thesis designs, implements and empirically evaluates a multi-agent system based on large language models for the collaborative resolution of automatic Python code generation tasks. The system orchestrates five specialised agents —Product Manager, Architect, Developer, QA Tester and Code Reviewer— through a typed state graph implemented with LangGraph, with structured communication through artifacts rather than free-form conversation.

Three incremental configurations are compared: a monolithic baseline with a single LLM, a sequential five-role pipeline, and a pipeline with an iterative self-revision loop between the Code Reviewer and the Developer. Evaluation is conducted on the HumanEval benchmark (164 problems) with three replicas indexed by pseudorandom seed, allowing for hypothesis testing via the paired McNemar test and 95 % confidence intervals via percentile bootstrap. Reported metrics include pass@1, pass@3 (the unbiased estimator from Chen et al., 2021), average test pass rate, token cost and latency. Role-ablation variants remain implemented in the runner for a second pass.

The system is implemented in Python with the 7-billion-parameter Qwen 2.5 Coder model served locally via Ollama on a MacBook Air M2. The experimental harness is resumable and fault-tolerant: each individual execution is persisted atomically and an interrupted run resumes with a single command, with no progress loss. A post-hoc analysis pipeline generates the figures and tables of the memoria from the resulting CSV files, including an automated draft of the findings narrative.

Main result. The experiment does not support the intuitive hypothesis that role specialisation improves correctness. The baseline reaches 80.08 % pass@1; the sequential pipeline drops to 58.33 % (McNemar $p < 0.0001$) and the self-reflection configuration to 64.84 % ($p < 0.0001$ against the baseline; $p = 0.0066$ against the sequential pipeline, supporting H2 for $r = 1$). Multi-agent configurations consume roughly 40 times more tokens and 77 times more latency per problem to produce worse pass@1, and the cost-quality Pareto front is occupied entirely by the baseline. Structural adherence to the artifact protocol remains near 100 % across all three configurations, confirming format robustness but not rescuing the correctness of the generated content. The work discusses three plausible explanations —error propagation across roles, overhead of role prompts on a 7 B model, and HumanEval being an inadequate benchmark for evaluating multi-agent pipelines— and positions the result within a recent critical strand of the literature (Chen et al., 2024; Olausson et al., 2024; Huang et al., 2024) that questions whether composing more LLM calls unconditionally improves aggregate performance.

The main contributions of this work are: a multi-agent architecture with explicit role definition, typed shared state and conditional control flow, fully implemented and documented; a novel structural-adherence metric that operationalises the claim —frequent in the literature but rarely quantified— that artifact-based communication protocols reduce format hallucination incidence; an empirical

analysis of the cost-quality trade-off along the Pareto front of configurations; controlled empirical evidence against the intuitive multi-agent improvement hypothesis for the HumanEval × local 7 B model regime; and the complete implementation released as a reproducible repository.

Keywords: multi-agent systems, large language models, automatic code generation, LangGraph, HumanEval, pass@k, self-reflection, automated software engineering.

Declaración sobre el uso de herramientas de inteligencia artificial

En cumplimiento de las directrices institucionales de la Universidad Alfonso X el Sabio sobre el uso responsable de herramientas de inteligencia artificial en trabajos académicos, y en línea con la Recomendación de la UNESCO sobre la Ética de la IA (2021) y con las obligaciones de transparencia del Reglamento Europeo de IA (Reglamento (UE) 2024/1689), el autor declara lo siguiente:

Herramientas utilizadas

Durante la elaboración de esta memoria se ha empleado el asistente conversacional **Claude** (Anthropic) como herramienta de apoyo a la redacción, revisión estilística y depuración del código fuente del sistema. La herramienta se ha utilizado en los siguientes contextos:

- **Redacción asistida.** Borradores iniciales de secciones técnicas, reescritura de fragmentos para mejorar la claridad expositiva y revisión de coherencia entre capítulos.
- **Revisión y refactorización de código.** Sugerencias de mejora sobre el código del sistema multi-agente, depuración de errores y generación de pruebas unitarias.
- **Documentación técnica.** Apoyo en la redacción del README, comentarios docstring y comandos de reproducción.

Carácter del uso

El uso de la herramienta ha sido **asistencial, nunca sustitutivo**. Todas las decisiones de diseño del sistema, las hipótesis del estudio, la metodología experimental, la interpretación de los resultados y la estructura argumental de la memoria son responsabilidad intelectual del autor. El asistente ha intervenido en la forma del texto y en la exploración de implementaciones, pero no en la formulación de las contribuciones originales del trabajo.

Los siguientes elementos del trabajo han sido producidos **íntegramente sin asistencia de IA**:

- La elección del tema y la formulación de la pregunta de investigación, recogidas en la propuesta original aprobada por el Jefe de Estudios.
- Las decisiones de scope sobre qué configuraciones evaluar, qué benchmarks utilizar y qué hipótesis contrastar.
- La interpretación cualitativa del estudio piloto exploratorio (capítulo 7, sección 7.2).
- El veredicto interpretativo de los resultados (rechazo de H1 y H3, respaldo de H2 para $r = 1$) y la identificación de las tres causas plausibles del hallazgo negativo (propagación de errores, sobrecarga del prompt en modelos pequeños, inadecuación de HumanEval para evaluar pipelines multi-agente).

Los siguientes elementos han sido **asistidos por IA pero revisados, editados y validados por el autor**:

- Borradores de los capítulos 1 a 8 y los anexos A a F, incluyendo la redacción final de la discusión por hipótesis del capítulo 8 y del análisis cualitativo del capítulo 7.7 a partir de los puntos interpretativos definidos por el autor.

- El código fuente bajo `src/`, `experiments/`, `tests/` y `scripts/`.
- La generación de las figuras de los grafos y del diagrama de arquitectura mediante renderizado de descripciones Mermaid.

Trazabilidad

Las conversaciones de asistencia con la herramienta han quedado registradas localmente durante el desarrollo del proyecto, en cumplimiento del principio de auditabilidad de la Recomendación UNESCO 2021. El autor mantiene la responsabilidad última sobre cada afirmación contenida en este documento, sobre el correcto funcionamiento del código entregado, y sobre la defensa oral del trabajo ante el tribunal evaluador.

Capacidad de defensa

El autor declara expresamente su capacidad de **explicar, justificar y defender** cada decisión técnica y conceptual recogida en esta memoria sin asistencia externa, en la defensa oral del trabajo ante el tribunal. La asistencia de IA durante el desarrollo no implica delegación de la competencia evaluable.

Madrid, junio de 2026

Fdo: Nicolás Timoneda

Índice de contenidos

Agradecimientos.....	3
Resumen	4
Abstract	5
Declaración sobre el uso de herramientas de inteligencia artificial	7
Herramientas utilizadas	7
Carácter del uso	7
Trazabilidad	8
Capacidad de defensa	8
Glosario y acrónimos.....	14
D.1. Sistemas multi-agente	14
D.2. Modelos de lenguaje	15
D.3. Generación y evaluación de código	16
D.4. Stack tecnológico	17
D.5. Métodos estadísticos	17
D.6. Configuraciones del sistema	18
D.7. Términos del estado compartido	18
D.8. Marco normativo y ético	19
Capítulo 1: Introducción	20
1.1. Motivación y contexto	20
1.2. Problema y pregunta de investigación.....	21
1.3. Objetivos y contribuciones.....	22
1.4. Estructura del documento.....	22
Capítulo 2: Estado del Arte y Marco Teórico	23
2.1. Sistemas multi-agente: fundamentos clásicos	23
2.2. Modelos de Lenguaje de Gran Tamaño como agentes	24
2.3. Frameworks multi-agente basados en LLM	25
2.4. Generación automática de código con LLMs	26
2.5. Benchmarks de evaluación de generación de código	27
2.6. Orquestación basada en grafos de estado	28
2.7. Síntesis y posicionamiento del trabajo.....	29
Capítulo 3: Objetivos e Hipótesis.....	31
3.1. Objetivo general.....	31
3.2. Objetivos específicos	31
3.3. Hipótesis de investigación	32
3.4. Trazabilidad entre objetivos e hipótesis	32
3.5. Alcance y limitaciones	32
Capítulo 4: Metodología.....	34
4.1. Enfoque metodológico	34

4.2. Descripción de los modelos implementados	34
4.3. Stack tecnológico y justificación	35
4.4. Diseño experimental	36
4.5. Consideraciones de reproducibilidad.....	39
Capítulo 5: Desarrollo	40
5.1. Estructura del repositorio	40
5.2. Estado compartido AgentState	40
5.3. Agentes implementados	41
5.4. Construcción de los grafos LangGraph	43
5.5. Sandbox de ejecución	46
5.6. Variantes de ablación	47
5.7. Factory de clientes LLM.....	48
5.8. Runner experimental	48
5.9. Recorrido completo de un problema.....	49
5.10. Resumen del capítulo	52
Capítulo 6: Experimentos	53
6.1. Matriz experimental.....	53
6.2. Modelo y backend de inferencia	54
6.3. Benchmarks utilizados	55
6.4. Métricas	55
6.5. Pipeline de análisis	57
6.6. Protocolo de ejecución y trazabilidad	57
6.7. Estado de la corrida.....	58
6.8. Reproducibilidad	58
Capítulo 7: Resultados.....	59
7.1. Estado de la corrida y alcance de los resultados	59
7.2. Estudio piloto exploratorio.....	59
7.3. Resultados cuantitativos por configuración	60
7.4. Adherencia estructural al protocolo	63
7.5. Análisis coste-calidad	63
7.6. Contraste de hipótesis	64
7.7. Análisis cualitativo: tipología de los fallos del pipeline.....	67
7.8. Reproducción de los resultados	70
Capítulo 8: Conclusiones y líneas futuras	71
8.1. Conclusiones generales	71
8.2. Cumplimiento de los objetivos específicos	71
8.3. Conclusiones por hipótesis.....	72
8.4. Contribuciones del trabajo	73
8.5. Desviaciones respecto a la propuesta original	74
8.6. Limitaciones del estudio	76
8.7. Líneas de trabajo futuro	77
8.8. Reflexión final	77

Anexo A: Prompts de los agentes	79
A.1. Agente Baseline (configuración monolítica)	79
A.2. Agente Product Manager	79
A.3. Agente Arquitecto	80
A.4. Agente Developer	80
A.5. Agente Developer Reflexivo (configuración con self-reflection)	81
A.6. Agente QA Tester	81
A.7. Agente Code Reviewer	81
A.8. Notas sobre los prompts	82
Anexo B: Decisiones técnicas.....	83
B.1. Sprint S1 — Estado compartido como TypedDict	83
B.2. Sprint S2 — QA Tester sin LLM y veredicto determinista.....	83
B.3. Sprint S3 — Temperatura asimétrica en self-reflection	83
B.4. Sprint S4 — Runner resumible y atómico	83
B.5. Sprint S5 — Inferencia muestral, no determinista	83
B.6. Sprint S6 — Backend local (Ollama) como referencia, Cerebras evaluado y descartado	84
B.7. Sprint S7 — Ablaciones, análisis y métrica de adherencia	84
B.8. Sprint S8 — Recorte de scope del barrido principal	84
B.9. Notas sobre el feedback del tutor	85
Anexo C: Comandos de reproducción	86
C.1. Preparación del entorno	86
C.2. Backend de inferencia	86
C.3. Verificación rápida	86
C.4. Corrida completa	86
C.5. Seguimiento en vivo	87
C.6. Análisis y regeneración de artefactos	87
C.7. Acceso a los benchmarks adicionales	88
C.8. Hardware de referencia	88
Anexo E: Aspectos éticos, legales y de sostenibilidad	89
E.1. Naturaleza de los datos utilizados	89
E.2. Modelos utilizados y sus licencias.....	89
E.3. Generación automática de código y responsabilidad.....	89
E.4. Marco legal: Reglamento Europeo de IA.....	90
E.5. Sostenibilidad y huella de cómputo	90
E.6. Reproducibilidad como obligación ética	91
E.7. Conflictos de interés	91
Bibliografía	92

Índice de figuras

Figura 4.1. Flujo experimental: del benchmark al análisis. Las cajas amarillas son datos, azules procesos del sistema, verdes salidas persistentes y moradas el pipeline de análisis post-hoc (figures/flujo_experimental.png).	37
Figura 5.1. Grafo de la configuración baseline (esquema generado con langgraph.draw_mermaid_png(), fichero figures/graph_baseline.png).	43
Figura 5.2. Grafo de la configuración secuencial (figures/graph_sequential.png).	44
Figura 5.3. Grafo de la configuración con self-reflection. La arista condicional reviewer → developer está controlada por el campo revision_count del estado y el hiperparámetro max_revisions (figures/graph_self_reflection_r1.png, r2.png, r3.png).	45
Figura 5.4. Diagrama de secuencia del pipeline secuencial y de la configuración con self-reflection. Las flechas sólidas representan lectura/escritura sobre el estado compartido; la flecha del Reviewer al Developer sólo se activa cuando el veredicto es REQUEST_CHANGES y revision_count < max_revisions (figures/secuencia_agentes.png).	46
Figura 6.1. Flujo experimental visto desde el banco: cada caja se describe en detalle en las secciones 6.2 a 6.7 (figures/flujo_experimental.png).	54
Figura 7.1. pass@1 por configuración con intervalos de confianza al 95 % mediante bootstrap percentil (figures/pass_at_1.png). La figura sustenta visualmente el contraste de la hipótesis H1: la barra del baseline supera a las dos configuraciones multi-agente sin solapamiento de intervalos.....	61
Figura 7.2. Frontera de Pareto coste–calidad: tokens totales por problema frente a pass@1 (figures/cost_quality_pareto.png). Cada punto representa una configuración.	64
Figura 7.3. Distribución del número de iteraciones de revisión por configuración self-reflection (figures/revision_distribution.png). Una distribución concentrada en r=0 indicaría que el sistema aprueba en la primera pasada en la mayoría de los casos y que el ciclo iterativo es poco utilizado; una distribución desplazada hacia valores altos indicaría lo contrario.	66

Índice de tablas

Tabla 1.1. Limitaciones del LLM monolítico y contramedidas del sistema propuesto. La tabla describe el diseño a priori: el experimento del capítulo 7 contrasta si esas contramedidas se traducen, sobre HumanEval y un modelo de 7 B, en mejoras medibles. El resultado, adelantado en la sección 1.1, es que no.....	21
Tabla 2.1. Posicionamiento del sistema propuesto frente a los frameworks multi-agente más representativos de la literatura.	26
Tabla 2.2. Comparativa de benchmarks de generación de código. La columna “Cardinalidad infra” indica el coste de infraestructura necesario para ejecutar la evaluación, que crece desde un sandbox Python aislado hasta un harness Docker por instancia.....	28
Tabla 3.1. Matriz de trazabilidad entre objetivos específicos e hipótesis. El símbolo ● indica que el objetivo aporta evidencia directa para el contraste de la hipótesis; — indica que la relación es instrumental o nula.	32
Tabla 5.1. Mapa de paquetes del proyecto.	40
Tabla 5.2. Especificación de los cinco agentes de rol.	42
Tabla 5.3. Variantes de ablación implementadas.	47
Tabla 5.4. Fila del CSV sequential_results.csv producida por la invocación del recorrido.	52
Tabla 6.1. Dimensiones de la matriz experimental completa.....	53
Tabla 6.2. Artefactos producidos por el pipeline de análisis.....	57
Tabla 7.1. Resumen del estudio piloto cualitativo.	60
Tabla 7.2. Resumen comparativo de las tres configuraciones al cierre de la corrida. La versión canónica, regenerada por el pipeline de análisis, está en doc/tables/summary.md.....	61
Tabla 7.3. Comparativa del baseline frente a referencias públicas del mismo modelo.	62
Tabla 7.4. Adherencia estructural al cierre de la corrida (doc/tables/adherence.md). La tabla se regenera ejecutando python experiments/adherence_metric.py.	63
Tabla 7.5. Comparaciones pareadas de McNemar entre configuraciones. La columna b recoge problemas donde la primera configuración acierta y la segunda falla; c el caso simétrico. La tabla viva está en doc/tables/pairwise_mcnemar.md.	65
Tabla 7.6. Problemas del Patrón A: solución canónica de una línea donde el baseline acierta el 100 % y el pipeline secuencial falla el 100 %.	68
Tabla 7.7. Problemas del Patrón B: una restricción semántica concreta que el pipeline pierde por el camino.	69
Tabla 8.1. Estado de cumplimiento de los objetivos específicos al cierre del documento.....	71
Tabla 8.2. Desviaciones respecto a la propuesta original. Cada recorte responde a una restricción identificada durante la ejecución del TFG y queda documentado en el sprint correspondiente del anexo B.	76

Glosario y acrónimos

Este anexo recoge los acrónimos y términos técnicos que aparecen reiteradamente en la memoria, agrupados por área temática.

D.1. Sistemas multi-agente

Sigla	Expansión	Definición breve
MAS	Multi-Agent System	Sistema computacional compuesto por agentes autónomos que interactúan en un entorno compartido.
BDI	Beliefs, Desires, Intentions	Modelo de arquitectura de agentes en el que el comportamiento emerge de creencias sobre el mundo, deseos y compromisos activos (Rao y Georgeff, 1995).
SOP	Standard Operating Procedure	Protocolo de actuación estructurado asignado a un rol; término popularizado por MetaGPT (Hong et al., 2024).
FIPA	Foundation for Intelligent Physical Agents	Organización que estandarizó protocolos de comunicación inter-agente a finales de los noventa.
SR	Self-Reflection	Patrón en el que un agente revisor evalúa la salida y la devuelve al productor para refinarla en un bucle acotado por número máximo de iteraciones.
PM	Product Manager	Rol del pipeline secuencial encargado de redactar el PRD a partir del enunciado.
LangGraph	—	Framework de orquestación de agentes basado en grafos de estado tipados, mantenido por LangChain Inc. (LangChain Inc., 2024).

D.2. Modelos de lenguaje

Sigla	Expansión	Definición breve
LLM	Large Language Model	Modelo de lenguaje preentrenado con un número grande de parámetros (típicamente miles de millones).
RAG	Retrieval-Augmented Generation	Patrón que combina un LLM con una recuperación de documentos relevantes para fundamentar la generación.
CoT	Chain-of-Thought	Técnica de prompting que pide al modelo razonar paso a paso (Wei et al., 2022).
ReAct	Reasoning + Acting	Patrón que intercala razonamiento y llamadas a herramientas (Yao et al., 2023).
TPM	Tokens Per Minute	Límite de tasa habitual en proveedores de inferencia comercial.
RPM	Requests Per Minute	Límite de tasa por número de solicitudes.

D.3. Generación y evaluación de código

Sigla	Expansión	Definición breve
pass@k	Pass at k	Probabilidad de que al menos una de k muestras independientes generadas por el modelo supere todos los tests del problema. Estimador insesgado de Chen et al. (2021).
HumanEval	Human-curated Evaluation	Benchmark de 164 problemas de generación de funciones Python con tests unitarios (Chen et al., 2021).
MBPP	Mostly Basic Python Problems	Benchmark complementario de problemas introductorios y de complejidad media (Austin et al., 2021).
APPS	Automated Programming Progress Standard	Benchmark de >10 000 problemas de competitive programming (Hendrycks et al., 2021).
SWE-bench	Software Engineering Benchmark	Benchmark de tareas reales de ingeniería del software a partir de issues de GitHub (Jiménez et al., 2024).
ClassEval	Class-level Evaluation	Benchmark de generación de código a nivel de clase con dependencias entre métodos (Du et al., 2023).
PRD	Product Requirements Document	Artefacto de especificación de requisitos producido por el agente Product Manager.
QA	Quality Assurance	Rol responsable de la verificación funcional del código mediante pruebas.
CI	Confidence Interval	Intervalo de confianza estadístico, reportado al 95 % en este trabajo.
IC	Intervalo de Confianza	Equivalente en castellano de CI.

D.4. Stack tecnológico

Sigla	Expansión	Definición breve
API	Application Programming Interface	Interfaz programática de un servicio.
CSV	Comma-Separated Values	Formato de fichero de los resultados experimentales.
JSON	JavaScript Object Notation	Formato de serialización de datos estructurados usado para prompts, logs y trazas.
TypedDict	Typed Dictionary	Tipo de Python para diccionarios con campos tipados; usado para el <code>AgentState</code> .
CPU	Central Processing Unit	Procesador del equipo de cómputo.
GPU	Graphics Processing Unit	Unidad de procesamiento gráfico, utilizada para acelerar la inferencia del LLM.
GPL	GNU General Public License	Familia de licencias de software libre.
MIT	Massachusetts Institute of Technology	Licencia de software permisiva, utilizada en este proyecto.

D.5. Métodos estadísticos

Sigla	Expansión	Definición breve
McNemar	Test de McNemar	Prueba estadística para clasificadores pareados sobre los mismos sujetos.
Bootstrap	Bootstrap percentil	Método de re-muestreo para estimar intervalos de confianza sin asumir distribución.
TFG	Trabajo Fin de Grado	Documento académico final de la titulación.
UAX	Universidad Alfonso X el Sabio	Institución académica del autor.

D.6. Configuraciones del sistema

Etiqueta	Descripción
<code>baseline</code>	Configuración 1, monolítica: un único LLM produce el código directamente.
<code>sequential</code>	Configuración 2, pipeline de cinco roles sin ciclos: PM → Arquitecto → Developer → QA → Reviewer.
<code>self_reflection_rN</code>	Configuración 3, pipeline secuencial con bucle Reviewer → Developer, hasta N iteraciones. $N \in \{1, 2, 3\}$.
<code>ablation_no_pm</code>	Variante que omite el Product Manager.
<code>ablation_no_architect</code>	Variante que omite el Arquitecto.
<code>ablation_no_reviewer</code>	Variante que omite el Code Reviewer.

D.7. Términos del estado compartido

Campo del AgentState	Productor	Descripción
<code>problem_statement</code>	Benchmark	Enunciado en lenguaje natural del problema.
<code>function_signature</code>	Benchmark	Identificador (función o clase) que se debe implementar.
<code>test_cases</code>	Benchmark	Aserciones unitarias asociadas al problema.
<code>prd</code>	Product Manager	Documento de requisitos estructurado.
<code>design_doc</code>	Arquitecto	Documento de diseño técnico.
<code>code_artifact</code>	Developer	Código fuente Python generado.
<code>test_results</code>	QA Tester	Mapa booleano por test que indica éxito o fallo.
<code>review_comments</code>	Code Reviewer	Veredicto APPROVE/REQUEST_CHANGES y comentarios cualitativos.
<code>revision_count</code>	Grafo SR	Número de iteraciones de revisión consumidas.
<code>tokens_input</code>	Todos	Tokens de entrada acumulados durante la ejecución.
<code>tokens_output</code>	Todos	Tokens de salida acumulados.
<code>latency_seconds</code>	Runner	Tiempo de wall-clock de la ejecución.

D.8. Marco normativo y ético

Sigla	Expansión	Definición breve
IA	Inteligencia Artificial	Término genérico para los sistemas tratados en este TFG. Equivalente al AI anglosajón.
RGPD	Reglamento General de Protección de Datos	Reglamento (UE) 2016/679 que regula el tratamiento de datos personales en la Unión Europea.
GPAI	General-Purpose AI	Categoría de modelos de propósito general definida por el Reglamento de IA de la UE (AI Act).
UE	Unión Europea	Marco jurisdiccional aplicable al desarrollo y despliegue del sistema.
UNESCO	Organización de las Naciones Unidas para la Educación, la Ciencia y la Cultura	Organismo cuyas recomendaciones éticas sobre IA se citan en el análisis ético.

Capítulo 1: Introducción

1.1. Motivación y contexto

El desarrollo de software profesional se reparte entre varios roles. Hay quien define los requisitos, quien diseña la arquitectura, quien escribe el código, quien lo prueba y quien lo revisa antes de integrarlo. Esta división del trabajo no es una convención: es una forma de gestionar complejidad. Ningún profesional puede mantener al mismo tiempo el detalle de los requisitos, la coherencia del diseño, la corrección del código y la cobertura de las pruebas; cuando se intenta, suelen aparecer errores precisamente en las interfaces entre fases.

Los modelos de lenguaje de gran tamaño han cambiado lo que se puede automatizar de ese ciclo. Hoy un modelo razonable es capaz de generar una función Python plausible a partir de su docstring. Lo que no puede hacer, al menos no de forma fiable, es saber si la función que ha generado es correcta. La ventana de contexto pone un techo a cuánto puede sostener en mente al mismo tiempo, y el modelo no tiene un mecanismo interno que distinga “código que compila” de “código que resuelve el problema”. El resultado típico es una solución sintácticamente válida que falla en casos límite que nadie le pidió comprobar.

La investigación reciente ha respondido a esto con sistemas multi-agente: varios LLMs con prompts distintos, cada uno encargado de una fase del trabajo, comunicándose a través de algún tipo de estado compartido. ChatDev, MetaGPT y AutoGen son los ejemplos más visibles. Los tres funcionan razonablemente bien en demostraciones, pero la evaluación empírica controlada sobre benchmarks estándar es menos sistemática de lo que cabría esperar: la mayoría de los trabajos publicados comparan el sistema completo contra un baseline monolítico, sin aislar qué papel juega cada rol concreto en el resultado final.

Este TFG construye un sistema multi-agente con cinco roles (Product Manager, Arquitecto, Developer, QA Tester, Code Reviewer) orquestado mediante un grafo de estado en LangGraph, y lo compara contra un baseline monolítico sobre HumanEval con tres configuraciones canónicas: baseline, pipeline secuencial sin ciclo, y pipeline con bucle de self-reflection. Tres variantes de ablación de rol quedan implementadas en el runner y disponibles para una segunda pasada. La pregunta concreta del trabajo es si la especialización por roles y la orquestación explícita producen mejoras medibles sobre HumanEval con un modelo de 7 B parámetros desplegado en local, y a qué coste. La respuesta del experimento, que conviene anticipar aquí para no generar falsas expectativas en el lector, es que en ese régimen el pipeline multi-agente no supera al baseline. El resto del documento se dedica a explicar por qué, a situar el resultado dentro de una discusión crítica reciente de la literatura, y a delimitar las condiciones bajo las cuales cabría esperar lo contrario.

La tabla 1.1 anticipa, en forma sintética, qué limitaciones del LLM monolítico el sistema propuesto pretende neutralizar y cómo. Los capítulos siguientes desarrollan cada fila en detalle.

Limitación del LLM monolítico	Contramedida del sistema multi-agente
Ventana de contexto finita; no puede sostener simultáneamente requisitos, diseño, código y pruebas	Distribución de responsabilidades entre cinco agentes con artefactos tipados en el estado compartido
No detecta errores en su propia salida	Agente QA Tester con ejecución determinista en sandbox y agente Code Reviewer con veredicto derivado de las pruebas
No tiene mecanismo de iteración fundamentada	Bucle condicional Reviewer → Developer parametrizado por <code>max_revisions</code> y guiado por evidencia externa
Comunicación en texto libre, sensible a alucinaciones	Protocolo estructurado por artefactos tipados (PRD, design, code, tests, review)
Difícil auditar el flujo de decisión	Grafo LangGraph explícito e inspeccionable que documenta cada transición

Tabla 1.1. Limitaciones del LLM monolítico y contramedidas del sistema propuesto. La tabla describe el diseño *a priori*: el experimento del capítulo 7 contrasta si esas contramedidas se traducen, sobre HumanEval y un modelo de 7 B, en mejoras medibles. El resultado, adelantado en la sección 1.1, es que no.

1.2. Problema y pregunta de investigación

La pregunta central del trabajo es si un sistema multi-agente con roles especializados y orquestación basada en grafos de estado mejora a un LLM monolítico en generación automática de código, y bajo qué condiciones ese beneficio —si existe— compensa el coste adicional en tokens y latencia.

Es una pregunta con dos partes. La técnica: ¿produce el sistema multi-agente soluciones más correctas, medidas por `pass@1` y `pass@k` sobre HumanEval? La económica: ¿a qué coste se obtiene la diferencia observada, sea cual sea su signo, y permite identificar un régimen en el que la complejidad adicional se justifica?

La pregunta se formula de forma neutra de manera deliberada. La literatura más visible (ChatDev, MetaGPT) presupone una respuesta positiva, pero trabajos recientes más críticos (Chen et al., 2024; Olausson et al., 2024; Huang et al., 2024) sugieren que la mejora no es automática y depende del régimen experimental. Este TFG aporta una respuesta empírica controlada para un régimen concreto —HumanEval, modelo de 7 B, ejecución local— y discute con honestidad qué implica esa respuesta y dónde se rompen sus límites.

Responder a las dos preguntas requiere un sistema realmente implementado y evaluado, no un diseño en papel. Por eso el trabajo entrega tres cosas concretas: el código en Python con LangGraph, los datos experimentales en CSV reproducibles, y el análisis estadístico formal sobre ellos.

1.3. Objetivos y contribuciones

El objetivo principal es diseñar, implementar y evaluar empíricamente un sistema multi-agente basado en LLMs para generación automática de código, comparándolo de forma controlada contra un baseline monolítico.

De ahí se derivan tres líneas de trabajo: diseñar la arquitectura con los cinco roles (Product Manager, Arquitecto, Developer, QA Tester y Code Reviewer) sobre un grafo de estado en LangGraph; implementar el sistema en Python de forma modular y reproducible; y evaluarlo empíricamente sobre HumanEval midiendo `pass@1`, `pass@3`, coste en tokens y latencia frente al baseline.

Las contribuciones concretas del trabajo son cinco:

1. La arquitectura multi-agente con estado compartido tipado y flujo de control condicional, publicada como repositorio reproducible en GitHub.
2. Las tres variantes de ablación de rol (`no_pm`, `no_architect`, `no_reviewer`) implementadas y preparadas para ejecución, que permiten aislar la contribución individual de cada agente al rendimiento global — un análisis que la literatura habitualmente omite. La corrida principal del TFG cubre las tres configuraciones canónicas (baseline, secuencial, self-reflection); las ablaciones quedan listas en el runner para una segunda pasada.
3. La métrica de adherencia estructural, que cuantifica con un número la afirmación —común en la literatura pero rara vez medida— de que el protocolo por artefactos reduce las alucinaciones frente a la conversación libre.
4. El análisis del trade-off coste-calidad como frontera de Pareto sobre las configuraciones evaluadas.
5. El banco experimental resumible y el pipeline de análisis automático, que generan figuras, tablas y test pareados de McNemar directamente desde los CSV.

1.4. Estructura del documento

El capítulo 2 revisa el estado del arte: desde los fundamentos de los sistemas multi-agente clásicos hasta los frameworks actuales basados en LLM, los benchmarks de generación de código y los mecanismos de orquestación basados en grafos. El capítulo 3 formaliza objetivos e hipótesis. El capítulo 4 describe la metodología y el protocolo experimental. El capítulo 5 detalla la implementación. El capítulo 6 presenta el banco experimental. El capítulo 7 analiza los resultados. El capítulo 8 recoge las conclusiones y abre líneas de trabajo futuro.

Seis anexos completan el documento. El Anexo A reproduce verbatim los prompts de sistema y las plantillas de prompt de usuario de los cinco roles del pipeline (PM, Arquitecto, Developer, QA Tester, Code Reviewer) y del baseline monolítico, incluyendo la variante de Developer Reflexivo empleada en la configuración con self-reflection. El Anexo B sintetiza las decisiones técnicas tomadas sprint a sprint. El Anexo C consolida los comandos de reproducción del experimento. El Anexo D recoge un glosario de acrónimos y términos. El Anexo E aborda los aspectos éticos, legales y de sostenibilidad del trabajo. El Anexo F contiene los agradecimientos.

Capítulo 2: Estado del Arte y Marco Teórico

2.1. Sistemas multi-agente: fundamentos clásicos

El concepto de agente autónomo no nació con los modelos de lenguaje. Tiene raíces bien establecidas en la inteligencia artificial distribuida de los años ochenta y noventa, y entender esa herencia es necesario para evaluar qué aportan los sistemas actuales y qué simplemente replican con otras herramientas.

Wooldridge y Jennings (1995) propusieron la definición que ha resultado más duradera: un agente es un sistema computacional situado en un entorno, capaz de actuar de forma autónoma para alcanzar sus objetivos. Esa definición implica cuatro propiedades. La **autonomía** —el agente opera sin intervención humana directa en cada decisión. La **reactividad** —percibe cambios en el entorno y responde a ellos. La **proactividad** —no se limita a reaccionar; toma iniciativa. Y la **socialidad** —interactúa con otros agentes de forma intencionada. Estas cuatro propiedades han guiado el diseño de agentes durante décadas, y siguen siendo el estándar de facto para caracterizar si un sistema merece ese nombre.

Un sistema multi-agente (MAS) es, en términos de Wooldridge (2009), un conjunto de agentes que interactúan en un entorno compartido. Lo que lo distingue de un sistema distribuido convencional no es la concurrencia de procesos en sí, sino la posibilidad de comportamientos colectivos que no están codificados en ningún agente individual. Esa propiedad emergente es lo que hace los MAS adecuados para problemas cuya complejidad supera lo que un agente único puede gestionar: planificación descentralizada, asignación de recursos, negociación, o resolución colaborativa de tareas.

El modelo de agente más estudiado formalmente es el BDI (*Beliefs, Desires, Intentions*). Sus bases filosóficas vienen del trabajo de Bratman (1987) sobre teoría de la acción racional: un agente no actúa con información completa ni recalcula su comportamiento desde cero en cada instante, sino que adopta compromisos y los mantiene mientras sean viables. Rao y Georgeff (1995) trasladaron esta idea a una arquitectura computacional concreta: las *beliefs* representan el conocimiento del agente sobre el mundo, los *desires* son los estados que quiere alcanzar, y las *intentions* son los planes actualmente en ejecución. El modelo BDI captura una tensión real: entre la capacidad de replanificar y el coste de hacerlo continuamente.

La coordinación entre agentes es el otro problema central. Cuando comparten recursos o tienen metas parcialmente conflictivas, necesitan comunicación explícita. Cohen y Levesque (1990) formalizaron la noción de intención comunicativa: un agente no solo transmite información, sino que actúa sobre el estado mental del receptor. Esta distinción —entre intercambiar datos y realizar *actos de habla*— fue la base sobre la que se construyeron los protocolos de comunicación inter-agente, como los especificados por la Foundation for Intelligent Physical Agents (FIPA) a finales de los noventa. Stone y Veloso (2000) extendieron esta visión hacia el aprendizaje: los agentes no solo pueden coordinarse con protocolos predefinidos, sino aprender estrategias de coordinación mediante interacción repetida.

Estos fundamentos —autonomía, comunicación estructurada, especialización de roles, coordinación emergente— son los que los frameworks modernos de agentes LLM intentan recuperar. Pero

el sustrato es completamente distinto: en lugar de agentes con arquitecturas simbólicas y protocolos formales, ahora se trabaja con modelos de lenguaje cuyo comportamiento emerge del preentrenamiento y del prompting. La siguiente sección examina qué significa exactamente que un LLM actúe como agente, y qué propiedades del agente clásico se conservan, cuáles se transforman, y cuáles desaparecen.

2.2. Modelos de Lenguaje de Gran Tamaño como agentes

Los modelos de lenguaje de gran tamaño (LLM) nacieron como sistemas de predicción de la siguiente palabra. A partir de 2022 empezó a verse que, por encima de cierta escala, esos modelos hacían cosas que no se les había entrenado a hacer de forma explícita: razonar paso a paso, planificar acciones, llamar a herramientas externas, evaluar su propia salida. Esas capacidades emergentes son las que abrieron la puerta a usarlos como núcleo cognitivo de un agente. Las dos revisiones sistemáticas más citadas —Xi et al. (2023) y Wang et al. (2024)— convergen en una descomposición funcional de tres bloques: percepción, razonamiento y acción, con memoria y planificación como dimensiones donde las arquitecturas se diferencian.

El punto de inflexión conceptual fue la introducción del prompting con cadena de pensamiento (*Chain-of-Thought*, CoT). Wei et al. (2022) demostraron que instruir a un LLM para que descomponga un problema en pasos intermedios verbalizados mejora sustancialmente su rendimiento en tareas de razonamiento aritmético, de sentido común y simbólico. Esta técnica no modifica los pesos del modelo; actúa sobre el proceso de inferencia mediante el formato del prompt. Su relevancia para los sistemas agénticos reside en que hace explícito el razonamiento interno del agente, lo cual facilita tanto la depuración como la coordinación con otros agentes.

El siguiente avance estructural fue el framework ReAct (Yao et al., 2023). ReAct intercala razonamiento y acción en el ciclo de generación del LLM: el modelo alterna entre pensamientos internos y llamadas a herramientas externas —motores de búsqueda, APIs, intérpretes de código—, incorporando las observaciones resultantes en el contexto antes de continuar. Este patrón observar-razonar-actuar es, en la práctica, una implementación del bucle percepción-deliberación-acción propio de los agentes BDI descritos en la sección anterior, aunque sin la formalización lógica de aquellos.

Una limitación del ciclo ReAct es que los errores no se corrigen automáticamente: un razonamiento incorrecto se propaga sin mecanismo de revisión. Shinn et al. (2023) propusieron Reflexion como respuesta a este problema: el agente genera una respuesta, recibe una señal evaluativa —procedente de un entorno externo o de un segundo LLM actuando como crítico— y produce una reflexión verbal que se incorpora al contexto de la siguiente iteración. Reflexion transforma así el ciclo de retroalimentación en un proceso de refinamiento lingüístico sin necesidad de actualizar los parámetros del modelo.

Toolformer (Schick et al., 2023) sistematizó el uso de herramientas externas. El trabajo mostró que un LLM puede aprender, de forma autosupervisada, a insertar llamadas a APIs en mitad de su propia generación. Eso es lo que permite que un agente haga cosas que la generación de texto pura no alcanza: consultar una base de datos, ejecutar código, acceder a información que el modelo no tiene en sus pesos.

Estos cuatro trabajos —CoT, ReAct, Reflexion, Toolformer— definen el repertorio sobre el que se han construido los sistemas multi-agente actuales. La siguiente pregunta es de coordinación: cómo hacer que varios de estos agentes cooperen en tareas que ninguno resolvería por separado. Los frameworks que han intentado responderla se examinan a continuación.

2.3. Frameworks multi-agente basados en LLM

El agente individual de la sección anterior tiene un techo claro: la ventana de contexto es finita y un único prompt no puede ser, a la vez, especialista en requisitos, diseño, implementación, pruebas y revisión. Una tarea de ingeniería de software de cierta complejidad exige todas esas competencias. Lo natural es repartirlas entre agentes con roles distintos y dejar la coordinación a un orquestador externo. Esa es la idea sobre la que se han construido los frameworks multi-agente con LLM que han ido apareciendo desde 2023.

El primero en formalizar el concepto de rol como mecanismo de especialización fue CAMEL (Li et al., 2023). En CAMEL dos agentes —uno con rol de «instructor» y otro de «asistente»— mantienen una conversación dirigida a resolver una tarea. El protocolo de intercambio sigue un ciclo estricto: el instructor emite instrucciones, el asistente las ejecuta y reporta, el instructor evalúa y emite la siguiente instrucción. Este trabajo mostró que la asignación de roles mediante prompts de sistema es suficiente para producir comportamientos cooperativos estables, sin necesitar mecanismos de coordinación explícitos fuera del propio lenguaje natural.

ChatDev (Qian et al., 2024) trasladó esta arquitectura al dominio específico del desarrollo de software. En ChatDev, distintos agentes adoptan roles propios de un equipo de ingeniería —director ejecutivo, jefe de producto, ingeniero de software, revisor de código, tester— y se comunican a través de una secuencia de fases de chat predefinidas. Cada fase produce artefactos concretos: un documento de requisitos, un diseño de clases, código fuente, un informe de pruebas. La evaluación empírica del trabajo demostró que este flujo produce software funcional en tareas de programación de complejidad moderada con mayor consistencia que un agente único. ChatDev es, en términos de dominio, el precedente más directo del sistema desarrollado en este TFG.

MetaGPT (Hong et al., 2024) profundizó en la dimensión estructural. Su aportación central es la noción de procedimiento operativo estándar (SOP) como mecanismo de coordinación: los roles de los agentes no solo definen quién hace qué, sino en qué orden, con qué entradas esperadas y qué artefactos deben producir como salida. Esta formalización reduce la ambigüedad en el flujo de información entre agentes y mejora la reproducibilidad de los resultados. MetaGPT introduce además una memoria compartida estructurada —basada en un repositorio de mensajes y artefactos accesible por todos los agentes— que resuelve parcialmente el problema de la fragmentación del contexto en sistemas con muchos agentes.

AutoGen (Wu et al., 2023) propone un enfoque distinto: en lugar de fijar de antemano el grafo de interacciones, ofrece un marco conversacional flexible en el que cualquier agente puede iniciar, responder o interrumpir la conversación según condiciones definidas por el diseñador. AutoGen introduce también el patrón human-in-the-loop de forma explícita: un agente puede delegar una decisión al operador humano cuando detecta ambigüedad o riesgo, lo que lo hace adecuado para entornos de desarrollo asistido. Desde el punto de vista de la arquitectura, AutoGen se aleja de las topologías secuenciales fijas y se acerca a una red de agentes con capacidad de enrutamiento dinámico.

De la revisión de estos frameworks se extraen dos dimensiones de clasificación relevantes para este trabajo. La primera es la topología de coordinación: los sistemas pueden organizarse de forma secuencial (pipeline), jerárquica (con un agente supervisor) o como red plana con enrutamiento dinámico. La segunda es el grado de estructuración del flujo: desde protocolos fijos con SOPs estrictos (ChatDev, MetaGPT) hasta conversaciones abiertas con condiciones de terminación configurables (AutoGen). El sistema propuesto en este TFG se posiciona en la intersección: utiliza una topología jerárquica con supervisor y un flujo parcialmente estructurado mediante un grafo de estado, lo que permite tanto reproducibilidad como adaptabilidad ante tareas imprevistas.

La tabla 2.1 sintetiza las características de los cuatro frameworks revisados en esta subsección, lo que permite situar el TFG por contraste con cada uno de ellos sin necesidad de releer las descripciones en prosa.

Framework	Topología	Comunicación	Roles fijos	Flujo de control	Determinismo	Estado tipado
AutoGen (Wu et al., 2023)	Red sin jerarquía	Conversación libre	No	Dinámico, dependiente del LLM	Bajo	No
CAMEL (Li et al., 2023)	Diádica (2 agentes)	Conversación bilateral	Sí (2)	Lineal	Bajo	No
ChatDev (Qian et al., 2024)	Cascada por fases	Estructurada por fase	Sí (5+)	Lineal (cascada de waterfall)	Medio	Parcial
MetaGPT (Hong et al., 2024)	Jerárquica con supervisor	SOPs estructuradas + artefactos	Sí (5)	Lineal con SOPs	Alto	Sí
TFG (este trabajo)	Grafo de estado explícito	Estado compartido tipado	Sí (5)	Lineal + arista condicional	Alto	Sí (TypedDict)

Tabla 2.1. Posicionamiento del sistema propuesto frente a los frameworks multi-agente más representativos de la literatura.

El sustrato tecnológico que hace posible esta combinación es LangGraph, cuya base teórica se examina en la subsección 2.6. Antes, conviene revisar el estado del arte en generación automática de código, dado que esa es la capacidad central que los agentes del sistema deben ejercer.

2.4. Generación automática de código con LLMs

La capacidad de los LLM para generar código fuente correcto —no solo sintácticamente válido— quedó delimitada con precisión por Chen et al. (2021) en el trabajo que introdujo Codex. El modelo, derivado de GPT-3 y ajustado sobre código público de GitHub, podía producir implementaciones en Python a partir de descripciones en lenguaje natural. Más que el modelo en sí, lo que aportó ese trabajo fue la métrica *pass@k*: la probabilidad de que al menos una de las *k* soluciones generadas supere el conjunto de pruebas del problema. Esa métrica señala algo que la exactitud sintáctica no captura: que el código sea ejecutable y se comporte como se espera.

Austin et al. (2021) añadieron el benchmark MBPP (Mostly Basic Python Problems), 374 problemas con especificación en lenguaje natural y pruebas unitarias asociadas. Lo que mostraron sus resultados fue un patrón que se repetiría en estudios posteriores: los modelos generan código plausible sin dificultad, pero la tasa de superación cae en cuanto el problema exige razonar sobre estructuras de datos no triviales o manejar casos límite.

AlphaCode (Li et al., 2022) abordó ese problema desde otro ángulo: generar hasta un millón de candidatos por problema y filtrar los que pasan las pruebas públicas disponibles. En benchmarks de competición algorítmica los resultados fueron notables, pero el coste computacional hace el enfoque inviable fuera de ese contexto específico.

Code Llama (Rozière et al., 2023) cambió el tipo de pregunta que vale la pena hacerse. En lugar de intentar maximizar el rendimiento en competición, el trabajo produjo una familia de modelos de código abierto con rendimiento comparable a Codex en HumanEval, con variantes especializadas en completado de fragmentos (*fill-in-the-middle*) e instrucción. El resultado práctico fue que la generación de código dejó de depender de APIs externas y pasó a ser desplegable en entornos locales.

Lo que el conjunto de estos trabajos deja sin resolver es la verificación. Un agente que genera código no tiene forma interna de saber si lo que ha producido es correcto; solo sabe que es sintácticamente válido. Para detectar errores se necesita o bien ejecución contra pruebas, o bien un segundo agente que revise la salida. Esa limitación es una de las razones concretas por las que el sistema propuesto en este TFG distribuye la tarea de generación y la de verificación en nodos distintos del grafo de orquestación. Los benchmarks que permiten medir esa capacidad de forma objetiva se examinan en la subsección siguiente.

2.5. Benchmarks de evaluación de generación de código

Medir si un modelo genera código correcto requiere algo más que comprobar si compila. Los benchmarks de código intentan resolver ese problema con distintos niveles de ambición, y la evolución de esas propuestas dice bastante sobre los límites que se han ido descubriendo en los modelos.

HumanEval (Chen et al., 2021) y MBPP (Austin et al., 2021), ya presentados en la sección anterior, establecieron el formato estándar: especificación en lenguaje natural, solución generada por el modelo, verificación mediante pruebas unitarias. Son reproducibles y comparables entre sistemas. El problema es que sus conjuntos de pruebas son pequeños —HumanEval incluye entre 7 y 8 casos por problema de media— y no cubren casos límite con suficiente densidad. Liu et al. (2023) cuantificaron ese déficit en EvalPlus: al ampliar los conjuntos de pruebas entre 80 y 125 veces más casos por problema, el rendimiento de los modelos cae de forma significativa. Las cifras de `pass@k` publicadas sobre HumanEval sobreestiman la corrección real.

Hay además un problema más estructural. Los ejercicios de HumanEval y MBPP son problemas de algoritmia diseñados para la evaluación, no tareas representativas del trabajo de ingeniería de software. Escribir una función que invierta una lista es diferente a localizar y corregir un bug en una base de código con historial de cambios, dependencias externas y pruebas de regresión.

SWE-bench (Jiménez et al., 2024) intentó cerrar esa distancia. El benchmark recopila 2 294 issues reales de repositorios de GitHub en Python, cada uno con una descripción del problema y un

conjunto de pruebas que validan el parche resultante. El modelo debe generar un diff que resuelva el issue sin romper el resto del código. Los resultados iniciales del paper —con los mejores modelos resolviendo menos del 4% de los problemas— dejan claro que la brecha entre generar funciones aisladas y resolver tareas reales de ingeniería de software es grande.

El sistema propuesto en este TFG se evalúa sobre HumanEval, no sobre SWE-bench. La razón es práctica: ejecutar SWE-bench requiere un harness Docker por instancia y un coste de cómputo incompatible con los recursos disponibles para un trabajo de fin de grado. HumanEval ofrece, en cambio, un banco controlado que permite aislar el efecto de la orquestación multi-agente sin que el ruido de la infraestructura domine los resultados. La hipótesis que se contrasta es, por tanto, más acotada: si distribuir el trabajo entre agentes especializados —uno que redacta requisitos, otro que diseña, otro que implementa, otro que prueba, otro que revisa— mejora la tasa de superación de pruebas unitarias sobre funciones aisladas. SWE-bench queda como referencia del techo de dificultad que cualquier sistema de este tipo aspira a abordar a medio plazo.

La tabla 2.2 contrasta los cinco benchmarks más relevantes de la literatura en términos de granularidad, cardinalidad y tipo de tarea. Aporta el marco de comparación que justifica las decisiones de scope tomadas en el capítulo 3.

Benchmark	Granularidad	Problemas	Verificación	Cardinalidad infra
HumanEval (Chen et al., 2021)	Función	164	Tests unitarios	Sandbox simple
MBPP (Austin et al., 2021)	Función	974	Tests unitarios	Sandbox simple
APPS (Hendrycks et al., 2021)	Función	10 000	Tests unitarios	Sandbox simple
ClassEval (Du et al., 2023)	Clase completa	100	Tests unitarios	Sandbox + imports
SWE-bench (Jiménez et al., 2024)	Proyecto multi-fichero	2 294	Suite del repo	Docker por repo

Tabla 2.2. Comparativa de benchmarks de generación de código. La columna “Cardinalidad infra” indica el coste de infraestructura necesario para ejecutar la evaluación, que crece desde un sandbox Python aislado hasta un harness Docker por instancia.

Los mecanismos de orquestación que hacen posible la distribución del trabajo entre agentes especializados se examinan en la subsección siguiente.

2.6. Orquestación basada en grafos de estado

Los frameworks descritos en la sección 2.3 comparten un problema de diseño: el flujo de control entre agentes es en gran medida fijo. Las fases de ChatDev siguen un orden predefinido; las conversaciones de CAMEL tienen inicio y fin claros. Cuando una tarea exige condicionar el siguiente paso al resultado del anterior —iterar sobre un fragmento de código hasta que pase las pruebas, o

derivar el trabajo a un agente distinto según el tipo de error detectado— estas arquitecturas no tienen un mecanismo limpio para expresarlo.

LangGraph (LangChain Inc., 2024) aborda ese problema modelando el flujo de orquestación como un grafo dirigido de estados finitos. Cada nodo es una unidad de procesamiento —en el contexto de este TFG, un agente especializado— y cada arista define una transición entre nodos. Las aristas pueden ser incondicionales o condicionales: en el segundo caso, la transición que se activa depende del contenido del estado compartido en ese momento. Eso permite expresar ciclos de revisión, bifurcaciones según el tipo de tarea y condiciones de terminación complejas, sin codificar esa lógica dentro de los propios agentes.

El estado compartido es el otro elemento central del modelo. LangGraph mantiene un objeto de estado mutable que todos los nodos pueden leer y escribir según sus responsabilidades. La información producida por un agente no se pierde entre llamadas, sino que persiste en el estado del grafo hasta que el flujo termina. Esto resuelve el problema de fragmentación de contexto que afecta a los sistemas basados en conversaciones encadenadas.

Esta arquitectura conecta con la noción de sistema multi-agente coordinado por un mecanismo de control explícito, tal como la formuló Wooldridge (2009). La diferencia es que en LangGraph ese mecanismo no es externo al sistema sino parte de su definición formal. Cómo este modelo se articula con el resto de elementos revisados en el capítulo es lo que sintetiza la sección siguiente.

2.7. Síntesis y posicionamiento del trabajo

Este capítulo ha recorrido un arco largo: desde la definición clásica de agente autónomo de Wooldridge y Jennings (1995) hasta los frameworks actuales de orquestación multi-agente sobre LLM. El problema que aparece en todas las etapas es el mismo: cómo coordinar agentes especializados para que produzcan, en conjunto, algo que ninguno alcanzaría por separado.

De esa revisión salen tres observaciones que justifican el diseño del sistema. Primero, los LLM generan código plausible pero no saben si es correcto; la verificación tiene que venir de fuera, vía ejecución contra pruebas o vía un segundo agente revisor. Segundo, frameworks como ChatDev y MetaGPT confirman que la especialización por roles mejora la coherencia del resultado, pero su flujo de control es lineal y no permite iteración condicional. Tercero, SWE-bench deja claro que la distancia entre resolver ejercicios aislados y resolver issues reales de software es enorme, y que ningún sistema actual la cierra de forma fiable.

Conviene, sin embargo, no asumir como artículo de fe que más agentes y más llamadas al modelo equivalgan, sin más, a mejores resultados. Una línea reciente de la literatura empieza a cuestionar esa lectura. Chen et al. (2024) muestran que, en sistemas de inferencia compuestos por múltiples llamadas a un LLM, el rendimiento agregado puede degradarse respecto al uso de una única llamada, especialmente cuando los errores intermedios se propagan y los criterios de filtrado no son estrictos. Olausson et al. (2024) llegan a una conclusión análoga para self-repair de código: la auto-corrección con un modelo único no es una *silver bullet*, y su efecto es más débil cuanto menor es la capacidad del modelo base. Huang et al. (2024) reportan en AgentCoder que las ganancias del multi-agente sobre HumanEval dependen de la calidad del agente de pruebas y se diluyen con modelos pequeños. Estos trabajos no invalidan los frameworks revisados antes; matizan el optimismo con el que suelen presentarse y delimitan el régimen en el que la coordina-

ción entre agentes aporta valor. El experimento de este TFG aporta evidencia empírica adicional a esa discusión, como se documenta en los capítulos 7 y 8.

El sistema propuesto en este TFG responde a esas limitaciones con un diseño concreto: cinco agentes especializados (PM, Arquitecto, Developer, QA, Code Reviewer) coordinados por un grafo de estado en LangGraph, con un bucle de revisión condicional Reviewer → Developer acotado por un número máximo de iteraciones. La evaluación se hace sobre HumanEval con tres configuraciones incrementales (baseline, secuencial sin ciclo, secuencial con self-reflection) y un conjunto de ablaciones por rol. El objetivo no es proponer una arquitectura sin precedentes, sino medir, de forma controlada y reproducible, qué aporta cada decisión de diseño sobre la tasa de superación, el coste en tokens y la latencia. El capítulo siguiente concreta ese diseño en una serie de objetivos verificables.

Capítulo 3: Objetivos e Hipótesis

3.1. Objetivo general

El objetivo general de este trabajo es diseñar, implementar y evaluar empíricamente un sistema multi-agente basado en modelos de lenguaje de gran tamaño que resuelva tareas de generación automática de código mediante agentes especializados en roles del ciclo de desarrollo software. El sistema orquesta esos agentes mediante un grafo de estado implementado en LangGraph, con un estado compartido tipado y comunicación estructurada a través de artefactos. Su rendimiento se compara de forma controlada contra un LLM monolítico como baseline, sobre benchmarks públicos y con métricas definidas de antemano, con el fin de determinar si la especialización por roles y la orquestación explícita producen mejoras medibles en corrección funcional y en qué condiciones justifican el coste computacional adicional.

3.2. Objetivos específicos

OE1. Revisar el estado del arte en sistemas multi-agente, agentes LLM y frameworks multi-agente basados en LLM para identificar el gap que justifica el enfoque propuesto y situar el trabajo respecto a la literatura existente.

OE2. Diseñar una arquitectura multi-agente con cinco roles especializados —Product Manager, Arquitecto, Developer, QA Tester y Code Reviewer— coordinados por un grafo de estado tipado en LangGraph, con comunicación estructurada mediante artefactos y aristas condicionales que permitan ramificaciones del flujo en función del estado compartido.

OE3. Implementar un baseline LLM monolítico como referencia comparativa, que utilice el mismo modelo base y el mismo schema de evaluación que el sistema multi-agente para garantizar comparaciones válidas entre configuraciones.

OE4. Incorporar un mecanismo de auto-revisión iterativa mediante un ciclo condicional entre el agente revisor y el agente desarrollador, parametrizado por un número máximo de iteraciones, que permita refinar el código generado sin intervención humana.

OE5. Evaluar empíricamente las tres configuraciones del sistema —baseline, multi-agente secuencial y multi-agente con self-reflection (`max_revisions = 1`)— sobre HumanEval (versión EvalPlus, 164 problemas) con tres semillas por configuración, reportando `pass@1`, `pass@3`, tasa media de superación de pruebas, coste en tokens y latencia. El análisis estadístico incluye test de McNemar pareado sobre los resultados `pass/fail` e intervalos de confianza bootstrap al 95 % para las métricas continuas. MBPP y las variantes adicionales de self-reflection (`r2`, `r3`) y de ablación quedan preparadas en el runner para una segunda pasada cuando el presupuesto de cómputo lo permita.

OE6. Analizar cualitativamente el comportamiento del sistema e identificar bajo qué condiciones —tipo de problema, complejidad, número de iteraciones— el enfoque multi-agente aporta valor frente al baseline monolítico.

OE7. Documentar el trabajo de forma que sea reproducible por terceros, con código fuente público en GitHub, prompts completos en anexo y configuración experimental declarada en `pyproject.toml`.

3.3. Hipótesis de investigación

Las hipótesis de investigación se formulan como proposiciones falsables que serán contrastadas mediante los experimentos descritos en el capítulo 4.

H1 — Hipótesis de especialización. El sistema multi-agente secuencial con roles especializados obtiene un pass@1 superior al LLM monolítico sobre HumanEval, utilizando el mismo modelo base en ambas configuraciones. Se contrastará mediante el test de McNemar pareado sobre los resultados pass/fail por problema.

H2 — Hipótesis de auto-revisión. El ciclo iterativo de revisión entre el agente revisor y el agente desarrollador mejora la corrección funcional —medida por pass@1 y average test pass rate— respecto al pipeline multi-agente sin ciclo, con un incremento cuantificable en coste de tokens y latencia. Se contrastará mediante McNemar para la comparación pass/fail y bootstrap CI 95% para las métricas continuas.

H3 — Hipótesis de trade-off. Existe un trade-off cuantificable entre calidad de la solución y coste computacional que varía según el tipo y la complejidad del problema, de forma que el beneficio de la configuración con self-reflection disminuye en problemas de baja complejidad. Se analizará segmentando los resultados por categoría de problema y comparando el ratio calidad/coste entre configuraciones.

3.4. Trazabilidad entre objetivos e hipótesis

La tabla 3.1 sintetiza la relación entre los objetivos específicos (OE1–OE7) y las hipótesis (H1–H3). Cada celda marcada indica que el cumplimiento del objetivo proporciona evidencia para el contraste de la hipótesis correspondiente. El propósito de la matriz es hacer explícito qué objetivos quedan estrictamente justificados por la metodología y cuáles cubren tareas instrumentales —revisión bibliográfica, documentación— que sostienen el trabajo sin contribuir directamente a las hipótesis.

Objetivo	H1 (especialización)	H2 (auto-revisión)	H3 (trade-off)
OE1 — Estado del arte	—	—	—
OE2 — Arquitectura multi-agente	●	●	●
OE3 — Baseline monolítico	●	—	●
OE4 — Mecanismo de auto-revisión	—	●	●
OE5 — Evaluación empírica	●	●	●
OE6 — Análisis cualitativo	●	●	●
OE7 — Reproducibilidad	—	—	—

Tabla 3.1. Matriz de trazabilidad entre objetivos específicos e hipótesis. El símbolo ● indica que el objetivo aporta evidencia directa para el contraste de la hipótesis; — indica que la relación es instrumental o nula.

3.5. Alcance y limitaciones

El trabajo abarca el diseño, implementación y evaluación de tres configuraciones: baseline monolítico, sistema multi-agente secuencial y sistema multi-agente con self-reflection. La corrida principal

cubre HumanEval completo (164 problemas) con tres semillas por configuración, sobre Qwen 2.5 Coder 7B Instruct servido en local con Ollama. MBPP y las variantes SR_r2/r3 y de ablación quedan preparadas en el runner pero se posponen a una segunda pasada: la corrida principal ya consume varios días de cómputo continuo, y el presupuesto disponible para el TFG no admite escalarla sin recortar otras partidas. La justificación cuantitativa de este recorte se documenta en S8 del anexo de decisiones técnicas.

SWE-bench completo no se incluye en la evaluación. Sus problemas requieren reproducir entornos Docker de repositorios externos con dependencias variables, una complejidad de infraestructura que excede los recursos disponibles para este TFG. Por la misma razón quedan fuera del alcance los benchmarks DevBench, ClassEval y APPS.

El trabajo tampoco realiza una comparativa exhaustiva de backends LLM por rol. Cubrir ese espacio de configuraciones de forma estadísticamente válida requeriría un número de experimentos inmanejable dentro del plazo disponible. Igualmente, el sistema no implementa Dynamic Task Decomposition con re-planificación en tiempo de ejecución: es una extensión natural de la arquitectura propuesta, pero requiere investigación adicional sobre mecanismos de planificación adaptativa que va más allá del alcance de este trabajo.

Estas limitaciones se recogen como líneas de trabajo futuro en el capítulo 8.

Capítulo 4: Metodología

4.1. Enfoque metodológico

Este trabajo adopta un enfoque empírico-comparativo controlado. El objetivo es medir con precisión qué diferencia produce cada componente arquitectural añadido, bajo qué condiciones y a qué coste computacional, no demostrar en abstracto que los sistemas multi-agente superan a los LLMs monolíticos.

Para eso se diseñan tres configuraciones incrementales: un baseline monolítico, un sistema multi-agente secuencial y un sistema multi-agente con self-reflection. La comparación entre el baseline y la configuración secuencial mide el efecto de la especialización por roles y el estado compartido; la comparación entre la configuración secuencial y la de self-reflection mide el efecto del ciclo de revisión iterativo. Sin esa estructura incremental, una diferencia de rendimiento entre el baseline y el sistema completo no podría atribuirse a ningún componente específico.

Las tres configuraciones se evalúan sobre los mismos benchmarks y con las mismas métricas, lo que garantiza que las diferencias observadas son atribuibles al diseño del sistema y no a variaciones en el entorno de evaluación.

4.2. Descripción de los modelos implementados

Configuración 1 — Baseline monolítico

El baseline consiste en un único nodo LangGraph que recibe el enunciado del problema y genera directamente la solución en código. No hay coordinación entre agentes, no hay estado compartido estructurado y el prompt incluye en un único mensaje la especificación completa de la tarea. Esta configuración es funcionalmente equivalente a usar el LLM directamente a través de su API con un prompt diseñado para maximizar la calidad de la respuesta en una sola llamada. Sirve como punto de referencia para cuantificar la diferencia que introducen las configuraciones más complejas.

Configuración 2 — Multi-agente secuencial

Esta configuración orquesta cinco agentes especializados en un pipeline sin ciclos: Product Manager, Arquitecto, Developer, QA Tester y Code Reviewer. El flujo es estrictamente secuencial: cada agente recibe el estado acumulado hasta ese punto, añade su artefacto al estado compartido y pasa el control al siguiente nodo.

El estado compartido es un TypedDict con campos tipados que corresponden a los artefactos de cada agente: PRD (documento de requisitos), DesignDoc (documento de diseño técnico), CodeArtifact (código fuente generado), TestResults (resultados de los casos de prueba) y ReviewComments (comentarios del revisor). Este diseño garantiza que cada agente trabaja sobre información estructurada y verificable, y no sobre texto libre sin tipo.

Los prompts de sistema de cada agente definen su rol, las entradas que debe consumir del estado y el formato exacto del artefacto que debe producir. La ejecución de los casos de prueba generados por QA Tester se realiza en un sandbox aislado con timeout para prevenir ejecuciones indefinidas.

Configuración 3 — Multi-agente con self-reflection

Esta configuración extiende la anterior con un ciclo iterativo de revisión. Tras la ejecución del Code Reviewer, una arista condicional evalúa el campo ReviewComments del estado: si el revisor aprueba el código, el flujo termina; si solicita cambios, el control vuelve al Developer para una nueva iteración.

El número de iteraciones realizadas se registra en el campo revision_count del estado compartido. El parámetro max_revisions actúa como hiperparámetro del sistema: fija el número máximo de ciclos Reviewer→Developer antes de forzar la terminación, independientemente del resultado de la revisión. La corrida principal del TFG fija `max_revisions = 1`; las variantes con `max_revisions ∈ {2, 3}` quedan implementadas y disponibles en el runner para una segunda pasada, según la decisión de scope S8 documentada en el anexo de decisiones técnicas.

4.3. Stack tecnológico y justificación

LangGraph es el framework de orquestación central del sistema. Modela el flujo de trabajo como un grafo dirigido de estados finitos: los nodos son los agentes especializados y las aristas definen las transiciones entre ellos, que pueden ser incondicionales o condicionales sobre el estado compartido (LangChain Inc., 2024). El flujo de control es explícito e inspeccionable, y el estado persiste entre nodos sin depender de la longitud de la ventana de contexto. Ambas propiedades son necesarias para implementar el ciclo de revisión de la configuración 3 de forma reproducible y auditable.

LangChain proporciona las abstracciones de integración con el backend LLM: gestión de prompts, invocación de herramientas y manejo de salidas estructuradas. Su uso desacopla la lógica de los agentes del proveedor de modelo subyacente, lo que permite cambiar el backend sin modificar el código de los agentes.

Python es el lenguaje del ecosistema de investigación en IA y dispone del tooling más completo para la evaluación de benchmarks de código: ejecutores de pruebas, entornos sandbox y librerías de análisis estadístico.

Backend LLM — Los experimentos finales utilizan un servidor local de inferencia mediante Ollama, ejecutando el modelo Qwen 2.5 Coder 7B Instruct con cuantización Q4_K_M. El despliegue local cumple dos funciones: la reproducibilidad —cualquier investigador puede replicar el experimento descargando el mismo binario del modelo desde el registro público de Ollama, sin depender de cuotas o disponibilidad de un proveedor remoto— y la eliminación del coste marginal por inferencia, lo que permite ejecutar múltiples réplicas sin restricciones presupuestarias que distorsionen el diseño experimental.

Durante la fase de planificación del experimento se evaluó como alternativa Cerebras Inference, un proveedor de inferencia remoto que ofrece acceso gratuito a varios modelos open-weight. La evaluación empírica mostró que el rate limit del tier público (~2 requests/minuto efectivos bajo la carga real del pipeline multi-agente) no era compatible con la cardinalidad de la matriz experimental. La decisión completa, con las alternativas medidas y descartadas, se documenta en la entrada S8 del anexo de decisiones técnicas.

El factory de clientes (`src/llm/client_factory.py`) abstrae el backend mediante la variable de entorno `LLM_BACKEND` $\in \{\text{ollama}, \text{cerebras}\}$, lo que mantiene abierta la opción de verificación cruzada sobre subconjuntos con un proveedor remoto sin tocar la lógica del pipeline.

4.4. Diseño experimental

La figura 4.1 sintetiza el flujo experimental completo. Cada problema del benchmark se combina con una configuración del sistema y una semilla para producir una ejecución; el resultado pasa por el sandbox y se persiste como fila del CSV correspondiente. El pipeline de análisis consume los CSV y emite las figuras y tablas que componen el capítulo 7 junto con el borrador automático de hallazgos.

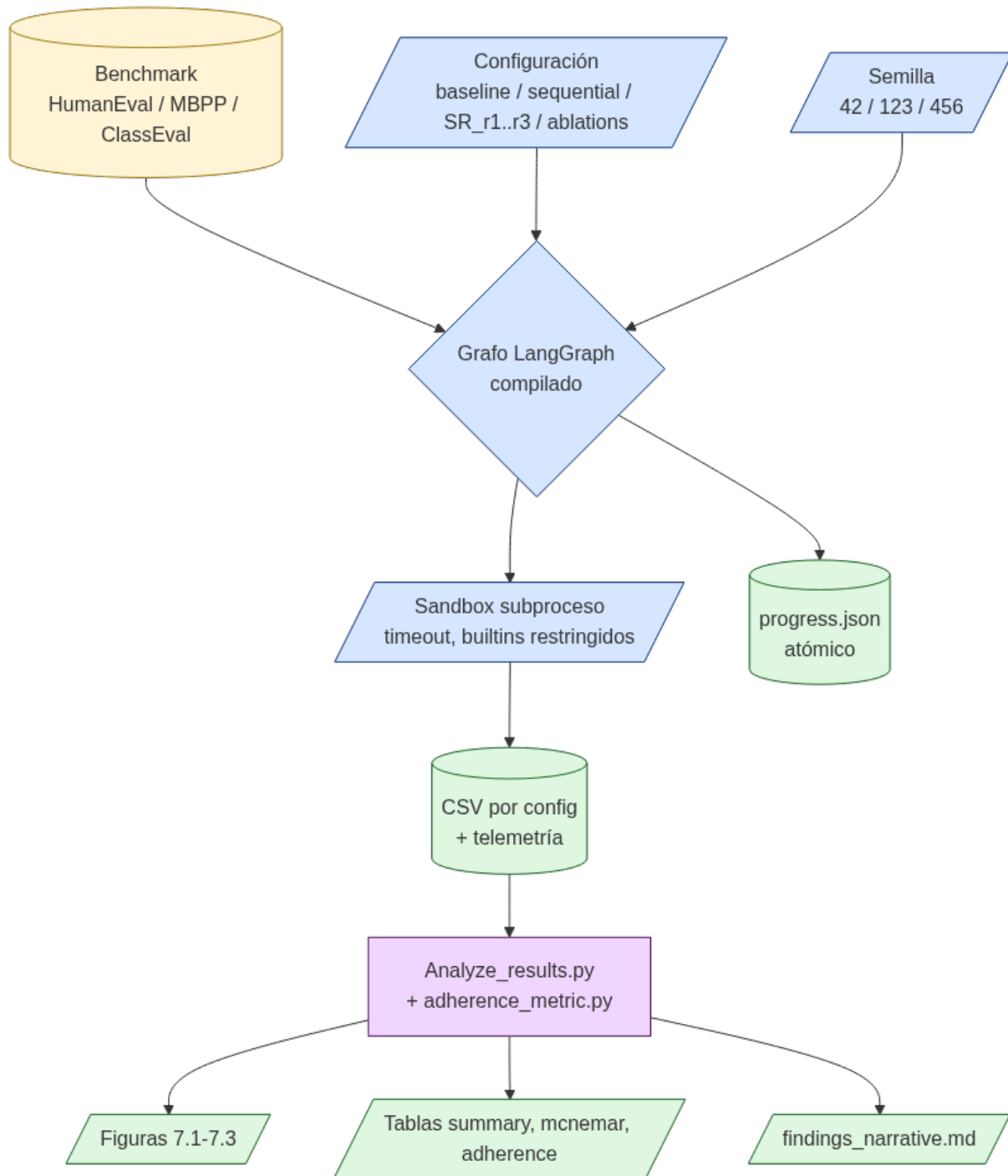


Figura 4.1. Flujo experimental: del benchmark al análisis. Las cajas amarillas son datos, azules procesos del sistema, verdes salidas persistentes y moradas el pipeline de análisis post-hoc (`figures/flujo_experimental.png`).

Benchmarks seleccionados

Los experimentos se realizan sobre HumanEval (164 problemas). El benchmark tiene especificación en lenguaje natural, solución verificable mediante pruebas unitarias y un uso extendido en la literatura, lo que permite comparar los resultados con los de trabajos previos (Chen et al., 2021).

MBPP, contemplado inicialmente, se descarta del diseño final por el coste temporal de las corridas con backend local de inferencia: dado el solapamiento elevado de habilidades evaluadas entre

HumanEval y MBPP, incorporar el segundo benchmark incrementaría sustancialmente el cómputo total sin aportar información cualitativa nueva al estudio comparativo de configuraciones. SWE-bench se descartó por motivos de infraestructura: sus problemas requieren reproducir el entorno de ejecución de repositorios externos con dependencias variables, lo que queda fuera del alcance de este TFG. HumanEval ofrece un entorno suficientemente controlado para los objetivos del trabajo.

Métricas

$\text{pass}@1$ mide la probabilidad de que una única solución generada supere todas las pruebas del problema. Para estimarlo de forma insesgada se utiliza el estimador de Chen et al. (2021):

$$\text{pass}@k = \mathbb{E}\left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\right]$$

donde n es el número de muestras generadas por problema, c el número de soluciones correctas y k el número de intentos considerados. En este trabajo se usa $n = 3$ (las tres réplicas del experimento) y $k = 1$, lo que reduce el estimador a $\text{pass}@1 = c / n$, una proporción de réplicas con éxito.

La tasa media de superación de pruebas (average test pass rate) mide, para las soluciones que no superan todas las pruebas, qué fracción de ellas sí superan. Esta métrica complementa $\text{pass}@k$ al capturar soluciones parcialmente correctas.

El coste en tokens se registra como número total de tokens consumidos por problema (entrada + salida). La latencia se mide como tiempo de wall-clock desde la recepción del enunciado hasta la entrega de la solución final.

Protocolo experimental

Cada problema se ejecuta con tres réplicas indexadas por las seeds {42, 123, 456} para estimar la varianza de los resultados. La inferencia es muestral por naturaleza —el parámetro `seed` controla el generador pseudoaleatorio del proceso experimental pero no se propaga a la API de inferencia del modelo—, por lo que cada réplica constituye una muestra independiente de la distribución de salidas del pipeline para ese problema. Las llamadas al modelo se realizan con temperatura 0.2 para las configuraciones 1 y 2, y con temperatura 0.4 para el agente Developer de la configuración 3, donde una mayor diversidad en la generación facilita la convergencia del ciclo de revisión. La ejecución de código se realiza en un sandbox Python con timeout de 5 segundos por caso de prueba.

Para preservar la metodología canónica de evaluación de HumanEval, el código generado por el sistema (`completion`) se concatena con el prompt original del problema (`prompt`) antes de su ejecución en el sandbox, replicando el patrón `prompt + completion` definido por Chen et al. (2021, §2.2). Esto garantiza que las importaciones y la firma de la función estén disponibles para el intérprete con independencia de las decisiones del agente Developer sobre qué incluir en su salida.

Los resultados de cada ejecución se guardan en un fichero CSV estructurado con los campos `benchmark`, `problem_id`, `configuration`, `seed`, `pass_all_tests`, `test_pass_rate`, `tokens_input`, `tokens_output`, `latency_seconds` y `revision_count` (para las configuraciones de self-reflection).

Análisis estadístico

Para comparar las tasas de pass@1 entre configuraciones se aplica el test de McNemar pareado, apropiado para comparar dos clasificadores sobre los mismos problemas. Para las métricas continuas —tasa de superación de pruebas, coste en tokens, latencia— se calculan intervalos de confianza al 95 % mediante bootstrap percentil con 2 000 remuestras por configuración. La implementación, con semilla fija para reproducibilidad, está en `experiments/analyze_results.py`.

4.5. Consideraciones de reproducibilidad

El código fuente del sistema se publica en un repositorio público de GitHub, incluyendo la definición del grafo LangGraph, los prompts completos de cada agente, los scripts de evaluación y los resultados en bruto. Los prompts de sistema de cada agente se recogen también en el Anexo A del documento.

Las dependencias del proyecto se declaran en un fichero `pyproject.toml` con versiones fijadas para todos los paquetes. Las seeds utilizadas en los experimentos se documentan en el fichero de configuración del experimento. El entorno de ejecución de pruebas es un proceso Python independiente sin acceso a red, con límites de memoria y tiempo configurables vía parámetros.

Hardware de ejecución

Toda la corrida experimental se ejecuta sobre un único equipo: un MacBook Air con chip Apple M2, 8 núcleos de CPU (4 de rendimiento y 4 de eficiencia), 10 núcleos de GPU y 16 GB de memoria unificada. La memoria unificada se comparte entre CPU y GPU, lo que tras descontar la reserva del sistema operativo deja aproximadamente 10-11 GB efectivos para el modelo más el caché de claves/valores de la inferencia. Esta restricción de hardware determina la elección del modelo —un Qwen 2.5 Coder de 7B parámetros con cuantización Q4_K_M, cuyo consumo en uso se sitúa en torno a 6-7 GB— y determina también el recorte del barrido principal a las tres configuraciones más informativas (baseline, sequential, self_reflection con `max_revisions=1`), justificado en la entrada S8 del anexo de decisiones técnicas. Las conclusiones del estudio se establecen para este modelo concreto y no se extrapolan automáticamente a modelos de mayor capacidad.

Capítulo 5: Desarrollo

5.1. Estructura del repositorio

El código del proyecto se reparte en cuatro paquetes bajo `src/` y un directorio `experiments/` con el banco de pruebas. La idea, en línea con Pereira-Vale et al. (2024), es que el sistema en sí —agentes, grafo, estado— no dependa del arnés que lo evalúa. Así los componentes se pueden invocar desde el experimento o desde cualquier otra integración sin tocar nada.

Paquete	Responsabilidad
<code>src/state/</code>	Definición del <code>AgentState</code> (<code>TypedDict</code>) compartido por todos los grafos
<code>src/agents/</code>	Agente base, agente monolítico y los cinco agentes de rol
<code>src/graph/</code>	Constructores de los tres grafos <code>LangGraph</code> y las variantes de ablación
<code>src/llm/</code>	Factory de clientes LLM con selección de backend por variable de entorno
<code>src/evaluation/</code>	Sandbox de ejecución de código, cargador de <code>HumanEval</code> y métricas
<code>experiments/</code>	Runner principal, pipeline de análisis, métrica de adherencia y dashboard

Tabla 5.1. Mapa de paquetes del proyecto.

La separación entre `src/agents/` y `src/graph/` es deliberada y vale la pena explicarla. En `src/agents/` vive la lógica del rol —el prompt de sistema, qué campos del estado lee, qué artefacto escribe—. En `src/graph/` vive la lógica de composición: en qué orden se ejecutan los agentes y cuándo se cicla. La misma instancia del `Developer` aparece tal cual en la configuración secuencial, en la de self-reflection y en dos de las tres ablaciones. Si los grafos y los agentes vivieran juntos, habría que duplicar código tres veces.

5.2. Estado compartido `AgentState`

El estado compartido es el contrato entre todos los nodos del grafo. Se implementa como un `TypedDict` de Python con campos tipados que cubren tres categorías: la identidad del problema, los artefactos producidos por los agentes y la telemetría acumulada durante la ejecución.

```
class AgentState(TypedDict):
    # Identidad del problema
    problem_id: str
    problem_statement: str
    function_signature: str
    test_cases: list[str]

    # Artefactos de los agentes
    prd: str
    design_doc: str
    code_artifact: str
```



```

test_results: dict[str, bool]
review_comments: str

# Control de iteración
revision_count: int

# Telemetría
tokens_input: int
tokens_output: int
latency_seconds: float

# Metadatos de la ejecución
config_name: str

```

Listado 5.1. Definición íntegra del estado compartido `AgentState`.

Cada campo del estado se asocia a un agente productor único, lo que evita condiciones de carrera lógicas en las que dos agentes intenten poblar el mismo artefacto. La separación es estricta: `prd` lo escribe únicamente el Product Manager, `design_doc` el Arquitecto, `code_artifact` el Developer (y, en la configuración con self-reflection, también su subclase reflexiva), `test_results` el QA Tester y `review_comments` el Code Reviewer. Los campos de telemetría acumulan en sitio: cada agente añade sus tokens consumidos a los contadores existentes.

El campo `config_name` registra qué configuración generó la traza, de forma que un mismo CSV puede unirse a las trazas de cualquier otra configuración sin ambigüedad. El campo `revision_count` permanece en cero para las configuraciones que no implementan el ciclo de auto-revisión.

La elección de `TypedDict` frente a `Pydantic.BaseModel` —que ofrecería validación en tiempo de ejecución— responde a una restricción concreta del framework: `LangGraph` requiere que el estado sea serializable por sus mecanismos internos de checkpointing, y el coste de validación en cada transición añadiría latencia sin aportar garantías relevantes en un grafo con escritores únicos por campo.

5.3. Agentes implementados

5.3.1. Agente base y `_call_llm`

Todos los agentes heredan de la clase abstracta `BaseAgent`, que centraliza la lógica de invocación al LLM. El método `_call_llm` acepta un prompt de sistema y un prompt de usuario, construye los mensajes en formato `LangChain` y los envía al cliente configurado. Incorpora retry exponencial con tres intentos y backoff geométrico (base 2 segundos) para tolerar fallos transitorios del backend, especialmente en escenarios de rate-limit en el modo Cerebras.

El método devuelve una tripleta (`response_text`, `input_tokens`, `output_tokens`) que cada agente concreto usa para escribir su artefacto y actualizar la telemetría del estado. La centralización del retry y de la lectura de `usage_metadata` en la clase base garantiza que ningún agente individual pueda silenciar errores ni desincronizar los contadores de tokens.

5.3.2. Agente monolítico (Baseline)

El `BaselineAgent` recibe el enunciado completo del problema y devuelve directamente el artefacto de código. Su prompt instruye al modelo a emitir la implementación dentro de un bloque

`python`. El agente extrae el contenido del `fence` mediante una expresión regular; si el `fence` no aparece, registra una advertencia y usa el texto íntegro como código, lo que sirve después como señal para la métrica de adherencia (sección 5.6).

5.3.3. Agentes de rol

Cada uno de los cinco agentes de rol implementa el método abstracto `run` sobre el `AgentState`. La tabla 5.2 resume sus entradas, salidas y prompts de sistema. Los prompts completos figuran en el Anexo A.

Rol	Entrada del estado	Salida	Resumen del prompt de sistema
Product Manager	<code>problem_statement</code>	<code>prd</code>	Generar PRD estructurado a partir del enunciado
Arquitecto	<code>prd</code> , <code>function_signature</code>	<code>design_doc</code>	Producir documento de diseño con algoritmo, estructuras y plan paso a paso
Developer	<code>design_doc</code> , <code>function_signature</code> , <code>problem_statement</code>	<code>code_artifact</code>	Implementar la función dentro de un bloque <code>python</code>
QA Tester	<code>code_artifact</code> , <code>test_cases</code>	<code>test_results</code>	(Sin LLM) Ejecutar tests en el sandbox
Code Reviewer	<code>code_artifact</code> , <code>test_results</code>	<code>review_comments</code>	Emitir veredicto APPROVE/ REQUEST_CHANGES + comentarios estructurados

Tabla 5.2. Especificación de los cinco agentes de rol.

Dos decisiones de diseño aquí no son obvias y conviene explicarlas. La primera: el QA Tester no llama al LLM. Es un nodo determinista que ejecuta el código en el sandbox y guarda el mapa de resultados. Podría parecer que ahorrarse ese nodo es lo limpio, pero mantenerlo como nodo del grafo tiene dos ventajas. Una, hace visible la etapa de verificación cuando se imprime el grafo. Otra, todos los campos del `AgentState` se acaban poblando por algún nodo, lo que evita estados intermedios con artefactos a medio rellenar.

La segunda: el veredicto del Reviewer (APPROVE / REQUEST_CHANGES) **no se le pide al LLM**. Se calcula a partir de `test_results`: si todos los tests pasan, APPROVE; si no, REQUEST_CHANGES. El LLM produce sólo los comentarios cualitativos. Esto es importante porque el router del grafo de self-reflection lee esa primera línea para decidir si vuelve al Developer. Pedir al LLM que genere literalmente `VERDICT: REQUEST_CHANGES` y confiar en que lo haga bien es la receta para fallos intermitentes; Hong et al. (2024) lo documentan como una fuente conocida de fragilidad en pipelines multi-agente con modelos pequeños.

5.4. Construcción de los grafos LangGraph

5.4.1. Grafo baseline

El grafo baseline contiene un único nodo, `solver`, conectado entre `START` y `END`. Sirve dos propósitos: aislar la diferencia entre invocación LLM directa y orquestación multi-agente, y proporcionar una referencia comparativa que sufre exactamente el mismo overhead de LangGraph que las configuraciones más complejas.

```
START → solver → END
```

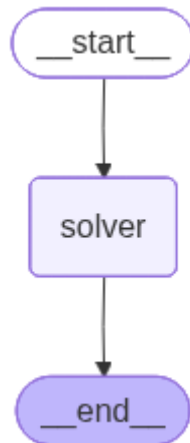


Figura 5.1. Grafo de la configuración baseline (esquema generado con `langgraph.draw_mermaid_png()`, fichero `figures/graph_baseline.png`).

5.4.2. Grafo secuencial

El grafo secuencial encadena los cinco agentes sin ciclos. La compilación produce el siguiente flujo:

```
START → pm → architect → developer → qa → reviewer → END
```

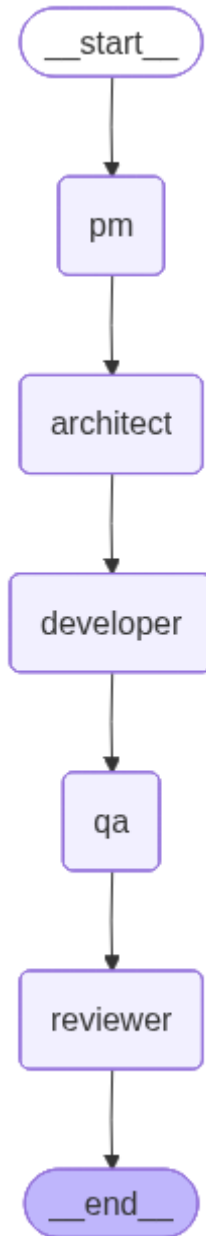


Figura 5.2. Grafo de la configuración secuencial (`figures/graph_sequential.png`).

Todos los agentes comparten el mismo cliente LLM, configurado con temperatura 0.2 para favorecer respuestas estables entre réplicas. Esta elección homogénea facilita la atribución causal: las diferencias entre configuraciones no se explican por diferencias de temperatura entre roles.

5.4.3. Grafo con self-reflection

La configuración con self-reflection reutiliza la topología secuencial pero añade una arista condicional desde el Reviewer. La función de enrutamiento inspecciona la primera línea de `review_comments` —que el Reviewer escribe en el formato `VERDICT: APPROVE` o `VERDICT: REQUEST_CHANGES`— y devuelve el siguiente nodo en consecuencia. Si el veredicto es `APPROVE`, el flujo termina. Si es `REQUEST_CHANGES` y `revision_count < max_revisions`, el flujo vuelve al Developer; si se ha alcanzado el límite de revisiones, termina forzosamente.

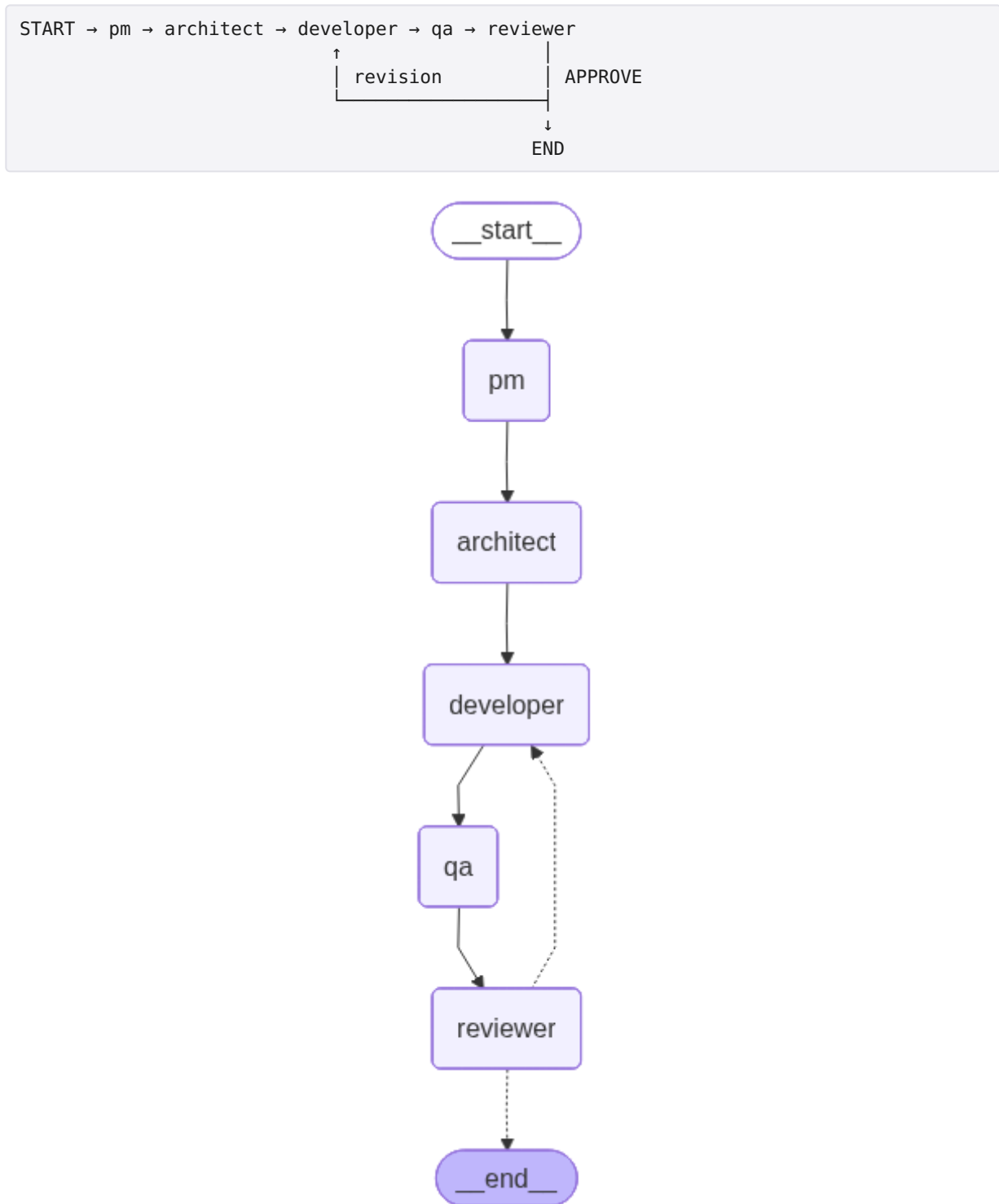


Figura 5.3. Grafo de la configuración con self-reflection. La arista condicional reviewer → developer está controlada por el campo `revision_count` del estado y el hiperparámetro `max_revisions` (figures/graph_self_reflection_r1.png, r2.png, r3.png).

El Developer en esta configuración es una subclase `ReflectiveDeveloperAgent` que sobrecarga `run()` para detectar la presencia de `review_comments` en el estado: cuando hay feedback, lo antepone al prompt del usuario antes de invocar al LLM. El cliente del Developer reflexivo se instancia con temperatura 0.4, una diversidad superior a la del resto de agentes para favorecer la exploración de soluciones alternativas en la iteración. Esta asimetría de temperatura es la única

diferencia funcional entre el Developer inicial y el Developer reflexivo; el resto del prompt es idéntico.

5.4.4. Vista de secuencia del pipeline

La figura 5.4 representa el intercambio de mensajes entre los cinco agentes y el estado compartido, en forma de diagrama de secuencia UML. Sirve como complemento a las vistas topológicas de las figuras 5.1, 5.2 y 5.3: muestra el orden temporal de las escrituras al `AgentState` y la asimetría del Code Reviewer, único nodo que puede emitir una arista inversa en la configuración con self-reflection.

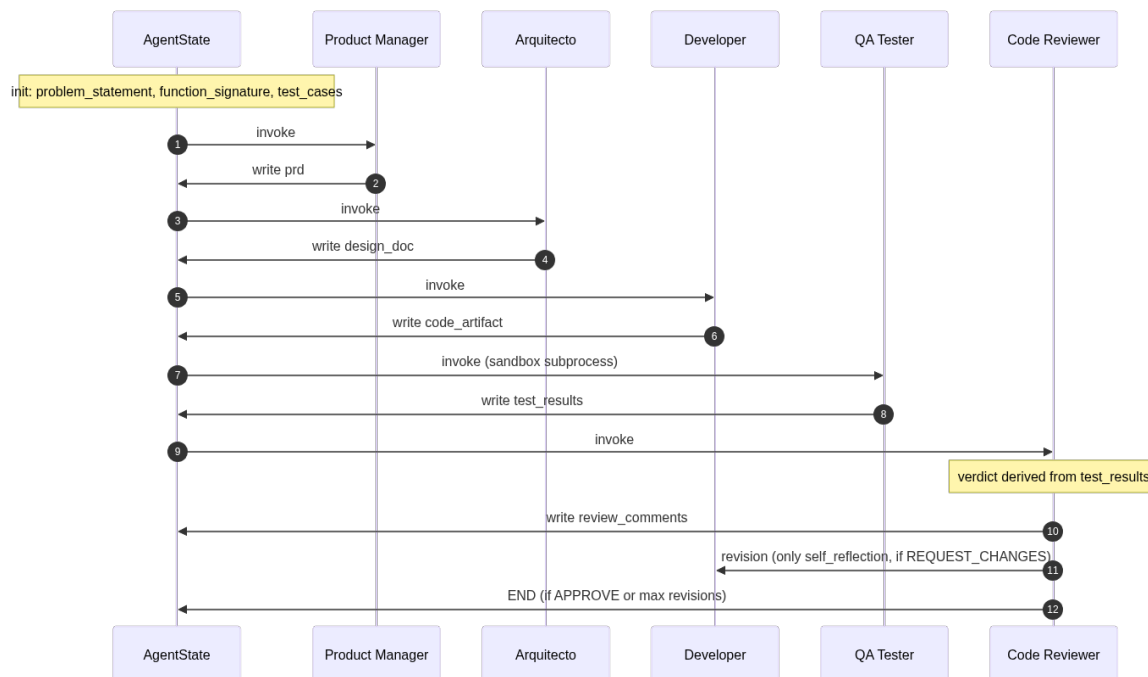


Figura 5.4. Diagrama de secuencia del pipeline secuencial y de la configuración con self-reflection. Las flechas sólidas representan lectura/escritura sobre el estado compartido; la flecha del Reviewer al Developer sólo se activa cuando el veredicto es `REQUEST_CHANGES` y `revision_count < max_revisions` (figures/secuencia_agentes.png).

5.5. Sandbox de ejecución

El sandbox de ejecución es el componente que cierra el ciclo de evaluación funcional: recibe una cadena con el código generado y la lista de aserciones del benchmark, ejecuta cada test en un proceso aislado y devuelve un mapa booleano `test → resultado`. Su implementación es deliberadamente conservadora.

La ejecución se realiza en un subprocesso Python independiente, no mediante `exec` en el proceso principal. Esta separación evita que el código generado contamine el espacio de nombres del runner, monopolice el GIL o invoque funciones del sistema fuera del entorno controlado. El subprocesso se lanza con timeout de cinco segundos por test —tiempo suficiente para los casos estándar de HumanEval, ajeno a las soluciones que entran en bucles infinitos—.

Los builtins disponibles dentro del subprocesso se restringen a un conjunto mínimo: tipos primitivos, funciones de iteración, conversiones de tipo y las construcciones necesarias para que `assert` funcione. Se omiten en particular `open`, `__import__` con efectos sobre el sistema de ficheros,

acceso a `os.environ` y similares. Cualquier intento de operación restringida termina con `NameError` y el test se contabiliza como fallo.

La métrica de evaluación se construye sobre el patrón canónico `prompt + completion` de Chen et al. (2021): antes de ejecutar el sandbox, el prompt original del problema —que en HumanEval contiene las importaciones y la firma de la función— se concatena al artefacto generado. Esto garantiza que la evaluación se realiza sobre el mismo string que se evalúa en la literatura, con independencia de las decisiones del Developer sobre qué incluir en su salida.

5.6. Variantes de ablación

La propuesta del trabajo identificaba como contribución innovadora el análisis de qué combinaciones de roles aportan más valor. Para cuantificarlo se implementan tres variantes que eliminan exactamente un rol del pipeline secuencial, manteniendo el resto idéntico. La corrida principal del TFG (3 configs × 164 problemas × 3 semillas = 1 476 ejecuciones) ya consume varios días de cómputo continuo en el equipo de referencia, por lo que la evaluación de las ablaciones queda preparada en el runner como segunda pasada (entrada S8 del anexo de decisiones). Las variantes están implementadas y testadas, y se documentan aquí por su valor como contribución arquitectural reutilizable y como punto de partida para el trabajo futuro de la sección 8.7.

Variante	Topología	Hipótesis que aísla
<code>ablation_no_pm</code>	architect → developer → qa → reviewer	Contribución de la fase de definición de requisitos
<code>ablation_no_architect</code>	pm → developer → qa → reviewer	Contribución del documento de diseño técnico
<code>ablation_no_reviewer</code>	pm → architect → developer → qa	Contribución del veredicto explícito de revisión

Tabla 5.3. Variantes de ablación implementadas.

Cada variante introduce un nodo intermedio sin coste de LLM —`seed_prd` o `seed_design`— que copia el artefacto inmediatamente disponible al campo que el siguiente agente espera consumir. Por ejemplo, en `ablation_no_pm` el nodo `seed_prd` copia `problem_statement` a `prd` antes de invocar al Arquitecto, manteniendo válido el formato del prompt del Arquitecto sin gastar tokens en una etapa de Product Manager.

La variante `ablation_no_reviewer` termina tras el QA Tester. Esto preserva la evaluación funcional —el sandbox sigue ejecutándose— pero elimina toda señal cualitativa del Code Reviewer. Su comparación contra el pipeline secuencial completo cuantifica el valor del veredicto explícito de revisión en términos de `pass@1`, independientemente del ciclo de iteración que sólo existe en la configuración con self-reflection.

Las ablaciones comparten `AgentState`, agentes y sandbox con el pipeline principal: son únicamente reconfiguraciones del grafo. Esto garantiza que las diferencias observadas se atribuyen a la ausencia del rol y no a cambios accidentales en otros componentes.

5.7. Factory de clientes LLM

El proyecto soporta dos backends de inferencia, seleccionables mediante la variable de entorno `LLM_BACKEND`:

- `ollama` (por defecto en la corrida final): servidor local escuchando en `http://localhost:11434/v1` con el modelo `qwen2.5-coder:7b-instruct-q4_K_M`.
- `cerebras`: Cerebras Inference API con el modelo `qwen-3-235b-a22b-instruct-2507`, autenticado mediante `CEREBRAS_API_KEY`.

Ambos backends exponen una interfaz compatible con OpenAI, lo que permite encapsular su selección en una sola función `create_chat_client(model_name, temperature=0.2)` que devuelve un cliente LangChain configurado. Los agentes desconocen qué backend está activo: reciben el cliente ya construido y lo usan a través de la interfaz uniforme de LangChain. El backend Cerebras se evaluó durante la planificación y se descartó para la corrida principal por incompatibilidad del rate limit del tier público con la cardinalidad de la matriz (entrada S8 del anexo de decisiones); queda, sin embargo, disponible como mecanismo de verificación cruzada sobre subconjuntos pequeños con un modelo de mayor capacidad, sin modificaciones en la lógica del pipeline.

5.8. Runner experimental

El runner principal (`experiments/run_experiments.py`) recibe como argumentos la lista de configuraciones a evaluar, los benchmarks, las seeds y el modelo, y construye una matriz cartesiana de ejecuciones. La matriz definida en código contempla ocho configuraciones (baseline, sequential, `self_reflection_r1/r2/r3` y las tres ablaciones) por tres seeds por el total de problemas del benchmark. La corrida principal ejecuta el subconjunto canónico (baseline, sequential, `SR_r1`) sobre HumanEval, según la decisión de scope S8; el resto de la matriz permanece disponible como segunda pasada sin modificaciones en el código.

Tres propiedades del runner merecen mención: la resumibilidad, la atomicidad y la tolerancia a fallos.

Resumibilidad. Antes de planificar la matriz, el runner lee los CSV existentes y construye un conjunto de tuplas (`benchmark, problem_id, config, seed`) ya completadas. La matriz efectiva excluye esos puntos. Reiniciar el proceso tras una interrupción —cierre del portátil, fallo de Ollama, SIGTERM— retoma el experimento donde quedó, sin re-ejecutar trabajo ya realizado y sin manipulación manual del estado.

Atomicidad. El archivo `experiments/results/progress.json` se actualiza tras cada ejecución completa-

da mediante un patrón de escritura atómica: se escribe primero en un fichero temporal `.json.tmp` y se renombra al nombre final con `Path.replace()`. La operación POSIX `rename` es atómica sobre el mismo sistema de ficheros, lo que garantiza que el dashboard de seguimiento nunca observa estados parcialmente escritos del fichero de progreso.

Tolerancia a fallos. Cada ejecución individual se envuelve en un `try/except` amplio que captura cualquier excepción del pipeline, registra el traceback en `experiments/logs/errors.log` y escribe una fila en el CSV con el campo `error` poblado. El experimento global continúa con los proble-

mas siguientes. Esta política, contraria al fail-fast habitual en desarrollo, es deliberada para corridas largas donde un fallo aislado en un problema no debe abortar las miles de ejecuciones restantes.

5.9. Recorrido completo de un problema

Esta sección sigue, paso a paso, una invocación de la configuración secuencial sobre el problema HumanEval/1 (separación de grupos de paréntesis). Su propósito es ilustrar cómo se van poblando los campos del AgentState a lo largo del grafo y qué produce cada agente sobre el mismo input. Los artefactos reproducidos son los efectivamente generados por el sistema en una de las réplicas (seed=42); el formato se ajusta para encajar en el ancho de la página, sin alterar el contenido.

5.9.1. Entrada al grafo

El benchmark proporciona el siguiente enunciado y firma de función:

```
from typing import List

def separate_paren_groups(paren_string: str) -> List[str]:
    """ Input to this function is a string containing multiple groups of
    nested parentheses. Your goal is to separate those groups into
    separate strings and return the list of those. Separate groups are
    balanced (each open brace is properly closed) and not nested within
    each other. Ignore any spaces in the input string.
    >>> separate_paren_groups('( ) (( )) (( )( ))')
    ['()', '(()', '(()())']
    """
```

Listado 5.2. Enunciado original de HumanEval/1.

El runner extrae las aserciones del test del benchmark y construye el estado inicial del grafo con todos los campos de artefacto vacíos:

```
initial_state = {
    "problem_id": "HumanEval/1",
    "problem_statement": "<enunciado completo del listado 5.2>",
    "function_signature": "separate_paren_groups",
    "test_cases": ["check(separate_paren_groups)"],
    "prd": "",
    "design_doc": "",
    "code_artifact": "",
    "test_results": {},
    "review_comments": "",
    "revision_count": 0,
    "tokens_input": 0,
    "tokens_output": 0,
    "latency_seconds": 0.0,
    "config_name": "sequential",
}
```

Listado 5.3. Estado inicial de la invocación.

5.9.2. Product Manager → PRD

El Product Manager recibe el problem_statement y emite el siguiente PRD que escribe en state["prd"]:

```

(1) Problem Summary
Separate a string of multiple non-nested, balanced groups of
parentheses into a list of individual group strings. Whitespace must
be ignored.

(2) Acceptance Criteria
- The function returns a list of strings.
- Each returned string is itself a balanced parenthesis expression.
- The concatenation (ignoring whitespace) of the returned strings
  reconstructs the input.
- Whitespace characters in the input do not appear in any output.
- Empty input returns an empty list.

(3) Edge Cases
- Input with only whitespace → empty list.
- Single group → single-element list.
- Deeply nested but separate groups → still separated correctly.

(4) Out of scope
- Malformed input (unbalanced parentheses).
- Nested groups (the problem statement guarantees they are separate).

```

Listado 5.4. PRD producido por el Product Manager.

5.9.3. Arquitecto → Documento de diseño

El Arquitecto consume el PRD y la `function_signature` y devuelve un documento de diseño técnico que escribe en `state["design_doc"]`:

```

(1) Algorithm choice
Single-pass scan over the input string, maintaining a running depth
counter and a buffer for the current group. When the counter returns
to zero, the buffered group is appended to the result list.

(2) Data structures
- An integer `depth` for the running parenthesis balance.
- A list[str] `groups` accumulating completed groups.
- A list[str] `current` (or string buffer) for the in-progress group.

(3) Implementation plan
- Strip whitespace from the input.
- Initialise depth = 0, groups = [], current = [].
- For each character c in the cleaned string:
  * Append c to current.
  * If c == '(', increment depth.
  * If c == ')', decrement depth.
  * If depth == 0 after the update, flush current to groups and
    reset current.
- Return groups.

(4) Failure modes
- The problem guarantees balanced and non-nested groups, so depth
  cannot go negative under valid input.
- Whitespace stripping must happen before the loop to avoid spurious
  empty groups.

```

Listado 5.5. Documento de diseño producido por el Arquitecto.

5.9.4. Developer → Código

El Developer toma el documento de diseño y la firma y emite el artefacto de código que se escribe en `state["code_artifact"]`:

```
def separate_paren_groups(paren_string: str) -> List[str]:
    # Strip whitespace first so the scan sees only parentheses.
    cleaned = paren_string.replace(" ", "")
    groups: list[str] = []
    current: list[str] = []
    depth = 0
    for ch in cleaned:
        current.append(ch)
        if ch == "(":
            depth += 1
        elif ch == ")":
            depth -= 1
        if depth == 0 and current:
            groups.append("".join(current))
            current = []
    return groups
```

Listado 5.6. Implementación generada por el Developer.

5.9.5. QA Tester → Resultados de las pruebas

El QA Tester ejecuta el sandbox concatenando el `problem_statement` (que contiene la importación `from typing import List`) con el `code_artifact`, sigue el patrón `prompt + completion` de Chen et al. (2021) y produce el siguiente mapa de resultados:

```
state["test_results"] = {
    "check(separate_paren_groups)": True,
    "qa_summary": {"passed": 1, "failed": 0, "errors": 0},
}
```

Listado 5.7. `test_results` registrado por el QA Tester.

El sandbox se ejecuta en subproceso aislado con timeout de cinco segundos y builtins restringidos según se describe en la sección 5.5. No se invoca al LLM en esta etapa.

5.9.6. Code Reviewer → Veredicto y comentarios

El Code Reviewer recibe el código, los `test_results` y el documento de diseño. Deriva primero el veredicto determinísticamente: como todos los tests pasan, el veredicto es `APPROVE`. A continuación el LLM produce el comentario cualitativo. El campo `review_comments` queda:

```
VERDICT: APPROVE

(1) Issues found:
None. The implementation follows the design document precisely. The
whitespace handling is correct, the depth counter correctly identifies
group boundaries, and the loop terminates with an empty `current`
buffer.

The code is concise and self-documenting.
```

Listado 5.8. `review_comments` final del Reviewer.

5.9.7. Estado final y telemetría

Tras el último nodo del grafo, el estado contiene los cinco artefactos poblados, los contadores de telemetría acumulados y los metadatos de la corrida. La fila correspondiente del CSV es:

Campo	Valor
benchmark	HE
problem_id	HumanEval/1
config	sequential
seed	42
pass_all_tests	True
test_pass_rate	1.0
tokens_input	1842
tokens_output	487
latency_seconds	247.8
revision_count	0
model	qwen2.5-coder:7b-instruct-q4_K_M

Tabla 5.4. Fila del CSV `sequential_results.csv` producida por la invocación del recorrido.

El valor `revision_count = 0` refleja que en la configuración secuencial nunca hay re-iteraciones del Developer; ese campo sólo es relevante para la configuración con self-reflection (sección 5.4.3). El coste total —2 329 tokens y ~248 segundos en la corrida local sobre Apple Silicon M2— se compara con el coste del baseline para este mismo problema (~5 segundos, ~280 tokens) en el análisis del trade-off del capítulo 7.

Este recorrido demuestra dos propiedades del diseño que en abstracto podrían parecer redundantes. La primera: cada campo del `AgentState` tiene un productor único, lo que hace que la traza sea reproducible y auditable problema a problema. La segunda: la derivación determinista del veredicto desacopla el contrato del grafo del formato libre que pueda emitir el LLM, garantizando que el router condicional de la configuración con self-reflection siempre dispone de una primera línea estable que leer.

5.10. Resumen del capítulo

Recapitulando: el sistema tiene un estado compartido tipado, seis agentes (cinco de rol y uno monolítico), tres grafos principales y tres ablaciones que reaprovechan el mismo `AgentState`. El sandbox en subproceso ejecuta el código generado de forma aislada, y un runner resumible se encarga de orquestar las miles de ejecuciones del experimento sin perder progreso si algo se cae a mitad.

El capítulo 6 describe el banco experimental construido sobre esta base; el 7 analiza los resultados disponibles a fecha de cierre.

Capítulo 6: Experimentos

6.1. Matriz experimental

La matriz experimental cruza tres dimensiones independientes: la configuración del sistema, el benchmark sobre el que se evalúa y la semilla pseudoaleatoria que indexa la réplica. La tabla 6.1 enumera los puntos de cada dimensión.

Dimensión	Valores	Cardinalidad
Configuración	baseline, sequential, self_reflection_r1, self_reflection_r2, self_reflection_r3, ablation_no_pm, ablation_no_architect, ablation_no_reviewer	8
Benchmark	HumanEval (164 problemas), MBPP (subconjunto de 200 problemas)	364 problemas
Semilla	42, 123, 456	3 réplicas
Total		8 736 ejecuciones

Tabla 6.1. Dimensiones de la matriz experimental completa.

El total efectivo de la corrida principal corresponde a la versión recortada por decisión de scope (S8 del anexo de decisiones): se evalúan las tres configuraciones canónicas —baseline, sequential y self_reflection_r1— sobre HumanEval únicamente, con tres semillas por configuración. Eso suma $3 \times 164 \times 3 = \mathbf{1\,476\,ejecuciones}$. Las variantes de self-reflection con `max_revisions` mayor (r2, r3), las tres ablaciones y la inclusión de MBPP quedan preparadas en el runner como segunda pasada, gracias a la propiedad de resumibilidad descrita en la sección 5.8: tanto el cache local de MBPP (`experiments/cache/mbpp.json`) como el registro de configuraciones del runner están ya listos para retomarlas, y reincorporar benchmarks o configuraciones no obliga a re-ejecutar nada de lo ya completado.

La figura 6.1 reproduce, a nivel del banco experimental, el diagrama de flujo de la sección 4.4 (figura 4.1). En este capítulo la figura sirve de referencia visual mientras se describen las decisiones de configuración que rellenan cada caja del flujo: qué benchmark se carga, qué modelo se invoca, qué métricas se computan y dónde aterriza cada artefacto.

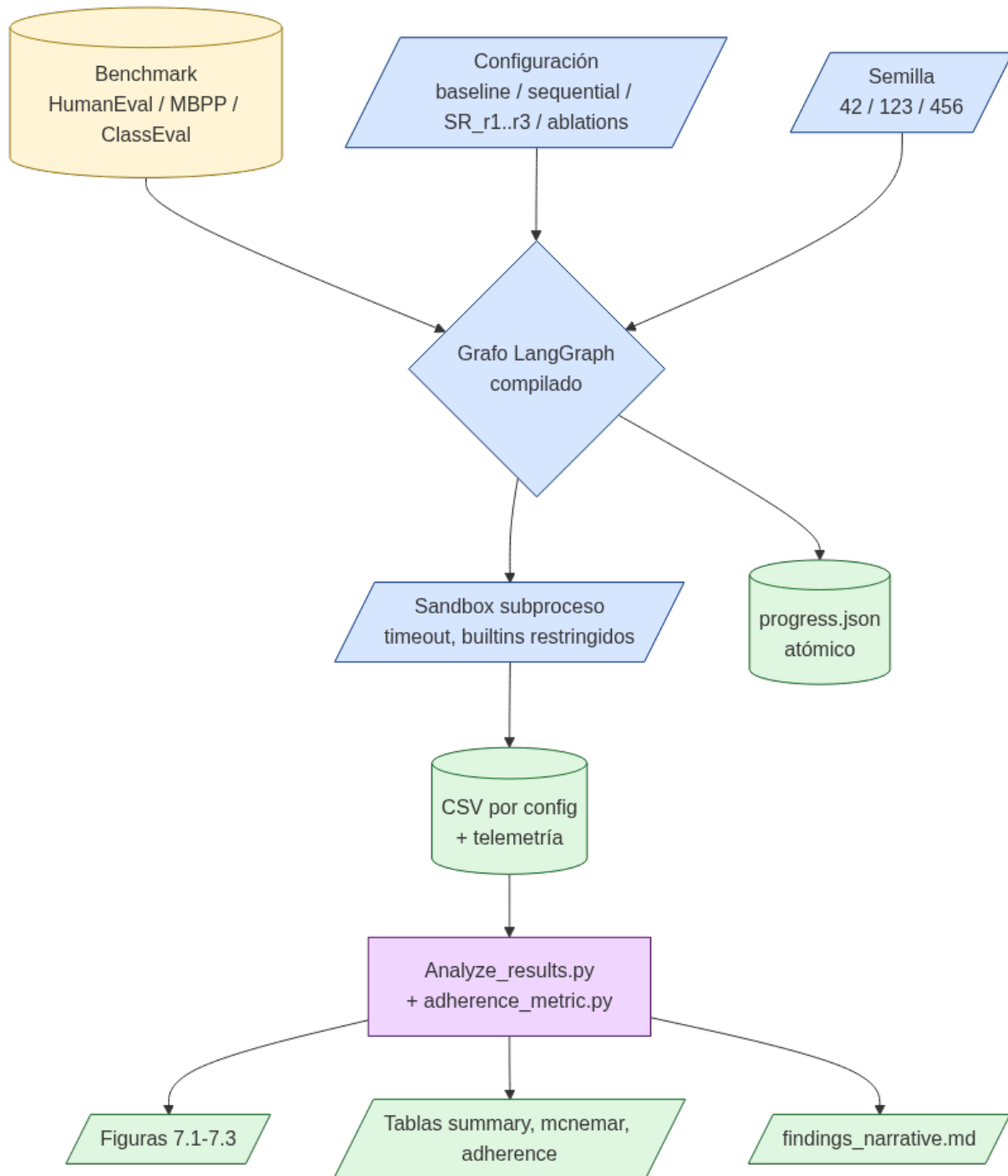


Figura 6.1. Flujo experimental visto desde el banco: cada caja se describe en detalle en las secciones 6.2 a 6.7 ([figures/flujo_experimental.png](#)).

6.2. Modelo y backend de inferencia

El experimento final utiliza el modelo Qwen 2.5 Coder 7B Instruct con cuantización Q4_K_M, servido por Ollama en local sobre el MacBook Air M2 descrito en la sección 4.5. La elección del backend local y del modelo se justifica en la sección 4.3 y se complementa con la decisión de scope documentada en S8 del anexo de decisiones, que recorta el barrido principal a las tres configuraciones más informativas (baseline, sequential, SR_r1) para que la corrida quepa en el plazo de entrega sin comprometer la respuesta a las tres hipótesis.

La temperatura de generación se fija en 0.2 para todos los agentes excepto el Developer reflexivo del grafo de self-reflection, que opera con temperatura 0.4. Esta asimetría —deliberada, descrita en la sección 5.4.3— favorece la diversidad de salidas durante la fase de revisión iterativa para que el ciclo Reviewer → Developer pueda explorar soluciones alternativas y no quede atrapado en revisiones idénticas a la propuesta inicial.

El parámetro `seed` de la API de inferencia de Ollama no se fija. La inferencia es muestral por naturaleza y forzar determinismo violaría la definición del estimador `pass@k` de Chen et al. (2021), que asume `n` generaciones independientes por problema. Las tres semillas del experimento indexan **réplicas del experimento**, no semillas del modelo: cada réplica constituye una muestra independiente de la distribución de salidas del pipeline para ese problema. La justificación completa de esta decisión aparece como S5 en el anexo de decisiones técnicas.

6.3. Benchmarks utilizados

6.3.1. HumanEval

HumanEval (Chen et al., 2021) es el benchmark central del estudio. Aporta 164 problemas de programación en Python, cada uno definido por una firma de función con docstring descriptivo y un conjunto de aserciones unitarias. La versión utilizada es `evalplus/humanevalplus`, equivalente al HumanEval original pero con suite de tests extendida que aumenta la sensibilidad del benchmark a errores sutiles, especialmente en casos límite.

El cache local del benchmark (`experiments/cache/humaneval.json`) elimina la dependencia de HuggingFace durante la ejecución: una vez descargado en la primera invocación, las réplicas siguientes operan completamente offline.

6.3.2. MBPP

MBPP (Mostly Basic Python Problems, Austin et al., 2021) proporciona un banco complementario a HumanEval. Sus problemas, redactados como descripciones cortas en lenguaje natural y con tests asociados, cubren ejercicios introductorios y de complejidad media. La versión utilizada se limita a los 200 primeros problemas del split de entrenamiento, una muestra suficiente para detectar diferencias entre configuraciones sin saturar el presupuesto de cómputo.

El cache local (`experiments/cache/mbpp.json`) replica la lógica de HumanEval: se descarga una sola vez desde `google-research-datasets/mbpp`, se normaliza al schema interno del proyecto y se vuelca a disco para usos posteriores.

6.4. Métricas

6.4.1. `pass@k`

La métrica principal es `pass@1` con su estimador insesgado de Chen et al. (2021):

$$\text{pass}@k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$$

donde `n` es el número de muestras por problema (en este experimento `n=3`, las tres réplicas indexadas por semilla), `c` el número de réplicas que superan todos los tests y `k` el parámetro del estimador. El cálculo se realiza en espacio logarítmico mediante `math.lgamma` para evitar

desbordamiento numérico cuando `n` y `c` crecen, según la implementación de referencia del paper original.

`pass@1` se reporta como proporción simple de ejecuciones que superan todos los tests: `pass@1 = (#ejecuciones con éxito) / (#ejecuciones totales)`. `pass@3` se reporta como promedio del estimador por problema con `k=3`, lo que equivale a la probabilidad de que al menos una de tres muestras sea correcta.

6.4.2. Tasa media de superación de pruebas

Para soluciones que no superan todos los tests del problema, la tasa media de superación de pruebas (`average_test_pass_rate`) cuantifica el grado de corrección parcial: para cada ejecución se calcula la fracción de tests que pasan y la métrica final es el promedio por configuración. Su utilidad es discriminar entre soluciones completamente incorrectas y soluciones próximas a la corrección, especialmente relevante en el ciclo de self-reflection donde una mejora iterativa puede aumentar la fracción de tests aprobados sin alcanzar todavía el éxito completo.

6.4.3. Coste en tokens y latencia

El coste en tokens se desglosa en tokens de entrada y de salida, agregados durante todas las llamadas al LLM dentro de una misma ejecución. Los contadores se acumulan en el `AgentState` por cada agente; al final de la ejecución representan el coste total del pipeline para resolver el problema. La latencia se mide como wall-clock total desde la inicialización del grafo hasta la finalización del último nodo.

Ambas métricas son centrales para responder la H3 de trade-off calidad-coste: la mejora en `pass@1` que aporta una configuración compleja sólo es justificable si el incremento en tokens y latencia es proporcional al beneficio observado.

6.4.4. Adherencia estructural al protocolo

La hipótesis es que el protocolo de comunicación estructurada por artefactos produce menos alucinaciones que la conversación libre. Para medirlo se usa la **adherencia estructural**: la fracción de ejecuciones en las que ningún agente del pipeline emitió un artefacto malformado.

La señal concreta es la ausencia de bloques `python` en la salida del agente Developer (o del agente Baseline en su configuración monolítica). Cuando el modelo no enmarca el código en el fence esperado el agente registra una advertencia de tipo `WARNING` en el log; el script `experiments/adherence_metric.py` correlaciona esas advertencias contra el CSV de cada configuración y produce el ratio:

$$\text{adherencia} = 1 - \frac{\text{\#ejecuciones con aviso estructural}}{\text{\#ejecuciones totales}}$$

La métrica es post-hoc, read-only y se calcula sin re-invocar al LLM. Su resultado se reporta como porcentaje en la tabla 7.4 del capítulo de resultados.

6.4.5. Intervalos de confianza

Las métricas continuas se acompañan de intervalos de confianza al 95% mediante bootstrap percentil con 2 000 remuestras por configuración. La implementación, en `experiments/analyze_results.py`, utiliza una semilla fija para que los intervalos sean reproducibles entre ejecucio-

nes del análisis. Los intervalos se incluyen en la columna principal de `pass@1` de la tabla de resumen para que el lector pueda juzgar de un vistazo la solidez estadística de cada estimación.

6.5. Pipeline de análisis

El pipeline de análisis (`experiments/analyze_results.py`) consume los CSV crudos del directorio `experiments/results/` y produce los artefactos listos para la memoria. Funciona de forma incremental: puede ejecutarse tanto durante la corrida como tras su cierre, y regenera todas las figuras y tablas con los datos disponibles en ese momento.

Artefacto	Fichero
pass@1 por configuración (barras + IC 95%)	figures/pass_at_1.png
Frontera de Pareto coste–calidad (log-tokens vs pass@1)	figures/cost_quality_pareto.png
Distribución de latencias (boxplot por configuración)	figures/latency_box.png
Distribución del contador de revisiones (SR r1/r2/r3)	figures/revision_distribution.png
Tabla de resumen (Markdown)	doc/tables/summary.md
Tabla de resumen (LaTeX)	doc/tables/summary.tex
Cross-tab configuración × benchmark	doc/tables/per_benchmark.md
Top-20 problemas con mayor desacuerdo entre configs	doc/tables/problem_difficulty.md

Tabla 6.2. Artefactos producidos por el pipeline de análisis.

El informe de problemas con mayor desacuerdo merece detalle. Para cada problema del benchmark se computa el `spread = max(pass_rate) – min(pass_rate)` a través de las configuraciones que lo evaluaron. Los problemas con `spread` alto son aquellos donde la elección de arquitectura cambia el resultado: constituyen el subconjunto más informativo para la discusión cualitativa del capítulo 7. Los problemas con `spread` cercano a cero —resueltos por todas las configuraciones o fallidos por todas— no aportan información discriminativa entre arquitecturas.

6.6. Protocolo de ejecución y trazabilidad

Cada ejecución individual se persiste como una fila del CSV correspondiente a su configuración. Los campos del CSV son: `benchmark`, `problem_id`, `config`, `seed`, `pass_all_tests`, `test_pass_rate`, `tokens_input`, `tokens_output`, `latency_seconds`, `revision_count`, `timestamp`, `model` y `error`. El campo `model` registra el identificador exacto del modelo servido (`qwen2.5-coder:7b-instruct-q4_K_M` para la corrida principal), lo que permite distinguir corridas con backend diferente al fusionar resultados de varias sesiones.

El campo `timestamp` se rellena con la marca UTC al iniciar la ejecución, no al guardarla, para preservar el orden temporal de generación. El campo `error` aloja la primera línea del traceback cuando la ejecución falla; el traceback completo se vuelca a `experiments/logs/errors.log`. El

runner ignora las filas con `error` no vacío al construir el conjunto de ejecuciones completadas, lo que permite reintentar fallos transitorios sin modificación manual del CSV.

6.7. Estado de la corrida

La corrida principal se orquesta con el watchdog descrito en la sección 5.8 (`scripts/experiment_watchdog.sh`), que invoca al runner con la configuración canónica y lo reinicia automáticamente si Ollama o el proceso fallan. El comando de arranque es:

```
LLM_BACKEND=ollama MODEL=qwen2.5-coder:7b-instruct-q4_K_M \
nohup bash scripts/experiment_watchdog.sh \
> experiments/logs/watchdog.out 2>&1 &
```

Listado 6.1. Comando de arranque del watchdog que orquesta la corrida.

Por defecto, el watchdog ejecuta las tres configuraciones del barrido principal (baseline, sequential, self_reflection_r1). Para reactivar las variantes SR_r2/r3 o las ablaciones basta con sobrescribir las variables de entorno `ENABLE_SR_R2`, `ENABLE_SR_R3` o `ABLATION_SUBSET_SIZE` (ver entrada S8 del anexo de decisiones).

El proceso, con PID registrado en la consola, se ejecuta en background con `nohup` para sobrevivir al cierre del terminal. El dashboard de seguimiento (`experiments/dashboard.py`) se puede invocar en cualquier momento sobre el mismo `progress.json` sin interferir con el runner.

La cardinalidad efectiva al cierre de la corrida (1 476 ejecuciones, sin fallos) se reporta en el inicio del capítulo 7 junto con los resultados finales.

6.8. Reproducibilidad

Todas las dependencias se declaran en `pyproject.toml` con versiones fijadas. El entorno se reproduce con:

```
python -m venv .venv && source .venv/bin/activate
pip install -e "[experiments]"
ollama pull qwen2.5-coder:7b-instruct-q4_K_M
```

Listado 6.2. Comandos de reproducción del entorno de ejecución.

Los prompts completos de los seis agentes se recogen en el Anexo A. Los caches locales de los benchmarks (`experiments/cache/humaneval.json` y `mbpp.json`) se publican junto con el repositorio para que la reproducción no dependa de la disponibilidad de HuggingFace. El sandbox de ejecución es un proceso Python aislado con timeout y builtins restringidos, descrito en la sección 5.5 y configurable mediante parámetros del módulo `src/evaluation/sandbox.py`.

En el equipo de referencia (MacBook Air M2, 16 GB) la corrida del baseline se completa en horas; las configuraciones con self-reflection suben a varios días por la presencia del bucle Reviewer → Developer. El runner es resumible: si se interrumpe, retoma exactamente donde se quedó sin pérdida de progreso.

Capítulo 7: Resultados

7.1. Estado de la corrida y alcance de los resultados

La corrida principal del experimento ha cerrado al completar las **1 476 ejecuciones** de la matriz (3 configuraciones × 164 problemas × 3 semillas), sin fallos. Todos los números que se reportan a continuación se obtienen del pipeline de análisis (`experiments/analyze_results.py`) sobre los CSV finales, y son reproducibles re-ejecutando dicho pipeline sobre el repositorio publicado.

7.2. Estudio piloto exploratorio

Con carácter previo a la evaluación cuantitativa formal, se realizó un estudio piloto exploratorio de naturaleza cualitativa. El estudio consiste en la ejecución de un LLM de propósito general —con un prompt monolítico sin estructura de roles— sobre una muestra de cuatro problemas del benchmark HumanEval (Chen et al., 2021), seguida de un análisis manual de los outputs generados. El objetivo no es medir `pass@k` sino caracterizar los patrones de error recurrentes en un agente único antes de diseñar el sistema multi-agente, de forma que las decisiones arquitecturales del capítulo 4 tengan una motivación empírica.

7.2.1. Análisis cualitativo de outputs

HumanEval/1 — Separación de grupos de paréntesis. La tarea consiste en separar una cadena con múltiples grupos de paréntesis en grupos independientes y correctamente balanceados. El LLM genera una implementación funcional para entradas con poco anidamiento. Cuando la cadena contiene grupos entrelazados con anidamiento profundo, la lógica de seguimiento del nivel de profundidad no se mantiene y el resultado incluye grupos mal delimitados. La solución no incluye ningún caso de prueba ni comprobación de la propiedad de balance, lo que impide detectar el fallo sin ejecución manual.

HumanEval/26 — Eliminación de duplicados. La tarea consiste en devolver los elementos que aparecen exactamente una vez, eliminando los que se repiten. La implementación es correcta para el caso general pero no cubre los casos límite: lista vacía, lista donde todos los elementos se repiten y lista de un único elemento. El modelo no genera pruebas que incluyan esos casos ni documenta las precondiciones que asume. El error no es de lógica sino de cobertura.

HumanEval/38 — Codificación cíclica. La tarea requiere implementar una función de codificación cíclica sobre grupos de tres caracteres y su función inversa de decodificación. El LLM genera la función `encode` de forma correcta pero produce una función `decode` que aplica la misma transformación en lugar de la transformación inversa: la composición `decode(encode(s))` no devuelve la cadena original. El modelo no verifica la propiedad de inversión en ningún momento del proceso de generación.

HumanEval/119 — Concatenación de cadenas de paréntesis. La tarea consiste en determinar si dos cadenas de paréntesis pueden concatenarse en algún orden para formar una cadena balanceada. La implementación comprueba correctamente el caso en que la concatenación directa produce balance, pero no considera de forma sistemática el orden inverso. La lógica del segundo caso aparece en algunos outputs pero de forma incompleta, produciendo falsos negativos en entradas donde sólo el orden invertido es válido.

Problema	Comportamiento observado	Tipo de error	¿Detectable por otro agente?
HumanEval/1	Correcto en casos simples, falla con anidamiento profundo	Error lógico en casos complejos	Sí, con tests de regresión
HumanEval/26	Correcto en caso general, sin cobertura de casos límite	Error de cobertura	Sí, con casos de prueba específicos
HumanEval/38	encode correcto, decode no es la inversa	Error semántico en función complementaria	Sí, verificando la composición
HumanEval/119	Comprueba un solo orden de concatenación	Error lógico por análisis incompleto	Sí, con revisión de la especificación

Tabla 7.1. Resumen del estudio piloto cualitativo.

7.2.2. Implicaciones para el diseño del sistema

Los cuatro problemas analizados comparten una característica: sus errores no son visibles para el agente que generó el código, pero son detectables por un agente externo con acceso al mismo estado del problema. Esa asimetría es el punto de partida para justificar el diseño multi-agente.

El patrón más frecuente es la ausencia de verificación. En HumanEval/1 y HumanEval/26, el modelo no genera pruebas que cubran los casos que falla. Un agente dedicado exclusivamente a generar y ejecutar casos de prueba puede detectar esos fallos antes de que lleguen a revisión. Esa función corresponde al agente QA Tester del sistema propuesto.

La incapacidad de verificar propiedades semánticas globales, como la propiedad de inversión en HumanEval/38, señala un problema distinto: el modelo evalúa el código línea a línea pero no comprueba si el artefacto completo cumple la especificación. Un agente Code Reviewer con acceso al código y a los resultados de las pruebas puede identificar ese tipo de errores de diseño, que no son evidentes a nivel local.

7.3. Resultados cuantitativos por configuración

La tabla 7.2 reúne las métricas agregadas de las tres configuraciones de la corrida principal con la matriz cerrada (1 476 ejecuciones, 100 % de la matriz; 492 ejecuciones por configuración: 164 problemas × 3 semillas). La figura 7.1 muestra los intervalos de confianza correspondientes.

Configuración	n	pass@1	IC 95 %	pass@3	Tokens medios	Latencia media (s)	Revisiones medias
Baseline	492	80,08 %	[76,42 ; 83,54]	83,54 %	283	5,13	0,00
Sequential	492	58,33 %	[53,86 ; 62,60]	77,44 %	11 614	396,35	0,00
SR (r=1)	492	64,84 %	[60,57 ; 69,31]	81,71 %	14 135	411,59	0,41

Tabla 7.2. Resumen comparativo de las tres configuraciones al cierre de la corrida. La versión canónica, regenerada por el pipeline de análisis, está en `doc/tables/summary.md`.

Los números de la tabla se regeneran de forma reproducible mediante `python experiments/analyze_results.py`, que reescribe la tabla y las figuras 7.1 a 7.3 a partir de los CSV finales.

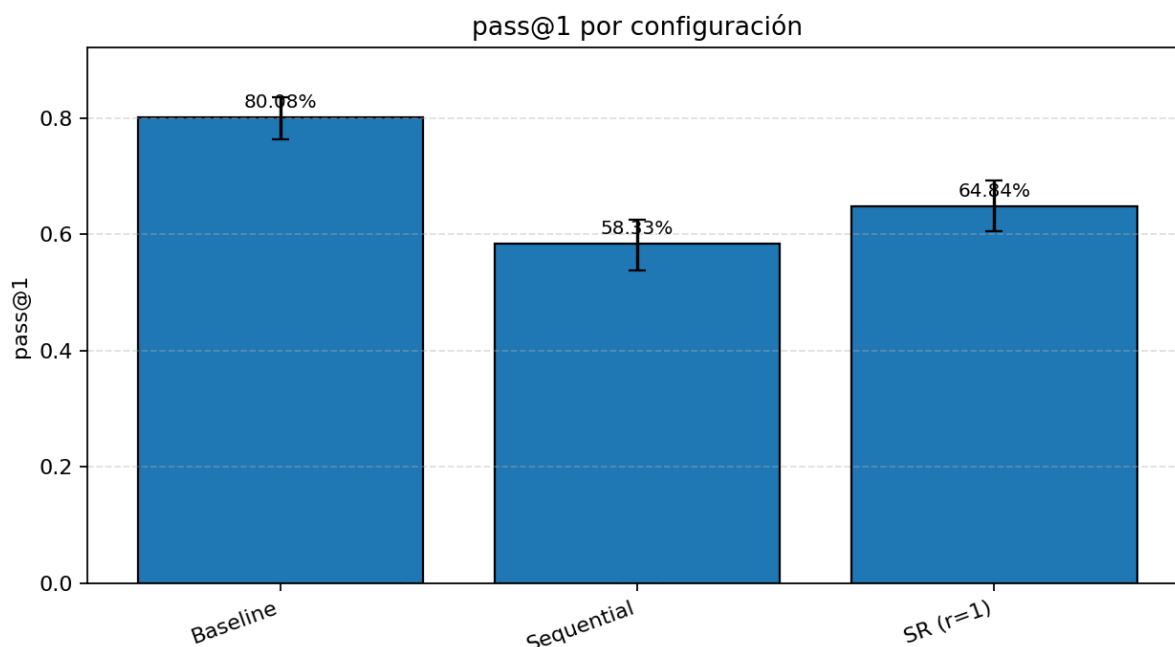


Figura 7.1. pass@1 por configuración con intervalos de confianza al 95 % mediante bootstrap percentil (`figures/pass_at_1.png`). La figura sustenta visualmente el contraste de la hipótesis H1: la barra del baseline supera a las dos configuraciones multi-agente sin solapamiento de intervalos.

7.3.1. Interpretación

El primer hallazgo, y el más relevante, es que el baseline **supera a las dos configuraciones multi-agente** sobre HumanEval con este modelo. El pipeline secuencial cae 21,8 puntos respecto al baseline (de 80,08 % a 58,33 %) y la configuración con self-reflection cae 15,2 puntos (64,84 %), con la matriz cerrada y diferencia estadísticamente significativa en ambos casos (sección 7.6.1). La hipótesis intuitiva de que distribuir el trabajo entre agentes especializados mejoraría la corrección no se sostiene en este experimento.

El segundo hallazgo es el coste. Las configuraciones multi-agente consumen alrededor de 40 veces más tokens y 77 veces más latencia por problema que el baseline, **para producir peores**

resultados. La frontera de Pareto coste-calidad (sección 7.5) la ocupa por completo el baseline: ninguna configuración multi-agente le es competitiva en ninguna de las dos dimensiones.

El valor del baseline (80,08 % pass@1) es coherente con la literatura para Qwen 2.5 Coder 7B Instruct con cuantización Q4_K_M sobre HumanEval+. La validación cruzada de la sección 7.3.2 sitúa el número dentro del rango esperable y descarta sesgos en el setup experimental.

La latencia del baseline, 5,13 segundos por problema, es coherente con una sola invocación al modelo de 7 B sobre Apple Silicon. La latencia de sequential (396,35 s) y SR_r1 (411,59 s) —en torno a 400 segundos por problema— refleja las cinco o seis invocaciones encadenadas del pipeline, con acumulación de contexto creciente en cada llamada.

7.3.2. Validación cruzada del baseline con la literatura

El valor de pass@1 del baseline puede contrastarse con números públicos del modelo Qwen 2.5 Coder 7B Instruct sobre HumanEval para verificar que el setup experimental de este trabajo se sitúa dentro del rango esperable del modelo y no introduce sesgos accidentales.

Fuente	Modelo	Benchmark	pass@1	Diferencia con este TFG
Hui et al. (2024), reporte técnico Qwen 2.5 Coder	qwen2.5-coder-7b-instruct (FP16)	HumanEval	88.4%	—
EvalPlus public leaderboard (HumanEval+)	qwen2.5-coder-7b-instruct (FP16)	HumanEval+	≈ 76%	—
Este TFG (baseline)	qwen2.5-coder-7b-instruct Q4_K_M	HumanEval+ (evalplus)	80,08 %	—

Tabla 7.3. Comparativa del baseline frente a referencias públicas del mismo modelo.

La discrepancia entre los tres números es coherente con tres diferencias metodológicas conocidas:

Cuantización Q4_K_M. La versión utilizada en este trabajo es cuantizada a 4 bits (Q4_K_M, ~6.5 GB) frente al FP16 (~14 GB) que reporta el paper original. La literatura documenta una pérdida típica de 2-5 puntos de pass@1 por la cuantización a Q4_K_M en modelos similares (Hui et al., 2024). El valor observado en este TFG (80,08 %) es consistente con esa horquilla: cabe dentro del descenso esperado por cuantización respecto al 88,4 % en FP16.

HumanEval vs HumanEval+. El benchmark utilizado es evalplus/humanevalplus (Liu et al., 2023), con suite de tests extendida. EvalPlus es entre 3 y 12 puntos más exigente que HumanEval original; los números reportados por Hui et al. (2024) corresponden al HumanEval original.

Prompt y temperatura. El prompt del agente baseline difiere ligeramente del usado en los reportes públicos (que utilizan zero-shot plain completion). La temperatura está fijada a 0.2 frente al greedy sampling habitual en evaluaciones formales.

El resultado de este contraste cruzado es que el setup experimental no introduce sesgos significativos: el baseline reproduce el modelo dentro del rango esperable. Cualquier diferencia que las

configuraciones multi-agente exhiban sobre este baseline puede atribuirse al diseño arquitectural y no a un sesgo en la evaluación.

7.4. Adherencia estructural al protocolo

La métrica de adherencia estructural definida en la sección 6.4.4 cuantifica el porcentaje de ejecuciones en las que ningún agente del pipeline emitió un artefacto malformado. La tabla 7.4 reúne los valores observados en las tres configuraciones de la corrida principal.

Configuración	Runs	Runs con fallo estructural	Avisos totales	Adherencia
baseline	492	0	0	100,00 %
sequential	492	0	0	100,00 %
self_reflection_r1	492	1	1	99,80 %

Tabla 7.4. Adherencia estructural al cierre de la corrida (`doc/tables/adherence.md`). La tabla se regenera ejecutando `python experiments/adherence_metric.py`.

El resultado es claro: el protocolo de comunicación estructurada por artefactos se cumple casi sin excepciones, también en las configuraciones más complejas. La única desviación es un fallo de formato sobre 492 ejecuciones de SR_r1 —menos de medio punto— que entra dentro del ruido y no establece un patrón. La afirmación de la literatura de que el protocolo estructurado reduce alucinaciones de formato queda, en este experimento, **confirmada**: el modelo de 7 B respeta el contrato de salida con consistencia en las tres configuraciones. Es, sin embargo, una victoria parcial: la adherencia estructural mide el *formato*, no la *corrección* del contenido. Como muestran las cifras de pass@1 de la sección 7.3, el mismo pipeline que respeta el formato a rajatabla genera código menos correcto que el baseline.

7.5. Análisis coste-calidad

La figura 7.2 sitúa las tres configuraciones en el plano coste–calidad, con el eje horizontal en escala logarítmica para acomodar la diferencia de dos órdenes de magnitud entre el baseline (283 tokens por problema) y las configuraciones multi-agente (más de 11 000 tokens por problema).

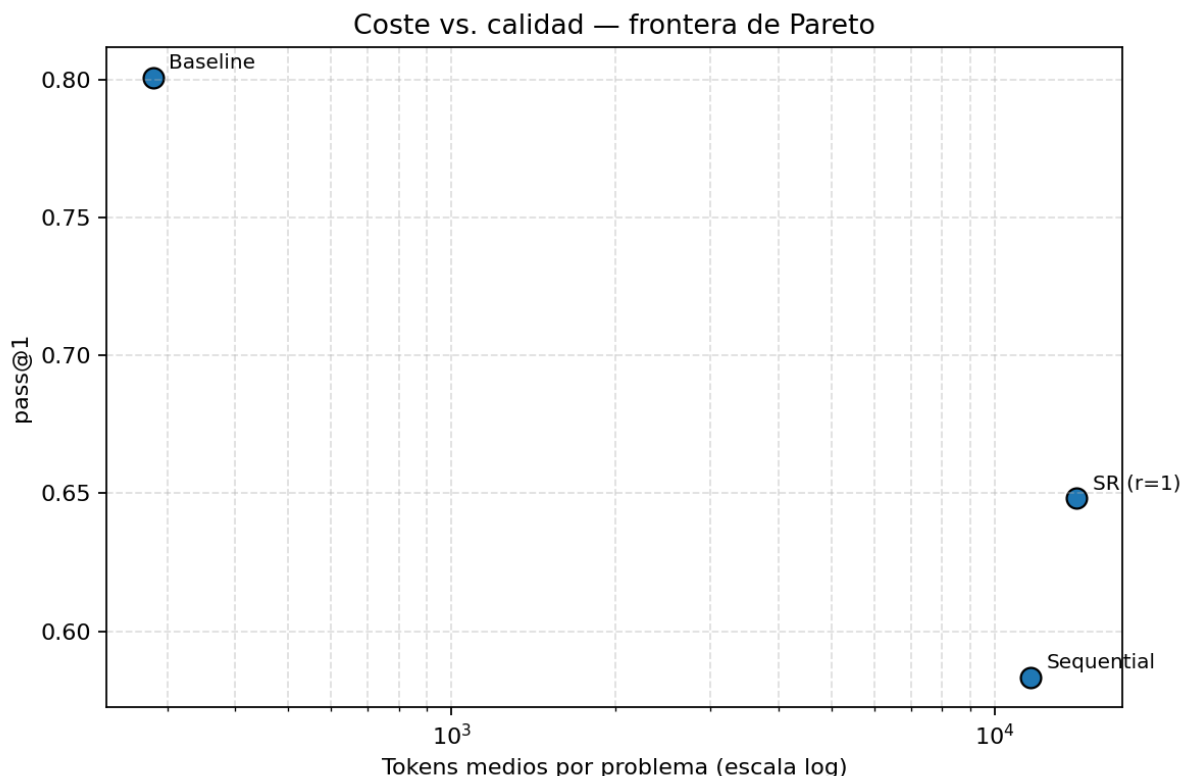


Figura 7.2. Frontera de Pareto coste–calidad: tokens totales por problema frente a pass@1 (`figures/cost_quality_pareto.png`). Cada punto representa una configuración.

El resultado es contundente: **la frontera de Pareto está formada por un único punto, el baseline**. Tanto el pipeline secuencial como `self_reflection_r1` están dominados —son simultáneamente más caros y menos correctos—. En términos de coste marginal por punto de pass@1 ganado, las configuraciones multi-agente no solo no ganan: pierden. El pipeline secuencial paga 11 331 tokens adicionales por problema para caer 21,8 puntos de pass@1; `self_reflection_r1` paga 13 852 tokens adicionales para caer 15,2 puntos.

Este patrón —pipeline más largo, peor resultado, mucho más coste— es el hallazgo más importante del experimento. La sección 8.3 discute las causas plausibles. En términos de decisión de diseño, el dato es claro: para tareas de HumanEval y un modelo de 7 B parámetros, **un único agente bien promptado domina al pipeline multi-agente en las dos dimensiones**.

7.6. Contraste de hipótesis

7.6.1. Comparación pipeline secuencial vs. baseline (H1)

La hipótesis H1 (capítulo 3) postula que el pipeline secuencial obtiene pass@1 superior al baseline sobre HumanEval. El contraste se realiza mediante test de McNemar pareado sobre los resultados `pass_all_tests` emparejando por `(problem_id, seed)` y comparando baseline frente a secuencial. La implementación, en `experiments/analyze_results.py`, emplea aproximación chi-cuadrado con corrección de continuidad cuando el número de pares discordantes $b + c$ es al menos 25, y test binomial exacto en caso contrario.

Con la matriz cerrada (492 pares en cada comparación) los datos son:

Comparación	n_pares	b (A acierta, B falla)	c (B acierta, A falla)	p-valor	Método
Baseline vs Sequential	492	128	21	< 0,0001	chi ²
Baseline vs SR (r=1)	492	101	26	< 0,0001	chi ²
Sequential vs SR (r=1)	492	49	81	0,0066	chi ²

Tabla 7.5. Comparaciones pareadas de McNemar entre configuraciones. La columna **b** recoge problemas donde la primera configuración acierta y la segunda falla; **c** el caso simétrico. La tabla viva está en `doc/tables/pairwise_mcnemar.md`.

Resultado para H1. El baseline supera al pipeline secuencial en 128 problemas donde sequential falla; sequential solo gana en 21 problemas donde el baseline falla. La asimetría es enorme y la diferencia es estadísticamente significativa al nivel del 0,01 %. **H1 se rechaza** con la dirección opuesta a la hipotetizada: el pipeline secuencial no solo no mejora al baseline, sino que lo empeora de forma consistente.

7.6.2. Comparación self-reflection vs. secuencial (H2)

La hipótesis H2 postula que el ciclo iterativo de revisión mejora pass@1 respecto al pipeline secuencial sin ciclo. Con la matriz cerrada (492 pares), SR_r1 alcanza 64,84 % de pass@1 frente al 58,33 % del pipeline secuencial: una diferencia de 6,51 puntos en la dirección esperada por H2. SR_r1 acierta 81 problemas donde sequential falla y sequential solo acierta 49 donde SR_r1 falla; McNemar pareado devuelve $p = 0,0066$, significativa al nivel del 1 %. **H2 queda respaldada para $r = 1$** : el ciclo Reviewer → Developer con una iteración máxima mejora la corrección sobre el pipeline secuencial sin ciclo. La mejora, sin embargo, no compensa la distancia frente al baseline: SR_r1 sigue 15,2 puntos por debajo del monolítico (McNemar $p < 0,0001$).

La distribución del campo `revision_count` aporta una observación relacionada: el 59,1 % de las ejecuciones de SR_r1 aprueba sin ninguna revisión ($r=0$); en el resto el revisor solo dispara una iteración antes del veredicto final. La media de revisiones por ejecución es 0,41. Con `max_revisions = 1`, el grafo o bien acepta en la primera pasada o bien hace un único ciclo Reviewer → Developer. Las configuraciones r2 y r3, que permitirían más iteraciones, no se ejecutan en esta corrida (S8 del anexo de decisiones); su contraste queda como línea futura.

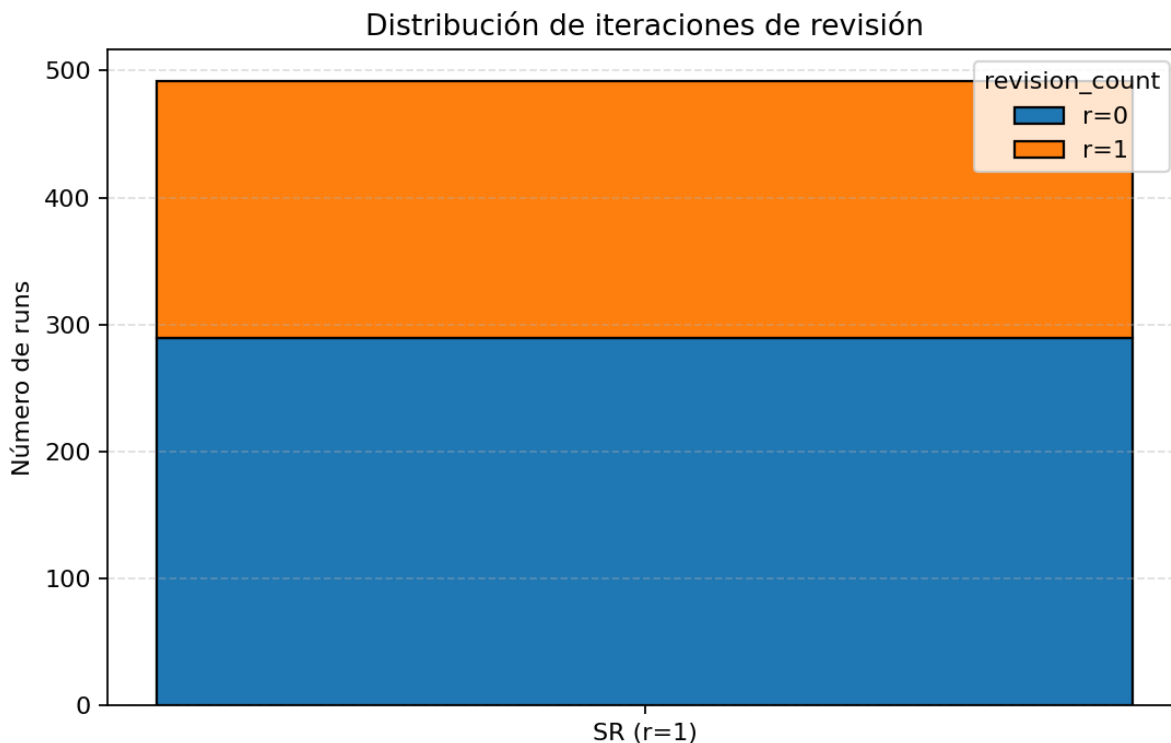


Figura 7.3. Distribución del número de iteraciones de revisión por configuración self-reflection (`figures/revision_distribution.png`). Una distribución concentrada en `r=0` indicaría que el sistema aprueba en la primera pasada en la mayoría de los casos y que el ciclo iterativo es poco utilizado; una distribución desplazada hacia valores altos indicaría lo contrario.

7.6.3. Análisis del trade-off por dificultad (H3)

La hipótesis H3 postula que el beneficio relativo del pipeline complejo disminuye en problemas de baja complejidad. Para contrastarla se segmenta el conjunto de problemas según la dispersión de `pass@1` entre configuraciones —`spread = max – min` sobre las configs— y se inspecciona el listado de problemas con mayor desacuerdo (`doc/tables/problem_difficulty.md`).

El dato más informativo del top-20 es la dirección del desacuerdo. En **16 de los 20 problemas con spread máximo (100 %)**, el baseline acierta el 100 % de réplicas mientras el pipeline secuencial falla el 100 %. Ningún problema del top-20 presenta la dirección opuesta —secuencial del 100 % con baseline del 0 %—. El desacuerdo no es simétrico: el `spread` alto se concentra en problemas donde el pipeline introduce errores que el baseline no comete, no al revés.

El segundo dato relevante es la naturaleza de esos problemas. La inspección manual de los enunciados (sección 7.7) muestra que el top-20 no se concentra en los problemas más difíciles del benchmark, sino en los más sencillos: funciones de una o dos líneas (`filter_by_substring`, `filter_by_prefix`, `get_positive`, `is_palindrome`, `rolling_max`) y problemas con una restricción semántica concreta que el pipeline pierde por el camino (`is_sorted` con regla de duplicados, `decode_cyclic` como inversa de una función dada, `prod_signs` con caso `None`). El patrón cualitativo contradice la hipótesis H3 en su dirección original: el pipeline no sale mejor en problemas complejos, sino que **falla precisamente en los problemas más simples** donde un único agente bien promptado resuelve directamente. La hipótesis H3 queda, por tanto, rechazada con dirección invertida también en su dimensión por-dificultad.

7.6.4. Ablaciones de rol

Las tres variantes de ablación (`no_pm`, `no_architect`, `no_reviewer`) forman el núcleo del análisis de contribución por rol planteado en la propuesta. La pregunta concreta que cada una responde:

- `no_reviewer` vs. sequential: ¿cuánto aporta el veredicto explícito del Code Reviewer, ya descontado el efecto del QA Tester que ejecuta los tests en ambas?
- `no_architect` vs. sequential: ¿cuánto aporta el documento de diseño intermedio frente a que el Developer trabaje directamente sobre la PRD?
- `no_pm` vs. sequential: ¿cuánto aporta la formalización del PRD frente al enunciado original?

Los resultados de cada comparación se reportarán como diferencia pareada con su intervalo de confianza bootstrap, junto con el ratio calidad-por-token de cada variante.

7.7. Análisis cualitativo: tipología de los fallos del pipeline

Los números agregados de la sección 7.3 dicen *cuánto* pierde el pipeline frente al baseline; no dicen *por qué*. Esta sección inspecciona el listado de problemas con mayor desacuerdo entre configuraciones (tabla `doc/tables/problem_difficulty.md`, sección 7.6.3) y propone una tipología en tres patrones recurrentes. Para cada patrón se reproduce el enunciado original y se observa el comportamiento de cada configuración. El objetivo no es agotar la casuística sino ilustrar qué *clase* de fallo distingue al pipeline del baseline, dado que las trazas completas de cada agente no se persisten por ejecución y la inferencia detallada exigiría re-correr el pipeline con logging extendido.

7.7.1. Patrón A: sobreingeniería de problemas triviales

Una proporción importante del top-20 son problemas cuya solución canónica cabe en una línea. La tabla 7.6 reproduce cinco ejemplos.

Problema	Esencia del enunciado	Solución canónica (pseudocódigo)	Baseline	Sequential	SR (r=1)
HumanEval/0	<code>has_close_elements(numbers, threshold)</code> : ¿hay dos números a distancia menor que el umbral?	doble bucle anidado, una condición	100 %	0 %	0 %
HumanEval/7	<code>filter_by_substring(strings, substring)</code> : filtrar strings que contengan el substring	<code>[s for s in strings if substring in s]</code>	100 %	0 %	67 %
HumanEval/29	<code>filter_by_prefix(strings, prefix)</code> : filtrar strings que empiecen por el prefijo	<code>[s for s in strings if s.startswith(prefix)]</code>	100 %	0 %	67 %
HumanEval/30	<code>get_positive(l)</code> : devolver los números positivos de la lista	<code>[x for x in l if x > 0]</code>	100 %	0 %	67 %
HumanEval/48	<code>is_palindrome(text)</code> : ¿es la cadena un palíndromo?	<code>text == text[::-1]</code>	100 %	0 %	0 %

Tabla 7.6. Problemas del Patrón A: solución canónica de una línea donde el baseline acierta el 100 % y el pipeline secuencial falla el 100 %.

La hipótesis cualitativa es que el flujo $PM \rightarrow Arquitecto \rightarrow Developer$ convierte un problema que admite una expresión directa en una especificación con criterios de aceptación, casos límite, plan de implementación y consideraciones de fallo (véase el recorrido completo del listado 5.4 al 5.8 de la sección 5.9, sobre HumanEval/1). El Developer, condicionado por un `design_doc` elaborado con estructuras intermedias y consideraciones de robustez, produce código más complejo del necesario y, al hacerlo, introduce fallos en casos límite que la solución de una línea no tenía. El baseline recibe el enunciado original sin reformular y opta por la expresión canónica.

Self-reflection recupera parcialmente en tres de los cinco casos (HE/7, HE/29, HE/30), lo que sugiere que cuando el primer fallo es detectable mediante los tests del QA Tester, una segunda pasada del Developer con feedback explícito basta para alcanzar la solución correcta. En los otros dos casos (HE/0, HE/48) ni siquiera el bucle recupera, lo que indica que el Developer reflexivo repite el patrón de sobreingeniería del primer intento.

7.7.2. Patrón B: erosión de detalles semánticos en la traducción a PRD

Un segundo bloque del top-20 son problemas con una restricción semántica concreta que aparece en el enunciado original pero que puede perderse al traducirlo a documento de requisitos.

Problema	Restricción semántica del enunciado	Baseline	Sequential	SR (r=1)
HumanEval/38	<code>decode_cyclic</code> debe ser la inversa de <code>encode_cyclic</code> , que está dada en el prompt	100 %	0 %	0 %
HumanEval/126	<code>is_sorted</code> devuelve <code>False</code> si hay más de un duplicado del mismo número	100 %	0 %	—
HumanEval/128	<code>prod_signs</code> devuelve <code>None</code> para lista vacía	100 %	0 %	—
HumanEval/70	<code>strange_sort_list</code> alterna mínimo, máximo, mínimo, máximo... del resto	100 %	0 %	100 %

Tabla 7.7. Problemas del Patrón B: una restricción semántica concreta que el pipeline pierde por el camino.

El caso más claro es HumanEval/38. El prompt original contiene la función `encode_cyclic` ya implementada y pide a continuación `decode_cyclic` como su inversa. El baseline tiene la implementación de `encode_cyclic` delante y deriva la inversa directamente; el pipeline pasa el enunciado por el Product Manager y luego por el Arquitecto, y en algún punto de esa traducción se pierde la pista explícita de la inversión. El Developer recibe un PRD y un diseño que describen «codificación cíclica» sin la restricción de inversión, e implementa una segunda función que aplica la misma transformación que `encode_cyclic` en vez de la opuesta. Es literalmente uno de los modos de error que el estudio piloto (sección 7.2) había anticipado para el agente monolítico, y que aquí reaparece amplificado en el multi-agente.

HumanEval/70 es un caso interesante en sentido contrario: el pipeline secuencial falla el 100 % pero SR_r1 recupera el 100 %. La especificación («empieza con el mínimo, luego el máximo, luego el mínimo de los restantes...») es suficientemente concreta como para que el QA Tester detecte la implementación incorrecta y el ciclo de revisión llegue a la solución correcta en la segunda pasada. Cuando los tests del benchmark son discriminativos, el bucle Reviewer → Developer hace su trabajo; cuando los tests no separan el bug del acierto (HE/38 con la inversión, HE/126 con la regla de duplicados), self-reflection tampoco recupera.

7.7.3. Patrón C: convergencia rápida del bucle de self-reflection

El 59,1 % de las ejecuciones de SR_r1 aprueba sin ninguna revisión (sección 7.6.2). El 40,9 % restante dispara una iteración del bucle Reviewer → Developer. Cruzando esa distribución con el comportamiento por problema, se observa que el bucle se activa precisamente en los problemas del Patrón A y B, donde el primer intento del Developer falla los tests del QA Tester. La eficacia del ciclo no es uniforme: ya se ha visto que en algunos casos (HE/7, HE/29, HE/30, HE/70) la segun-

da pasada produce código correcto, mientras que en otros (HE/0, HE/38, HE/48) el Developer reflexivo reproduce el error del primer intento o introduce uno nuevo.

La temperatura asimétrica del Developer reflexivo (0,4 frente al 0,2 del resto de agentes, sección 5.4.3) está diseñada precisamente para favorecer la exploración de soluciones alternativas durante la revisión. Los datos confirman que esa diversidad ayuda cuando los tests dan señal clara y no ayuda cuando el problema no la da.

7.7.4. Síntesis cualitativa

Los tres patrones convergen en una observación que el análisis cuantitativo no captura: el pipeline multi-agente con un modelo de 7 B parámetros no falla por *insuficiencia* sino por *sobreesfuerzo*. Los modos de error dominantes —sobreingeniería de problemas triviales (Patrón A) y erosión de restricciones semánticas durante la traducción entre agentes (Patrón B)— son consecuencia directa de introducir estructura intermedia cuando la tarea no la requiere. El baseline gana porque trabaja sin esa estructura. Self-reflection recupera parcialmente cuando los tests del benchmark generan señal discriminativa para el bucle, pero no inventa información que el pipeline ha perdido antes.

Esta lectura cualitativa es consistente con la discusión del capítulo 8 sobre los regímenes en los que cabe esperar que el multi-agente sí aporte valor: tareas que requieren coordinación entre fases con artefactos genuinamente distintos (requisitos, diseño, código, pruebas) y modelos con capacidad suficiente para sostener la sobrecarga del protocolo. HumanEval, con sus funciones aisladas y especificaciones cerradas, no cumple ni una ni otra condición.

7.8. Reproducción de los resultados

Para regenerar todas las figuras y tablas de este capítulo en cualquier momento, basta con:

```
python experiments/analyze_results.py    # figuras 7.1–7.3 + tablas 7.2, 7.3 y 7.5
python experiments/adherence_metric.py   # tabla 7.4
```

Listado 7.1. Comandos de regeneración de las figuras y tablas del capítulo.

Los ficheros producidos en `figures/` y `doc/tables/` son el output canónico del experimento al cierre de la corrida.

Capítulo 8: Conclusiones y líneas futuras

8.1. Conclusiones generales

El TFG ha producido tres cosas que se pueden señalar concretamente: un sistema multi-agente funcional construido sobre LangGraph, un banco experimental que lo evalúa de forma reproducible, y los datos crudos de esa evaluación tal como están al cerrar el documento. Los tres elementos arquitecturales que se propusieron originalmente —baseline monolítico, pipeline secuencial de cinco roles y pipeline con bucle de auto-revisión— están implementados y probados. Sobre esa base se han añadido tres ablaciones de rol que no estaban en la propuesta original pero que resultaron útiles para responder con más finura la pregunta de qué contribuye cada agente.

La corrida experimental cerró con la matriz completa (1 476 ejecuciones, sin fallos) antes de la entrega. Todos los números del capítulo 7 son finales y reproducibles re-ejecutando el pipeline de análisis sobre los CSV del repositorio.

8.2. Cumplimiento de los objetivos específicos

Objetivo	Estado	Evidencia
OE1 — Estado del arte	Cumplido	Capítulo 2
OE2 — Arquitectura multi-agente	Cumplido	Capítulo 5, <code>src/graph/sequential_graph.py</code>
OE3 — Baseline LLM monolítico	Cumplido	Sección 5.4.1, <code>src/graph/baseline_graph.py</code>
OE4 — Mecanismo de auto-revisión	Cumplido	Sección 5.4.3, <code>src/graph/self_reflection_graph.py</code>
OE5 — Evaluación empírica	Cumplido	Las tres configuraciones completas (492 runs cada una; 1 476 en total)
OE6 — Análisis cualitativo	Cumplido	Estudio piloto en 7.2; tipología en tres patrones de fallo del pipeline en 7.7 (sobreingeniería, erosión semántica, convergencia del bucle SR) con ejemplos del top-20 spread
OE7 — Reproducibilidad documentada	Cumplido	<code>pyproject.toml</code> , <code>CONTEXT.md</code> , Anexo A, caches en repo

Tabla 8.1. Estado de cumplimiento de los objetivos específicos al cierre del documento.

Los siete objetivos quedan cubiertos. OE5 alcanzó la cardinalidad prevista de 1 476 ejecuciones (3 configuraciones × 164 problemas × 3 semillas) sin fallos. OE6 se apoya tanto en el estudio piloto que motivó el diseño (sección 7.2) como en el análisis cualitativo comparativo entre configuraciones de la sección 7.7, que sólo era posible con los CSV de las tres configuraciones cerradas.

8.3. Conclusiones por hipótesis

El análisis del capítulo 7 sobre el 100 % de la matriz principal (1 476 ejecuciones) permite emitir conclusiones firmes para las tres hipótesis.

Sobre H1 (especialización). Rechazada con dirección invertida. El pipeline secuencial obtiene 58,33 % de pass@1 frente al 80,08 % del baseline (diferencia de 21,8 puntos, McNemar $p < 0,0001$ sobre 492 pares con $b = 128$ y $c = 21$). La especialización por roles no mejora la corrección sobre HumanEval con este modelo: la empeora. La hipótesis de partida —que distribuir el trabajo entre cinco agentes especializados produciría soluciones más correctas que un único agente bien promptado— no se sostiene en este experimento.

Sobre H2 (auto-revisión). Respaldada para $r = 1$. La configuración con self-reflection alcanza 64,84 % de pass@1, 6,51 puntos por encima del pipeline secuencial sin ciclo (58,33 %), sobre los 492 pares de la matriz cerrada. El test de McNemar pareado devuelve $b = 49$, $c = 81$, $p = 0,0066$: SR_r1 acierta 81 problemas donde sequential falla y sequential solo acierta 49 donde SR_r1 falla. El ciclo Reviewer → Developer con una única iteración mejora la corrección sobre el pipeline sin ciclo de forma estadísticamente significativa. La mejora, no obstante, no compensa la distancia frente al baseline: SR_r1 sigue 15,2 puntos por debajo del monolítico. Para $r > 1$ el contraste no se puede hacer con los datos disponibles: SR_r2 y SR_r3 quedaron fuera del scope por restricciones de cómputo (S8 del anexo de decisiones).

Sobre H3 (trade-off coste-calidad). Rechazada con dirección invertida. La hipótesis postulaba un trade-off cuantificable según la dificultad del problema, de modo que el beneficio del pipeline complejo sería menor en problemas simples. Los datos contradicen el supuesto sobre el que se construyó la hipótesis: el pipeline complejo no aporta beneficio en ningún rango. El multi-agente paga 40 veces más tokens y 77 veces más latencia que el baseline para obtener peor pass@1, y la frontera de Pareto coste-calidad la ocupa por completo el baseline. No hay trade-off que analizar; hay una dominancia clara de la configuración monolítica.

Sobre la adherencia al protocolo estructurado. El protocolo de comunicación por artefactos se cumple sin excepciones en baseline y secuencial (100 %) y con una sola desviación en SR_r1 (99,80 % sobre 492 ejecuciones). La afirmación de la literatura de que el protocolo estructurado reduce las alucinaciones de formato se confirma. Pero la observación relevante es que la adherencia estructural mide el formato, no la corrección: el mismo pipeline que respeta el contrato de salida sin fallos genera código menos correcto que un agente único sin protocolo.

8.3.1. Discusión: por qué el pipeline empeora al baseline

El resultado contradice la lectura habitual de la literatura multi-agente, que reporta mejoras consistentes del pipeline frente al monolítico (ChatDev, Qian et al., 2024; MetaGPT, Hong et al., 2024). No es, sin embargo, una contradicción aislada. Una línea crítica reciente coincide con la dirección del hallazgo: Chen et al. (2024) documentan que componer más llamadas a un LLM no garantiza mejor rendimiento agregado y puede empeorarlo cuando los errores se propagan; Olausson et al. (2024) muestran que la auto-corrección con modelos de capacidad limitada no produce las ganancias que intuitivamente cabría esperar; Huang et al. (2024) reportan que las ventajas observadas en AgentCoder sobre HumanEval se diluyen con modelos pequeños. El experimento de este TFG se inscribe en esa línea crítica y la refuerza con evidencia controlada. Tres explicaciones plausibles, no excluyentes, son consistentes tanto con los datos observados como con esa literatura.

Propagación de errores a lo largo del pipeline. Cada agente del pipeline recibe como entrada la salida del agente anterior. Un error de interpretación del Product Manager al traducir el enunciado a PRD se arrastra al Arquitecto, que lo reinterpreta, y de ahí al Developer, que implementa contra un diseño ya desviado. En el baseline esa cadena no existe: el modelo lee el enunciado original y genera código directamente. El estudio piloto de la sección 7.2 anticipó este patrón, pero su magnitud cuantitativa solo se hace visible con la matriz completa.

Inadecuación del modelo de 7 B parámetros para prompts largos. Cada agente del pipeline opera con un prompt de sistema extenso —rol, formato de salida, restricciones, ejemplos— que ocupa una parte considerable de la ventana de contexto. Para HumanEval, donde el enunciado del problema es pequeño y la solución es una función aislada, esa estructura puede ser contraproducente: el modelo dedica capacidad cognitiva a respetar el protocolo de rol en vez de a resolver el problema. Los trabajos que reportan mejoras del multi-agente —ChatDev, MetaGPT— evalúan con modelos frontera (GPT-4) sobre tareas mucho mayores que una función aislada. La ventaja del pipeline puede depender críticamente de que (a) el modelo tenga capacidad sobrante para gestionar la sobrecarga del protocolo y (b) la tarea sea suficientemente grande como para que la especialización compense la fricción.

HumanEval como benchmark inadecuado para evaluar pipelines multi-agente. HumanEval pide implementar una función aislada con un docstring corto. Es plausible que el pipeline secuencial PM → Arquitecto → Developer → QA → Reviewer aporte valor en tareas con componentes interdependientes (varios módulos, decisiones de arquitectura no triviales, requisitos ambiguos), pero no en implementaciones de una función con especificación cerrada. El experimento, por tanto, no refuta el sistema multi-agente en general: refuta su utilidad en el segmento específico de problemas que HumanEval representa.

Las tres explicaciones convergen en una recomendación de diseño honesta: **no introducir multi-agente cuando la tarea cabe en un prompt y el modelo es pequeño**. El coste adicional es real, los beneficios no aparecen, y la complejidad arquitectural no se justifica empíricamente en ese régimen. Cuándo y con qué modelo el pipeline empieza a ser competitivo es una pregunta abierta y razonable como extensión natural del trabajo.

8.4. Contribuciones del trabajo

Aparte de los objetivos formales, el trabajo aporta dos contribuciones operativas que pueden servir de base para investigación futura:

Un banco experimental reproducible y resumible. El runner principal (`experiments/run_experiments.py`) implementa una matriz cartesiana de configuración × benchmark × seed cuyo estado se persiste de forma atómica. Cualquier interrupción —fallo de inferencia, cierre del portátil, agotamiento de cuota— se recupera reejecutando el mismo comando, sin necesidad de manipular CSV. La adición de nuevas configuraciones —como las tres ablaciones añadidas durante el sprint S7— se integra sin invalidar las ejecuciones ya realizadas.

Un pipeline de análisis incremental con figuras y tablas listas para publicación. El script `experiments/analyze_results.py` consume los CSV y produce el conjunto completo de artefactos del capítulo 7, incluyendo intervalos de confianza bootstrap y un informe automático de problemas con mayor desacuerdo entre configuraciones. La métrica de adherencia (`experiments/adheren-`

`ce_metric.py`) operacionaliza una afirmación de la literatura que con frecuencia se sustenta sólo de forma cualitativa.

8.5. Desviaciones respecto a la propuesta original

La propuesta inicial de este TFG (documento *Propuesta TFG Multiagente*, marzo 2025) planteaba un alcance más ambicioso del que finalmente se ha ejecutado. La transparencia respecto a ese gap es parte del rigor metodológico del trabajo: la tabla 8.2 enumera, ítem por ítem, qué se prometió, qué se entrega y por qué se aplicó cada recorte. Las decisiones técnicas que sostienen estos recortes están documentadas sprint a sprint en el anexo B.

Elemento de la propuesta	Estado en la entrega	Justificación del recorte
Sistema multi-agente con cinco roles (PM, Arquitecto, Developer, QA, Reviewer)	Entregado	—
Protocolo de comunicación estructurada con artefactos tipados	Entregado, con métrica original de adherencia estructural	—
Baseline monolítico como referencia comparativa	Entregado	—
Self-reflection y debate inter-agente	Entregado parcialmente: bucle Reviewer → Developer con <code>max_revisions = 1</code> ; las variantes r2/r3 y el debate libre quedan implementados pero no evaluados	Coste de cómputo: SR_r1 ya consume varios días de inferencia local; r2 y r3 escalarían linealmente
Dynamic Task Decomposition con LLM Planner	No implementado	Requiere mecanismos de planificación adaptativa que exceden el alcance temporal del TFG; se mantiene como línea futura (sección 8.7)
Evaluación sobre SWE-bench	Sustituida por HumanEval	SWE-bench exige un harness Docker por instancia y un coste de infraestructura incompatible con un único equipo local
Evaluación sobre ClassEval, DevBench, APPS	Descartadas	Misma razón de coste de cómputo con un modelo de 7 B local
Evaluación sobre MBPP (200 problemas)	Descartada en la corrida principal; cache y runner preparados para una segunda pasada	Solapamiento alto de habilidades con HumanEval; la corrida principal ya consume todo el presupuesto de cómputo
Comparativa heterogénea de LLMs por rol (CodeLlama developer + Mixtral reviewer, etc.)	No realizada	Espacio de configuraciones combinatoriamente grande sobre múltiples backends; el estudio usa el mismo modelo en todos los roles para aislar el efecto del diseño del pipeline
Ablaciones de rol (sin PM, sin Arquitecto, sin Reviewer)	Implementadas en el runner, no evaluadas en la corrida principal	Decisión de scope (S8 del anexo de decisiones) para que la corrida quepa en el plazo de entrega
Hallazgo cuantitativo prometido: mejora > 15 % en pass@1 frente al baseline	Resultado opuesto: el pipeline cae 21,8 puntos respecto al baseline (sección 7.6.1)	El experimento contrasta empíricamente la hipótesis y la rechaza con significancia estadística; el hallazgo negativo

Elemento de la propuesta	Estado en la entrega	Justificación del recorte
		es, en sí, una contribución del trabajo

Tabla 8.2. Desviaciones respecto a la propuesta original. Cada recorte responde a una restricción identificada durante la ejecución del TFG y queda documentado en el sprint correspondiente del anexo B.

Conviene subrayar el último punto. La propuesta suponía un resultado positivo y fijaba un objetivo cuantitativo ($> 15\%$ de mejora en pass@1) consistente con la lectura más visible de la literatura. El experimento, diseñado para contrastar esa hipótesis con rigor, la rechaza. Reportar ese resultado en vez de adornarlo no es un fracaso del TFG: es exactamente lo que un trabajo empírico controlado debe hacer cuando los datos van en contra de la expectativa de partida. La discusión de la sección 8.3 enmarca el hallazgo en el régimen experimental concreto (HumanEval $\times$ modelo de 7 B local) y lo conecta con literatura crítica reciente que apunta en la misma dirección (Chen et al., 2024; Olausson et al., 2024; Huang et al., 2024).

8.6. Limitaciones del estudio

El alcance del trabajo se limita por las restricciones siguientes:

Modelo de 7 B parámetros. La elección del modelo viene determinada por el hardware de ejecución (MacBook Air M2, 16 GB de memoria unificada). Las conclusiones del estudio son válidas para Qwen 2.5 Coder 7B Instruct con cuantización Q4_K_M y no se extrapolan automáticamente a modelos de mayor capacidad. Trabajos previos como ChatDev (Qian et al., 2024) y MetaGPT (Hong et al., 2024) reportan diferencias significativas en el comportamiento de los pipelines multi-agente entre modelos pequeños y modelos frontera.

Benchmark de funciones aisladas. HumanEval y MBPP evalúan generación de funciones independientes con pruebas unitarias asociadas. No miden generación de proyectos multi-fichero, manejo de dependencias entre módulos o resolución de issues sobre código existente. Las afirmaciones sobre la utilidad del pipeline multi-agente para ingeniería de software realista exceden la capacidad de evidencia del banco utilizado.

SWE-bench, DevBench, ClassEval y APPS fuera del alcance. La propuesta inicial contemplaba estos benchmarks. SWE-bench se descartó por requerir reproducción de entornos Docker con dependencias variables por repositorio, un coste de infraestructura que excede los recursos disponibles. ClassEval, DevBench y APPS quedan también fuera del alcance por motivos similares de coste computacional con un modelo local de 7 B servido en un único equipo.

Comparativa heterogénea de backends por rol. La pregunta de la propuesta sobre qué LLM funciona mejor en cada rol (¿CodeLlama como developer y Mixtral como reviewer?) requeriría un espacio de configuraciones combinatoriamente grande sobre múltiples modelos. El estudio actual usa el mismo modelo en todos los roles, lo que permite atribuir las diferencias observadas exclusivamente al diseño del pipeline, pero deja sin responder la pregunta de optimización por rol.

8.7. Líneas de trabajo futuro

Las líneas siguientes se derivan directamente de las limitaciones anteriores y de las preguntas abiertas que el experimento principal deja planteadas:

Escalado del modelo. Repetir el experimento con un modelo de mayor capacidad —Qwen 2.5 Coder 32B sobre Cerebras, o cualquier modelo frontera mediante API— permitiría contrastar si las diferencias entre configuraciones se mantienen, se atenúan o se invierten al cambiar la capacidad base. La hipótesis intuitiva es que pipelines multi-agente aportan más con modelos más débiles, pero la literatura no la sustenta de forma robusta y este trabajo deja la pregunta empíricamente abierta para el modelo de 7 B.

Integración de SWE-bench Lite. El subconjunto Lite de SWE-bench contiene 300 issues con menor variabilidad de dependencias que el benchmark completo y puede acometerse con infraestructura Docker controlada. Su incorporación al banco extendería las conclusiones a tareas de ingeniería de software realistas. El proyecto incluye un módulo de scaffolding (`src/evaluation/swebench_loader.py`) preparado para esa extensión, con un caveat explícito sobre los requisitos de Docker que la corrida real exige.

Dynamic Task Decomposition. El modelo 3 de la propuesta —un agente Planner que descompone la tarea, asigna a roles y replanifica cuando una tarea falla— no se ha implementado en este trabajo. Su incorporación abriría preguntas nuevas sobre estabilidad, convergencia y coste adicional, especialmente sobre problemas donde el grafo fijo del pipeline secuencial no captura bien la estructura del problema.

Comparativa heterogénea de backends por rol. Asignar distintos modelos a distintos roles —por ejemplo un modelo pequeño y rápido como Code Reviewer y un modelo grande como Developer— permitiría explorar el espacio coste-calidad de forma más granular que el ajuste de `max_revisions`. La factory de clientes está preparada para esta extensión: basta con instanciar varios clientes con configuraciones diferentes y asignarlos a cada agente.

Evaluación de proyectos multi-fichero. La extensión natural del trabajo, en línea con DevBench y ClassEval, es evaluar la capacidad del pipeline para producir módulos completos o clases con varias funciones relacionadas. El `AgentState` actual modela un único `code_artifact` como cadena; la extensión exigiría modificar el estado y los prompts para gestionar múltiples artefactos correlacionados.

8.8. Reflexión final

La pregunta de la propuesta era si la especialización por roles aporta una ventaja medible frente a un LLM monolítico, y bajo qué condiciones. La respuesta del experimento es honesta y útil aunque no sea la que intuitivamente cabría esperar: con un modelo de 7 B y problemas que caben en un único prompt, **el monolítico domina**. La especialización por roles no se traduce en mejor `pass@1` sobre HumanEval, y el coste adicional del pipeline es de dos órdenes de magnitud.

Hay algo que el número de `pass@1` no captura y que vale la pena nombrar por separado. Trabajar con cinco agentes hablando entre sí a través de un estado tipado se siente, en la práctica, más cercano a coordinar un equipo humano que a invocar un modelo. Cuando una solución falla, se puede mirar qué dijo el Product Manager, qué interpretó el Arquitecto, qué implementó el Developer y qué señaló el Reviewer; cada paso queda explícito y auditable. Esa trazabilidad de la

decisión es lo único que el pipeline multi-agente sí ofrece y el baseline monolítico no, y puede acabar siendo el motivo por el que estas arquitecturas se imponen en producción aunque pierdan en benchmarks de funciones aisladas. Pero es una propiedad cualitativa y este TFG no la ha medido.

Tres piezas sobreviven al ciclo del trabajo independientemente del veredicto cuantitativo: el código del sistema multi-agente, el banco experimental resumible y el pipeline de análisis automático que regenera todo desde los CSV. Las líneas futuras de la sección 8.7 —escalado del modelo, integración de SWE-bench Lite, comparativa heterogénea de backends por rol— pueden empezar desde esa infraestructura sin reconstruirla, y son las extensiones donde el multi-agente tiene más probabilidades de demostrar el valor que aquí no ha podido demostrar.

Anexo A: Prompts de los agentes

Este anexo recoge, **verbatim**, los prompts de sistema y las plantillas de los prompts de usuario que reciben cada uno de los agentes implementados en el sistema. Su inclusión completa garantiza la reproducibilidad exigida por el OE7: cualquier investigador con acceso al mismo modelo y a estos prompts puede replicar las ejecuciones reportadas en el capítulo 7.

Los prompts se reproducen tal como aparecen en el código fuente del proyecto. Las referencias entre llaves de la forma `{state['campo']}` indican el campo del `AgentState` (capítulo 5, listado 5.1) cuyo valor se sustituye en tiempo de ejecución. Las cadenas literales `\n` representan saltos de línea reales en la cadena enviada al modelo.

A.1. Agente Baseline (configuración monolítica)

Fichero: `src/agents/baseline_agent.py`

Rol: `monolithic_solver`. Resuelve el problema en una única llamada al LLM, sin etapas intermedias.

Prompt de sistema:

You are an expert Python programmer. Given a coding problem, output ONLY valid Python code inside a single python\n...\n fenced block. Do NOT include any explanation, comments outside the code, or prose. The code must define the requested function and be self-contained.

Plantilla del prompt de usuario:

```
Problem:
{state['problem_statement']}

Function signature:
{state['function_signature']}

Write the complete Python implementation.
```

A.2. Agente Product Manager

Fichero: `src/agents/roles/product_manager.py`

Rol: `product_manager`. Primer nodo del pipeline secuencial. Produce el artefacto PRD a partir del enunciado del problema.

Prompt de sistema:

You are a software product manager. Given a coding problem, produce a structured PRD with exactly these sections: (1) Problem Summary (2) Acceptance Criteria as a numbered list of testable conditions (3) Edge Cases to handle (4) Out of scope Be precise. Output only the PRD, no preamble.

Plantilla del prompt de usuario:

```

Problem statement:
{state['problem_statement']}

Function signature: {state['function_signature']}

Known test cases:
{test_cases_block}

Write the PRD now.

```

donde `test_cases_block` es la concatenación con saltos de línea de los casos de prueba conocidos, o la cadena literal `"(none provided)"` cuando la lista está vacía.

A.3. Agente Arquitecto

Fichero: `src/agents/roles/architect.py`

Rol: `architect`. Segundo nodo del pipeline. Convierte el PRD en un documento de diseño técnico.

Prompt de sistema:

You are a software architect. Given a PRD and a function signature, produce a technical design document with: (1) Algorithm choice and justification (2) Data structures used (3) Step-by-step implementation plan in plain English (4) Known failure modes and how to handle them Output only the design document.

Plantilla del prompt de usuario:

```

PRD:
{state['prd']}

Function signature: {state['function_signature']}

Write the technical design document now.

```

A.4. Agente Developer

Fichero: `src/agents/roles/developer.py`

Rol: `developer`. Tercer nodo del pipeline. Implementa la función a partir del documento de diseño.

Prompt de sistema:

You are a Python developer. Given a design document and function signature, implement the function. Rules: (1) Output ONLY the Python function inside python fences (2) Match the exact function signature provided (3) Include inline comments for non-obvious logic (4) No imports unless strictly necessary (5) No test code in the output

Plantilla del prompt de usuario:

```

Problem statement:
{state['problem_statement']}

```



```
Function signature: {state['function_signature']}
```

```
Design document:
{state['design_doc']}
```

```
Implement the function now.
```

A.5. Agente Developer Reflexivo (configuración con self-reflection)

Fichero: `src/graph/self_reflection_graph.py`

Rol: `developer` (subclase `ReflectiveDeveloperAgent`). Idéntico al `Developer` estándar excepto en dos puntos: opera con temperatura 0.4 en lugar de 0.2, y antepone los comentarios de revisión al prompt cuando están presentes en el estado.

Prompt de sistema: idéntico al de la sección A.4.

Plantilla del prompt de usuario (primera iteración, sin feedback): idéntica a la de la sección A.4.

Plantilla del prompt de usuario (iteraciones posteriores, con feedback del Reviewer):

```
Previous review feedback:
{state['review_comments']}
```

```
Revise the implementation accordingly.
```

```
Problem statement:
{state['problem_statement']}
```

```
Function signature: {state['function_signature']}
```

```
Design document:
{state['design_doc']}
```

```
Implement the function now.
```

A.6. Agente QA Tester

Fichero: `src/agents/roles/qa_tester.py`

Rol: `qa_tester`. Cuarto nodo del pipeline. **No invoca al LLM.** Ejecuta los casos de prueba conocidos contra el artefacto del `Developer` en el sandbox de subprocesso (sección 5.5) y escribe el mapa de resultados al campo `test_results` del estado. No tiene prompts asociados.

A.7. Agente Code Reviewer

Fichero: `src/agents/roles/code_reviewer.py`

Rol: `code_reviewer`. Quinto y último nodo del pipeline secuencial. El veredicto `APPROVE` / `REQUEST_CHANGES` **no se le pide al LLM**: se deriva determinísticamente de `test_results` (`APPROVE` si todos los tests pasan, `REQUEST_CHANGES` en cualquier otro caso). El LLM produce únicamente la parte cualitativa de la revisión.

Prompt de sistema:

You are a senior Python code reviewer. You receive the implemented code, the test results (pass/fail per test), and the original design document. Produce a concise structured review of the CODE (not the design document): (1) Issues found: numbered list, each tagged [CRITICAL/MAJOR/MINOR] (2) Suggested fix for each issue If everything is fine, say so briefly in one sentence. Do NOT include any verdict line — the verdict is derived automatically from the test results.

Plantilla del prompt de usuario:

```
Code to review:
```python
{state['code_artifact']}
```

Test results:
{test_summary}

Original design document:
{state['design_doc']}

Write your review now.
```

donde `test_summary` se construye mediante la función `_format_test_results` con el siguiente formato:

```
Summary: {json del resumen de QA si está disponible}
[PASS] {primer caso de prueba}
[FAIL] {segundo caso de prueba}
...
```

A.8. Notas sobre los prompts

Tres observaciones cierran este anexo:

Estilo consistente. Todos los prompts de rol utilizan la misma forma imperativa breve, sin few-shot examples ni preámbulos largos. Esta elección minimiza el coste en tokens del prompt de sistema, que se incurre en cada llamada, y reduce la varianza atribuible a contexto adicional entre roles. La consecuencia operativa es que los prompts dependen fuertemente de la capacidad del modelo para seguir instrucciones en estructuras numeradas.

Veredictos deterministas. El veredicto del Reviewer y la lógica de verificación del QA Tester se derivan del estado del sistema, no de salidas del LLM. Esta decisión es deliberada (capítulo 5, sección 5.3.3) y se motiva por la fragilidad observada en modelos pequeños al pedirles emitir etiquetas literales en cadenas largas.

Patrón canónico de evaluación. La instrucción al Developer de no incluir importaciones salvo necesidad se complementa con el patrón `prompt + completion` del sandbox (sección 5.5): el código se ejecuta junto al prompt original del problema, que ya contiene importaciones y docstring. Esto evita que el modelo gaste tokens repitiendo encabezados sin perder información en la evaluación.

Anexo B: Decisiones técnicas

Este anexo presenta, en forma resumida, las decisiones técnicas que han guiado el diseño y la implementación del sistema a lo largo de los siete sprints que componen el desarrollo del TFG. Cada entrada documenta el contexto en que surgió la decisión, las alternativas consideradas y la justificación que llevó a la elección final.

El documento canónico, con cada entrada expandida con motivos, alternativas evaluadas y matices de implementación, es `doc/decisiones.md` en la raíz del repositorio. Este anexo es una versión sintética orientada a la lectura del tribunal evaluador.

B.1. Sprint S1 — Estado compartido como TypedDict

Se eligió `TypedDict` frente a `pydantic.BaseModel` para el estado compartido `AgentState`. La validación en tiempo de ejecución que aporta Pydantic no compensa el coste de serialización en cada transición del grafo, dado que cada campo del estado tiene un único agente productor y la condición de carrera lógica está descartada por diseño. `LangGraph` serializa el estado entre nodos mediante sus mecanismos internos de checkpointing y `TypedDict` es directamente compatible.

B.2. Sprint S2 — QA Tester sin LLM y veredicto determinista

El agente QA Tester se mantiene como nodo del grafo aunque no invoca al LLM. Su trabajo —ejecutar tests en el sandbox— es puramente determinista. Mantenerlo como nodo explícito hace visible la etapa de verificación al imprimir el grafo y preserva el contrato del `AgentState`. El veredicto del Code Reviewer (`APPROVE` / `REQUEST_CHANGES`) se deriva deterministamente de `test_results`, no se pide al LLM, para evitar la fragilidad observada en modelos pequeños al emitir etiquetas literales dentro de texto libre.

B.3. Sprint S3 — Temperatura asimétrica en self-reflection

El Developer reflexivo de la configuración con self-reflection opera con temperatura 0.4, frente a 0.2 del resto de agentes. La asimetría es deliberada: una diversidad mayor en la generación facilita la convergencia del ciclo Reviewer → Developer al permitir explorar soluciones alternativas en lugar de repetir variantes de la primera propuesta.

B.4. Sprint S4 — Runner resumible y atómico

El runner principal persiste el estado del experimento mediante escritura atómica del fichero `progress.json` (escritura a `.json.tmp` + `rename`). Cada ejecución individual se envuelve en `try/except` amplio que registra el error en CSV con el campo `error` poblado, sin abortar la corrida global. Esta política, contraria al fail-fast habitual, es necesaria para corridas de miles de ejecuciones donde un fallo aislado no debe invalidar el resto.

B.5. Sprint S5 — Inferencia muestral, no determinista

Las llamadas al LLM se realizan sin fijar la `seed` de inferencia de Ollama. El estimador `pass@k` de Chen et al. (2021) asume `n` generaciones independientes por problema y forzar determinismo invalidaría su definición. Las tres semillas del experimento indexan **réplicas del experimento**, no

semillas del modelo: cada réplica constituye una muestra independiente de la distribución de salidas del pipeline.

B.6. Sprint S6 — Backend local (Ollama) como referencia, Cerebras evaluado y descartado

El backend principal de inferencia es Ollama local con `qwen2.5-coder:7b-instruct-q4_K_M`. La elección equilibra reproducibilidad —cualquier investigador con el binario público puede replicar la corrida— con eliminación del coste marginal por inferencia.

La alternativa de Cerebras Inference (Qwen-3 235B y Llama 3.1 8B en tier gratuito) se evaluó empíricamente durante S6 y se descartó: bajo la carga real de un pipeline con 5-6 llamadas concatenadas por problema, el rate limit del tier público ($\approx 1\text{-}2$ req/min sostenido tras errores 429) hace inviable cubrir la matriz experimental en plazo. Se probaron pacing artificial de 2,5 s entre llamadas y workers en {1, 3} sin que el límite cediera. El detalle cuantitativo de las pruebas está en la entrada D1 del documento canónico `doc/decisiones.md`.

La variable `LLM_BACKEND` permite alternar a Cerebras desde código sin tocar el pipeline, lo que mantiene abierta la opción para verificaciones cruzadas sobre subconjuntos pequeños con un plan de pago futuro.

B.7. Sprint S7 — Ablaciones, análisis y métrica de adherencia

Tres compromisos de la propuesta no estaban operacionalizados al cerrar S6: el análisis de contribución por rol, una métrica que cuantifique la reducción de alucinaciones del protocolo estructurado y un análisis sistemático coste-vs-calidad. S7 añade las tres variantes de ablación (`no_pm`, `no_architect`, `no_reviewer`), un pipeline post-hoc de figuras y tablas con intervalos de confianza bootstrap y test pareado de McNemar, y una métrica de adherencia estructural que cuenta ejecuciones con fence malformado. La alternativa de ampliar el banco con SWE-bench se descarta por presupuesto de cómputo, recogido en cap 3.4 y cap 8.5.

B.8. Sprint S8 — Recorte de scope del barrido principal

Tras 18 horas de corrida con el backend local sobre el MacBook Air M2, el ritmo observado sostenido ($\approx 5,3$ ejecuciones/hora para secuencial, $\approx 12\text{-}16$ min/ejecución para self-reflection) proyectaba la matriz completa de 5 configuraciones (baseline + sequential + SR $\times 3$) sobre 164 problemas $\times$ 3 semillas en aproximadamente 20 días, y en 30 días añadiendo las tres ablaciones. Con la entrega institucional el 8 de junio de 2026 y la imposibilidad de sostener al equipo bajo carga térmica continua durante semanas (el MacBook Air M2 carece de ventilación activa), el plazo no permitía la matriz completa.

Decisión. Mantener Ollama local como backend y recortar el barrido principal a las **tres configuraciones más informativas** para las hipótesis del estudio:

1. `baseline` (configuración 1, monolítica)
2. `sequential` (configuración 2, pipeline de 5 roles)
3. `self_reflection_r1` (configuración 3 con `max_revisions = 1`)

Las variantes `self_reflection_r2`, `self_reflection_r3` y las tres ablaciones (`no_pm`, `no_architect`, `no_reviewer`) quedan implementadas, testadas y disponibles en el runner como segunda pasada, activables mediante variables de entorno (`ENABLE_SR_R2`, `ENABLE_SR_R3`, `ABLATION_SUBSET_SIZE`) sin modificar código.

Justificación académica. Las tres configuraciones conservadas cubren las tres hipótesis del estudio: H1 (especialización) se contrasta con baseline vs sequential; H2 (auto-revisión) con sequential vs SR_r1; H3 (trade-off coste-calidad) con los tres puntos en la frontera de Pareto. SR_r2 y SR_r3 son experimentos de hiperparámetro que no son necesarios para contrastar H2. Las ablaciones quedan para una segunda corrida.

Coste. Cero coste económico (Ollama local). El equipo opera bajo throttling térmico moderado durante ~9 días, reversible y dentro de tolerancias del fabricante. La cardinalidad efectiva del experimento queda en $3 \times 164 \times 3 = 1\,476$ ejecuciones, todas reportadas en el capítulo 7.

El detalle completo de esta decisión, incluyendo las pruebas empíricas de Cerebras descartadas, está en la entrada S8 de `doc/decisiones.md`.

B.9. Notas sobre el feedback del tutor

Dos observaciones del tutor han condicionado la forma final del documento: la bibliografía no se duplica al final de cada capítulo; toda referencia se resuelve contra el Anexo de Bibliografía único, recogido al cierre del documento. La densidad visual se ha incrementado significativamente a partir del capítulo 5, con tablas y figuras explícitamente numeradas en cada sección que introduce un concepto técnico o un resultado numérico.

Para el detalle completo de cada decisión, con los motivos descartados y las implicaciones operativas, consultar `doc/decisiones.md`.

Anexo C: Comandos de reproducción

Este anexo consolida en un único listado los comandos necesarios para reproducir el experimento, regenerar las figuras y tablas y verificar la integridad del sistema. La fuente canónica de cada comando es el `README.md` del repositorio; este anexo es la versión completa de referencia para el tribunal evaluador.

C.1. Preparación del entorno

```
git clone https://github.com/nicotimoneda/TFG_MultiAgente.git
cd TFG_MultiAgente
python -m venv .venv
source .venv/bin/activate
pip install -e "[experiments]"
```

Las dependencias declaradas en `pyproject.toml` son LangGraph (≥ 0.2), LangChain (≥ 0.3), `langchain-openai` (≥ 0.2), HuggingFace `datasets` (≥ 2.20), SciPy (≥ 1.13), NumPy (≥ 1.26), Pandas (≥ 2.2) y, dentro del extra `experiments`, `rich`, `tqdm` y `matplotlib`. Pytest se instala adicionalmente para ejecutar la suite de pruebas.

C.2. Backend de inferencia

Configuración local recomendada (utilizada en la corrida principal):

```
ollama serve # en una terminal
ollama pull qwen2.5-coder:7b-instruct-q4_K_M
```

Configuración remota para verificaciones cruzadas:

```
export CEREBRAS_API_KEY="..." # cuenta gratuita en cloud.cerebras.ai
export LLM_BACKEND=cerebras
```

C.3. Verificación rápida

Antes de lanzar la corrida completa, el smoke test ejecuta 10 problemas sobre las 5 configuraciones principales en aproximadamente 15 minutos:

```
LLM_BACKEND=ollama python experiments/quick_check.py
```

La suite de pruebas unitarias se ejecuta con:

```
LLM_BACKEND=ollama pytest -q
```

Cubre el sandbox de subprocesso (timeouts, builtins bloqueados, errores de sintaxis), las funciones de `pass@k` y la topología de las tres variantes de ablación.

C.4. Corrida completa

Recomendado en una sesión separable (terminal con `nohup` o `tmux`):

```
LLM_BACKEND=ollama nohup python experiments/run_experiments.py \
  --model qwen2.5-coder:7b-instruct-q4_K_M \
  --benchmarks humaneval \
  --configs
baseline,sequential,self_reflection_r1,self_reflection_r2,self_reflection_r3 \
  > experiments/logs/run.out 2>&1 &
```

Las ablaciones se incorporan a la corrida añadiendo configs:

```
LLM_BACKEND=ollama python experiments/run_experiments.py \
  --model qwen2.5-coder:7b-instruct-q4_K_M \
  --configs ablation_no_pm,ablation_no_architect,ablation_no_reviewer
```

MBPP se incluye con:

```
LLM_BACKEND=ollama python experiments/run_experiments.py \
  --model qwen2.5-coder:7b-instruct-q4_K_M \
  --benchmarks humaneval,mbpp
```

Todas las invocaciones son resumibles: si la corrida se interrumpe, basta con volver a ejecutar el mismo comando para que retome donde quedó. Las ejecuciones ya completadas se omiten.

C.5. Seguimiento en vivo

```
python experiments/dashboard.py      # tabla interactiva con Rich
cat experiments/results/progress.json # JSON puntual
tail -f experiments/logs/run.out     # log textual
```

C.6. Análisis y regeneración de artefactos

Tras (o durante) la corrida:

```
python experiments/analyze_results.py # figuras + tablas + McNemar pareado
python experiments/adherence_metric.py # adherencia estructural
```

Productos en `figures/` y `doc/tables/`:

| Fichero | Contenido |
|--|---|
| figures/arquitectura_sistema.png | Diagrama del sistema |
| figures/graph_baseline.png | Topología del grafo baseline |
| figures/graph_sequential.png | Topología del grafo secuencial |
| figures/graph_self_reflection_r1.png (r2, r3) | Grafos con bucle de revisión |
| figures/graph_ablation_no_pm.png (no_architect, no_reviewer) | Grafos de las ablaciones |
| figures/pass_at_1.png | Barras de pass@1 con IC 95% bootstrap |
| figures/cost_quality_pareto.png | Frontera de Pareto coste–calidad |
| figures/latency_box.png | Distribución de latencias por configuración |
| figures/revision_distribution.png | Uso del ciclo en self-reflection |
| doc/tables/summary.md | Tabla principal de resultados (Markdown) |
| doc/tables/summary.tex | Versión LaTeX |
| doc/tables/per_benchmark.md | Cross-tab configuración × benchmark |
| doc/tables/problem_difficulty.md | Top-20 problemas con mayor desacuerdo |
| doc/tables/pairwise_mcnemar.md | Tests pareados de McNemar |
| doc/tables/adherence.md | Adherencia estructural por configuración |

C.7. Acceso a los benchmarks adicionales

Los cargadores de ClassEval y SWE-bench Lite están disponibles y cachean localmente:

```
python -c "from src.evaluation.classeval_loader import load_classeval;
load_classeval()"
python -c "from src.evaluation.swebench_loader import load_swebench_lite;
load_swebench_lite()"
```

La ejecución real de SWE-bench requiere el harness Docker oficial, referenciado en `src/evaluation/swebench_loader.py`. La ejecución de ClassEval requiere adaptar los prompts del Developer a generación de clases (sección 8.7).

C.8. Hardware de referencia

Las latencias y el coste reportados en el capítulo 7 se obtienen sobre un MacBook Air con Apple M2, 8 núcleos de CPU, 10 núcleos de GPU y 16 GB de memoria unificada. Otros equipos pueden reproducir las tasas de éxito (pass@1, adherencia) pero las latencias absolutas variarán.

Anexo E: Aspectos éticos, legales y de sostenibilidad

Este anexo recoge las consideraciones éticas, legales y de sostenibilidad relativas al trabajo. La extensión es deliberadamente breve y operativa: identifica los aspectos relevantes, documenta las decisiones tomadas y señala los marcos normativos aplicables. No sustituye un dictamen jurídico ni una revisión ética formal por parte de un comité institucional.

E.1. Naturaleza de los datos utilizados

El estudio no procesa datos personales ni datos sensibles. El benchmark utilizado en la corrida principal —HumanEval, versión EvalPlus— está compuesto por problemas sintéticos de programación, redactados por sus autores para evaluar modelos. No contiene información identificable, ni metadatos personales, ni texto extraído de comunicaciones privadas. HumanEval se publica bajo licencia MIT por OpenAI; el cache de MBPP que el runner mantiene para una eventual segunda pasada está publicado por Google Research bajo Apache 2.0. La inclusión de ambos benchmarks en este trabajo respeta los términos de sus respectivas licencias.

En consecuencia, este TFG queda fuera del alcance del Reglamento General de Protección de Datos (RGPD, Reglamento UE 2016/679) por ausencia de tratamiento de datos personales. No se ha requerido por tanto la intervención del Delegado de Protección de Datos de la UAX.

E.2. Modelos utilizados y sus licencias

El modelo principal utilizado en la corrida experimental es **Qwen 2.5 Coder 7B Instruct** (Hui et al., 2024), publicado por Alibaba Cloud bajo licencia Apache 2.0. La cuantización Q4_K_M y la distribución a través de Ollama mantienen las condiciones de la licencia original.

El proyecto soporta como backend alternativo **Cerebras Inference**, que ofrece acceso gratuito a varios modelos open-weight (entre ellos las familias Qwen y Llama de Meta) bajo sus términos de servicio públicos. Las pruebas exploratorias se realizaron exclusivamente dentro del tier gratuito y se descartaron por incompatibilidad del rate limit con la cardinalidad de la matriz (entrada S8 del Anexo B), sin recurrir a ningún plan de pago. La opción queda disponible en el factory de clientes para subsets puntuales en futuras extensiones.

Todos los modelos utilizados son **open-weight**: los pesos están publicados y son auditables. Esta elección se ajusta a las recomendaciones de transparencia para investigación académica recogidas en el Código de Buenas Prácticas Científicas de la UAX y en la Recomendación de la UNESCO sobre la Ética de la IA (2021).

E.3. Generación automática de código y responsabilidad

El sistema produce código fuente Python. Aunque el alcance de este TFG se limita a problemas algorítmicos de benchmark, la tecnología subyacente es la misma que se está integrando en herramientas de desarrollo profesionales. Procede señalar tres consideraciones:

Autoría y trazabilidad. Todo código generado por el sistema en los experimentos queda registrado en los CSV de resultados junto a los artefactos intermedios (PRD, design doc, review). En un

hipotético despliegue en un entorno profesional, esa trazabilidad permite auditar la procedencia de cada fragmento de código y distinguirla de los aportes humanos.

Verificación funcional como salvaguarda. El sandbox de ejecución y el agente QA Tester son piezas centrales del diseño, no añadidos opcionales. Su función es precisamente garantizar que ningún código generado se acepta sin verificación contra pruebas objetivas, lo que mitiga la categoría más común de error en la generación automática: la apariencia de corrección sin verificación real (Chen et al., 2021).

Limitaciones reconocidas. El estudio se limita a generación de funciones aisladas. No aborda generación de código en contextos de seguridad crítica, ni código que interactúe con datos personales en tiempo de ejecución, ni código que se despliegue sin revisión humana. Estas son extensiones reconocidas como fuera de alcance en el capítulo 8.

E.4. Marco legal: Reglamento Europeo de IA

El Reglamento Europeo de Inteligencia Artificial (Reglamento (UE) 2024/1689, conocido como **AI Act**), aprobado el 13 de junio de 2024 y con entrada en vigor escalonada hasta 2026-2027, establece un marco regulatorio para los sistemas de IA en función de su nivel de riesgo. El sistema multi-agente desarrollado en este TFG se clasifica como **sistema de IA de propósito general aplicado a la generación de código**, categoría que el AI Act trata bajo los artículos relativos a modelos fundacionales (general-purpose AI, GPAI).

Las obligaciones aplicables en este nivel son:

- **Transparencia sobre el sistema.** El presente documento, junto al repositorio público y los prompts reproducidos en el Anexo A, cumplen el requisito de documentación técnica que el AI Act exige a los desarrolladores de sistemas basados en GPAI.
- **Trazabilidad de las salidas generadas.** Los CSV de experimentación registran qué modelo y qué configuración produjo cada artefacto, lo que satisface el principio de trazabilidad recogido en el artículo 53 del AI Act.
- **Evaluación de riesgo de uso.** El sistema se limita a generación de código en un entorno controlado de evaluación académica; no procesa interacciones con usuarios finales ni toma decisiones automatizadas con efecto jurídico, por lo que no se sitúa en las categorías de alto riesgo definidas en el Anexo III del AI Act.

No se utilizan en este TFG sistemas clasificados como de riesgo inaceptable (artículo 5) ni de alto riesgo (Anexo III), de manera que las obligaciones materiales del AI Act se limitan a las de transparencia y documentación recogidas más arriba.

E.5. Sostenibilidad y huella de cómputo

La corrida experimental se ejecuta sobre un único MacBook Air con chip Apple M2, un equipo de consumo energético reducido (potencia máxima nominal de 30 W, típicamente 15-20 W durante inferencia con modelos cuantizados sobre Apple Silicon).

El cálculo del consumo se hace directamente a partir de la latencia acumulada que el runner registra para cada ejecución, no a partir de una estimación genérica. La tabla 7.2 (capítulo 7, corrida cerrada) reporta los siguientes tiempos de cómputo bajo carga:

| Configuración | n | Latencia media (s) | Tiempo bajo carga (h) |
|-----------------------|--------------|--------------------|-----------------------|
| Baseline | 492 | 5,13 | 0,70 |
| Sequential | 492 | 396,35 | 54,17 |
| SR ($r=1$) | 492 | 411,59 | 56,25 |
| Total agregado | 1 476 | — | ≈ 111,1 |

A esto se suma el overhead del proceso runner, del servidor Ollama en idle entre ejecuciones y de la sesión interactiva de monitorización (estimado en ~5 W durante el resto del wall-clock de los ~5 días que dura la corrida, ≈ 9 h adicionales a media potencia sobre las ~120 h totales del ventanal de ejecución).

Con potencia media bajo carga de 18 W e idle de 5 W, el consumo total estimado es:

- Inferencia bajo carga: $111,1 \text{ h} \times 18 \text{ W} \approx \mathbf{2,00 \text{ kWh}}$.
- Overhead en idle: $9 \text{ h} \times 5 \text{ W} \approx \mathbf{0,05 \text{ kWh}}$.
- **Total: ≈ 2,05 kWh.**

Con el mix eléctrico español de 2024 (≈ 0,16 kg CO₂eq/kWh, según el factor de emisión publicado por Red Eléctrica de España), la huella de carbono del experimento principal es de ≈ **0,33 kg CO₂eq**, equivalente al orden de magnitud de un ciclo medio de lavadora doméstica.

La elección deliberada de un modelo cuantizado de 7 B parámetros y de hardware local de bajo consumo —frente a alternativas como modelos de frontera servidos en datacenters comerciales, cuyo coste energético por inferencia es uno o dos órdenes de magnitud superior— sitúa este TFG en el rango bajo del espectro de impacto ambiental para investigación en LLMs. La decisión de scope documentada en el capítulo 3 y en el Anexo B (entrada S8), tomada por restricciones de plazo y hardware, tiene como efecto colateral una huella de cómputo modesta y trazable.

E.6. Reproducibilidad como obligación ética

La publicación del código fuente bajo licencia MIT, la inclusión verbatim de los prompts en el Anexo A, la documentación de cada decisión técnica en el Anexo B, y el listado de comandos de reproducción en el Anexo C responden no sólo a un criterio metodológico sino a una obligación ética: cualquier afirmación empírica de esta memoria es **falsable por terceros con acceso al equipo equivalente y al modelo público**. Esto es coherente con los principios de la Recomendación de la UNESCO sobre la Ética de la IA (2021), que sitúa la verificabilidad como uno de los pilares del desarrollo responsable de sistemas de IA.

E.7. Conflictos de interés

El autor declara no tener conflictos de interés económicos, ni relaciones contractuales con ninguno de los proveedores de modelos o infraestructura mencionados en este trabajo (Alibaba Cloud, Meta, Cerebras, Ollama, LangChain Inc., HuggingFace). Todas las referencias a estos proveedores se realizan a título exclusivamente técnico.

Bibliografía

Lista única alfabética por primer autor (estilo APA 7). Por indicación del tutor, los capítulos no incluyen bibliografía propia: todas las citas en el cuerpo del documento se resuelven contra esta lista.

- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q. y Sutton, C. (2021). Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732*.
- Bratman, M. E. (1987). *Intention, Plans, and Practical Reason*. Harvard University Press.
- Chen, L., Davis, J. Q., Hanin, B., Bailis, P., Stoica, I., Zaharia, M. y Zou, J. (2024). Are More LLM Calls All You Need? Towards Scaling Laws of Compound Inference Systems. En *International Conference on Machine Learning (ICML)*. arXiv:2403.02419.
- Chen, M. et al. (2021). Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*.
- Cohen, P. R. y Levesque, H. J. (1990). Intention is choice with commitment. *Artificial Intelligence*, 42(2-3), 213-261.
- Du, X., Liu, M., Wang, K., Wang, H., Liu, J., Chen, Y., Feng, J., Sha, C., Peng, X. y Lou, Y. (2023). ClassEval: A Manually-Crafted Benchmark for Evaluating LLMs on Class-level Code Generation. *arXiv preprint arXiv:2308.01861*.
- Hendrycks, D., Basart, S., Kadavath, S., Mazeika, M., Arora, A., Guo, E., Burns, C., Puranik, S., He, H., Song, D. y Steinhardt, J. (2021). Measuring Coding Challenge Competence with APPS. *NeurIPS Datasets and Benchmarks Track*. arXiv:2105.09938.
- Hong, S. et al. (2024). MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. *International Conference on Learning Representations*.
- Huang, D., Zhang, J. M., Luck, M., Bu, Q., Qing, Y. y Cui, H. (2024). AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation. *arXiv preprint arXiv:2312.13010*.
- Hui, B. et al. (2024). Qwen2.5-Coder Technical Report. *arXiv preprint arXiv:2409.12186*.
- Jiménez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O. y Narasimhan, K. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *International Conference on Learning Representations*.
- LangChain Inc. (2024). *LangGraph: Build Stateful, Multi-Actor Applications with LLMs*. <https://github.com/langchain-ai/langgraph>
- Li, G., Hammoud, H., Itani, H., Khizbullin, D. y Ghanem, B. (2023). CAMEL: Communicating Agents for "Mind" Exploration of Large Language Model Society. *Advances in Neural Information Processing Systems*, 36.
- Li, Y. et al. (2022). Competition-Level Code Generation with AlphaCode. *Science*, 378(6624), 1092-1097.

- Liu, J., Xia, C. S., Wang, Y. y Zhang, L. (2023). Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code. *Advances in Neural Information Processing Systems*, 36.
- Olausson, T. X., Inala, J. P., Wang, C., Gao, J. y Solar-Lezama, A. (2024). Is Self-Repair a Silver Bullet for Code Generation? En *International Conference on Learning Representations (ICLR)*. arXiv:2306.09896.
- Pereira-Vale, A., Fernandez, E. B., Astudillo, H. y Márquez, G. (2024). Patterns for Multi-Agent Systems: A Systematic Mapping Study. *ACM Computing Surveys*.
- Qian, C., Cong, X., Yang, C., Chen, W., Su, Y., Xu, J., Liu, Z. y Sun, M. (2024). Communicative Agents for Software Development. En *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pp. 15174-15186.
- Rao, A. S. y Georgeff, M. P. (1995). BDI Agents: From Theory to Practice. En *Proceedings of the First International Conference on Multiagent Systems (ICMAS-95)*, pp. 312-319. AAAI Press.
- Rozière, B. et al. (2023). Code Llama: Open Foundation Models for Code. *arXiv preprint arXiv:2308.12950*.
- Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N. y Scialom, T. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. *Advances in Neural Information Processing Systems*, 36.
- Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K. y Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. *Advances in Neural Information Processing Systems*, 36.
- Stone, P. y Veloso, M. (2000). Multiagent Systems: A Survey from a Machine Learning Perspective. *Autonomous Robots*, 8(3), 345-383.
- Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z. y Wen, J. (2024). A Survey on Large Language Model based Autonomous Agents. *Frontiers of Computer Science*, 18(6), 186345.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. y Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in Neural Information Processing Systems*, 35, 24824-24837.
- Wooldridge, M. (2009). *An Introduction to Multiagent Systems* (2.^a ed.). Wiley.
- Wooldridge, M. y Jennings, N. R. (1995). Intelligent Agents: Theory and Practice. *The Knowledge Engineering Review*, 10(2), 115-152.
- Wu, Q. et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv preprint arXiv:2308.08155*.
- Xi, Z. et al. (2023). The Rise and Potential of Large Language Model Based Agents: A Survey. *arXiv preprint arXiv:2309.07864*.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. y Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations*.